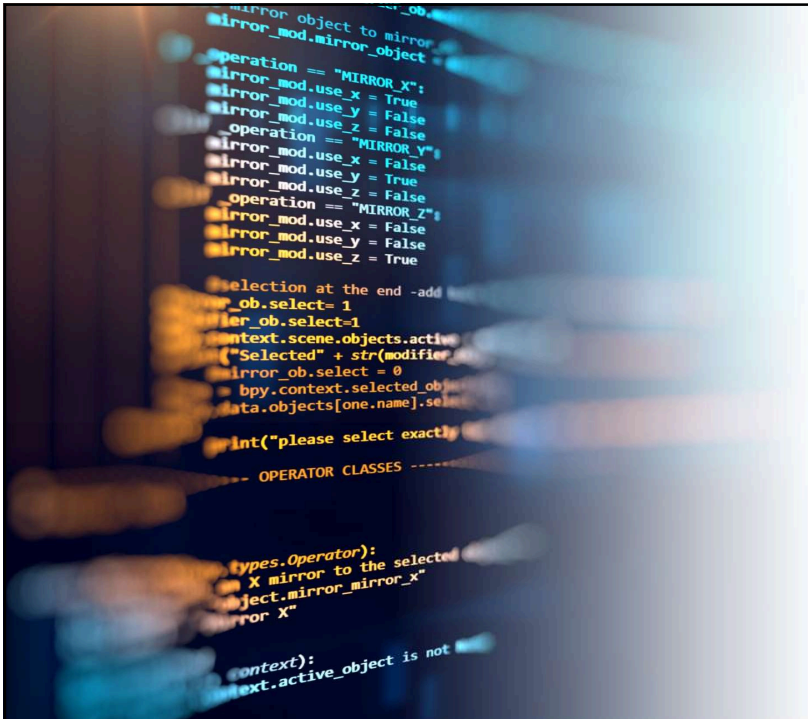




Python Programing

김민영(kmyco@deu.ac.kr)

1

- 파이썬 Python은 귀도 반 로섬 Guido van Rossum이 1991년에 개발한 대화형 프로그래밍 언어인데 최근 많은 인기를 얻고 있다.
- 가장 큰 이유는 생산성이 뛰어나기 때문이다.
- 파이썬을 이용하면 간결하면서도 효율적인 프로그램을 빠르게 작성할 수 있다.
- 파이썬은 오픈 소스이어서 무료이고 패키지들이 계속 추가되고 있어서 매일 진화하는 언어이기도 하다.



2

[스테이블 디퓨전]

1.설치파일 다운

<https://github.com/AUTOMATIC1111/stable-diffusion-webui/releases/tag/v1.0.0-pre>

위의 사이트에서 다운로드 받고 압축해제
먼저 update.bat 실행
실행하려면 run.bat

2.다른모델 다운로드 및 적용

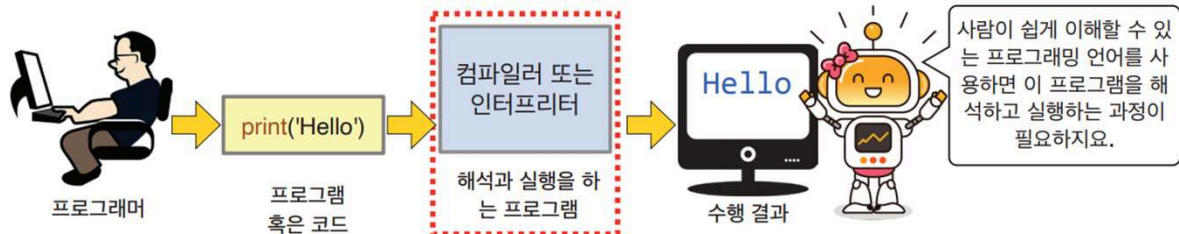
<https://civitai.com/models/43331/majicmix-realistic?modelVersionId=176425>

다운받은 파일 .\webui\models\Stable-diffusion 에 다운로드 받은 모델 복사

** 추천 영상 : <https://www.youtube.com/watch?v=ZBAmhZ8ekLM>

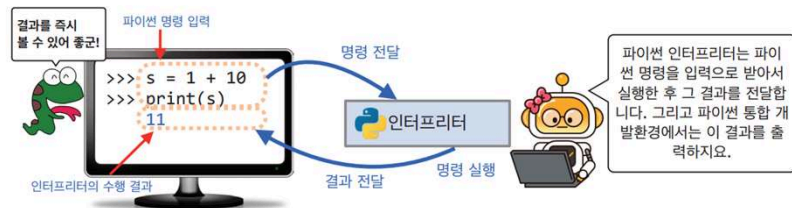


프로그래밍과 실행과정



5

- 파이썬이 **인터프리터 언어** *interpreted language*이다.
- 파이썬은 무엇보다도 초보자의 프로그래밍 입문에 적합한 언어이다. 그 이유는 프로그래머가 한 줄의 문장을 입력하고 엔터키를 치면 명령 해석기인 **인터프리터** *interpreter*가 이것을 바로 실행한다.
- 파이썬 프로그래머는 자신이 작성한 문장의 결과를 입력 즉시 볼 수 있다.
- 입문자도 쉽게 코딩을 익힐 수 있다.



6

파이썬 프로그래밍 모드

- **대화식 모드**는 파이썬 번역기를 실행한 후 >>>라는 프롬프트에서 커서가 깜박일 때 파이썬 명령을 입력하고 엔터키를 치면 번역기가 한 줄 또는 그 이상의 코드를 실행한다.
- **스크립트 모드**는 새로운 파일을 하나 생성하고 파이썬 명령을 입력하고 나서 이 명령으로 이루어진 코드를 저장해야 한다. 이 때 파일 이름이 file_name.py라고 하면 IDLE에서 "Run"이라는 메뉴를 선택해서 이 코드를 실행시킬 수 있다.

	대화식 모드	스크립트 모드(file_name.py 예시)
예시	<pre>>>> print('당신의 이름은 :') 당신의 이름은 : >>> name = '홍길동' >>> print(name) 홍길동</pre>	<pre>print("당신의 이름은 :") name = '홍길동' print(name)</pre>
실행	파이썬 명령어를 입력하고 엔터키를 입력하면 번역기가 코드를 실행함.	1. 파일을 만들고 저장하여 IDLE에서 "Run" 메뉴를 선택함. 2. python file_name.py 명령을 내림.
장단점	간단한 코드와 문장을 테스트하기에 편리함. 작성한 명령어를 저장할 수 없음.	비교적 긴 파이썬 명령어를 파일로 저장하고 필요할 경우 불러내어 실행함. 짧은 코드를 테스트하기 위해서 번거로운 파일 생성 작업을 해야 함.

7

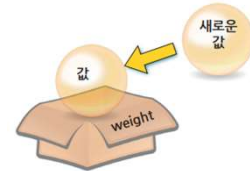
변수



8

데이터를 저장하는 공간 : 변수

- **변수**는 컴퓨터의 **메모리**memory 공간에 이름을 붙이는 것으로 우리는 여기에 정수, 실수, 문자열 등의 자료값을 저장할 수 있다.
- 저장된 것을 꺼내려면 변수의 이름을 적어주면 된다.
- 상자에 저장된 값은 수시로 바뀔 수 있다.
- 그래서 변수(변하는 수)라는 이름이 붙은 것이다.
- 이 때 사용된 '수'는 수치값이라는 의미가 아닌 **데이터**data로 이해하는 것이 더 정확하다.



9

데이터를 저장하는 공간 : 변수

- 예를 들어서 자신의 몸무게를 `weight`라는 이름의 변수에 저장한다고 하자. 이를 위해서는 다음과 같은 방법을 사용한다.

```
>>> weight = 78.2
```

- 파이썬 내부에 `weight`라는 이름의 변수가 생성되고 여기에 78.2가 저장된다.
- 위의 코드에 있는 '=' 기호는 '같다'는 등호의 의미가 아니라 '**오른쪽의 값을 왼쪽의 weight라는 이름의 변수에 저장하라**'는 의미이다. 이 연산자 = 를 **할당연산자** 혹은 **대입연산자**라고 한다. (Assignment operator)

10

변수의 이름을 지을 때 알아야 할 것

- 변수의 이름은 프로그래머가 마음대로 지을 수 있지만 몇 가지의 규칙을 지켜야 한다.
- 변수의 이름은 **식별자**identifier의 일종이다.
- "홍길동", "김철수"등의 이름이 사람을 구별하듯이 식별자는 변수들을 구별하는 역할을 한다.
- 식별자는 다음과 같은 규칙에 따라 만들어야 한다.

- 식별자는 문자와 숫자, 밑줄 문자(_)로 이루어지며 밑줄 문자 이외의 특수 문자를 사용할 수 없다.
- 식별자의 첫 글자는 숫자로 시작할 수 없다. 또한 중간에 공백을 가질 수 없다.
- 대문자와 소문자는 구별된다. 따라서 변수 `index`와 `INDEX`은 서로 다른 변수이다.
- 파이썬의 예약어(키워드)는 식별자로 사용할 수 없다.

11

변수의 이름을 지을 때 알아야 할 것 (계속)

- 우리는 변수의 이름을 마음대로 지을 수 있지만 파이썬이 내부적으로 사용하는 **예약어**는 **변수의 이름으로 사용할 수 없다**.
 - 예약어를 사용하여 변수를 생성하면 오류가 발생한다.
- 파이썬의 **예약어**는 다음과 같다.

False	class	return	is	finally	None
if	for	lambda	continue	True	def
from	while	nonlocal	and	del	global
not	with	as	elif	try	or
yield	assert	else	import	pass	break
except	in	raise			

12

파이썬 자료형

- 프로그램에서 사용할 수 있는 데이터의 종류를 **자료형** data type 이라고 한다.
- 파이썬에서 사용할 수 있는 가장 단순한 기본 자료형은 4가지 종류가 있다.

- 첫 번째는 1이나 2와 같은 **정수** integer이다.
- 두 번째는 3.14와 같은 **실수** floating-point이다.
- 세 번째는 'Hello World!'와 같은 **문자열** string이다.
- 마지막으로 네 번째는 참과 거짓을 나타내는 **부울형** bool

자료형	예
정수형(int)	..., -3, -2, -1, 0, 1, 2, 3, ...
실수형(float)	3.14, 4.28, 0.01, 123.432
문자열(str)	'Hello World', '123', "Hi"
부울형(bool)	True, False
리스트(list)	['ABC', 'DEF'], [3, 4, 5, 6]

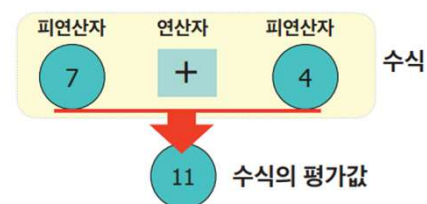
13

연산자



14

- 수식 **expression**이란 피연산자들과 연산자의 조합이라고 할 수 있다.
- 이때 **연산자operator**는 어떤 연산을 나타내는 기호를 의미하며, **피연산자operand**는 연산의 대상이 되는 숫자나 변수를 의미한다.
- 이때 연산자와 피연산자 사이에 공백을 삽입하는 것이 파이썬의 코딩 관습에 더 적합한 방식이다. 즉 **a*b**보다 **a * b**로 코딩하는 것이 좋다.



15

파이썬의 기본 연산자

연산자	기호	사용 예	결과값
덧셈	+	7 + 4	11
뺄셈	-	7 - 4	3
곱셈	*	7 * 4	28
지수	**	7 ** 2	$7^2 = 49$
나눗셈(정수 나눗셈의 몫)	//	7 // 4	1
나눗셈(실수 나눗셈)	/	7 / 4	1.75
나머지	%	7 % 4	3

16

- 피타고라스 정리에 의해 직각삼각형의 빗변의 길이 c 의 제곱은 밑변 길이 a 의 제곱과 높이 b 의 제곱을 더한 것과 같다는 것을 잘 알고 있을 것이다.
- 그러면 밑변과 높이를 입력하여 빗변을 계산하는 프로그램을 작성하려면 어떻게 해야 할까? 식을 조금만 변경하면 다음과 같다는 것을 알 수 있다.

$$c = \sqrt{a^2 + b^2} = (a^2 + b^2)^{\frac{1}{2}}$$

17

복합할당 연산자

연산자	사용 방법	의미
<code>+=</code>	<code>i += 10</code>	<code>i = i + 10</code>
<code>-=</code>	<code>i -= 10</code>	<code>i = i - 10</code>
<code>*=</code>	<code>i *= 10</code>	<code>i = i * 10</code>
<code>/=</code>	<code>i /= 10</code>	<code>i = i / 10</code>
<code>//=</code>	<code>i //= 10</code>	<code>i = i // 10</code>
<code>%=</code>	<code>i %= 10</code>	<code>i = i % 10</code>
<code>**=</code>	<code>i **= 10</code>	<code>i = i ** 10</code>

18

비교 연산자

- 다음은 **비교 연산자** `comparison operator`에 대해 알아보자.
- '크다' 혹은 '작다'와 같은 비교 연산은 수치 데이터를 담고 있는 두 개의 피연산자를 대상으로 크기 관계를 살펴본다.
- 그리고 그 결과인 True 혹은 False를 반환한다.
- 이렇게 True나 False의 값을 갖는 자료형을 **부울** `bool`형이라고 한다.

19

비교 연산자 (계속)

연산자	설명	a = 100 b = 200
==	두 피연산자의 값이 같으면 True를 반환	a == b는 False
!=	두 피연산자의 값이 다르면 True를 반환	a != b는 True
>	왼쪽 피연산자가 오른쪽 피연산자보다 클 때 True를 반환	a > b는 False
<	왼쪽 피연산자가 오른쪽 피연산자보다 작을 때 True를 반환	a < b는 True
>=	왼쪽 피연산자가 오른쪽 피연산자보다 크거나 같을 때 True 반환	a >= b는 False
<=	왼쪽 피연산자가 오른쪽 피연산자보다 작거나 같을 때 True 반환	a <= b는 True

20

논리연산자

- 파이썬은 and, or, not 연산자를 지원하는데 이러한 연산자를 **논리 연산자**logical operator라고 한다.
- 이 연산은 논리 and, or, not 연산을 통해 True나 False 중 하나의 값을 가지는 **부울**bool 값을 반환한다.
- **부울 자료형은 True와 False 값을 가지는 자료형인데, 원래 부울 값이 아닌 자료형의 데이터도 부울형으로 변환될 수 있다.**
 - 먼저 숫자 0을 bool 형으로 변환하면 False가 되고, 0을 제외한 모든 수를 bool 형으로 변환하면 True가 된다.
 - “와 같은 공백문자를 bool형으로 반환하면 False가 되고 공백문자 이외의 문자는 True가 된다
 - None은 값이 없음을 의미하는 키워드이다

21

논리연산자(계속)

연산자	의미
x or y	x나 y중에서 하나라도 참이면 참이 되며, 모두 거짓일 때만 거짓이 된다.
x and y	x와 y중 거짓(False)이 하나라도 있으면 거짓이 되며 모두 참(True)인 경우에만 참이다.
not x	x가 참이면 거짓, x가 거짓이면 참이 된다.

X or Y			X and Y			not X	
X	Y	출력	X	Y	출력	X	출력
0	0	0	0	0	0	0	1
0	1	1	0	1	0	1	0
1	0	1	1	0	0		
1	1	1	1	1	1		

22

논리연산자(계속)

- 109와 -1은 True, 0은 False로 간주되는 것을 볼 수 있다.
- **None**이나 공백 또한 False로 간주되는 것을 살펴볼 수 있다.
- 여기서 **None**이란 값이 없는 것을 표현하는 파이썬의 키워드이다.
- **0이나 비어 있는 것은 False, 0이 아닌 값이나 무엇인가 들어있는 것은 True로 간주된다는 것을 잘 확인해 두자.**

23

비트연산자

연산자	의미	설명
&	비트 단위 AND	두 개의 피연산자의 해당 비트가 모두 1이면 1, 아니면 0
	비트 단위 OR	두 피연산자의 해당 비트 중 하나라도 1이면 1, 아니면 0
^	비트 단위 XOR	두 개의 피연산자의 해당 비트의 값이 같으면 0, 아니면 1
~	비트 단위 NOT	0은 1로 만들고, 1은 0으로 만든다.
<<	비트 단위 왼쪽으로 이동	지정된 개수만큼 모든 비트를 왼쪽으로 이동시킨다.
>>	비트 단위 오른쪽으로 이동	지정된 개수만큼 모든 비트를 오른쪽을 이동시킨다.

24

복합 연산 할당자 : 비트 연산자

연산자	사용 예시	의미
&=	i &= 10	i = i & 10
=	i = 10	i = i 10
^=	i ^= 10	i = i ^ 10
<<=	i <<= 10	i = i << 10
>>=	i >>= 10	i = i >> 10

25

연산자 우선순위

연산자	설명
()	괄호 연산자로 가장 높은 우선 순위 연산자
**	지수 연산자
~, +, -	단항 연산자(예: -10, +20)
*, /, %, //	곱셈, 나눗셈, 나머지 연산자
+, -	덧셈, 뺄셈
>>, <<	비트 이동 연산자
&	비트 AND 연산자
^,	비트 XOR 연산자, 비트 OR 연산자
<=, <, >, >=	비교 연산자
==, !=	동등 연산자
=, %=, /=, //=, -=, +=, *=, **=	할당 연산자, 복합 할당 연산자
is, is not	아이덴티티 연산자
in, not in	소속 연산자
not, or, and	논리 연산자

높은
우선순위



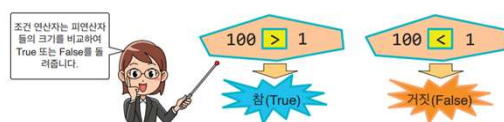
26

조건문

27

조건문

- 어떤 **조건condition**을 만족하는지 그렇지 않은지를 판정하는 식을 **조건식condition expression**이라고 한다.
- 그리고 이 조건식은 참 또는 거짓의 값을 갖는 **부울bool**형으로 평가된다.
- 관계 연산자relational operator**는 두 개의 피연산자를 비교하는데 사용되는데, 관계 연산자의 결과는 **참True** 아니면 **거짓False**으로 나타난다.
- 따라서 이것은 참과 거짓의 값을 갖는 조건식이 될 수 있다.
- 파이썬에서 참과 거짓은 True와 False로 표시되기 때문에 True 또는 False가 관계 연산의 값으로 생성된다.



28

if문

<기본 문법>

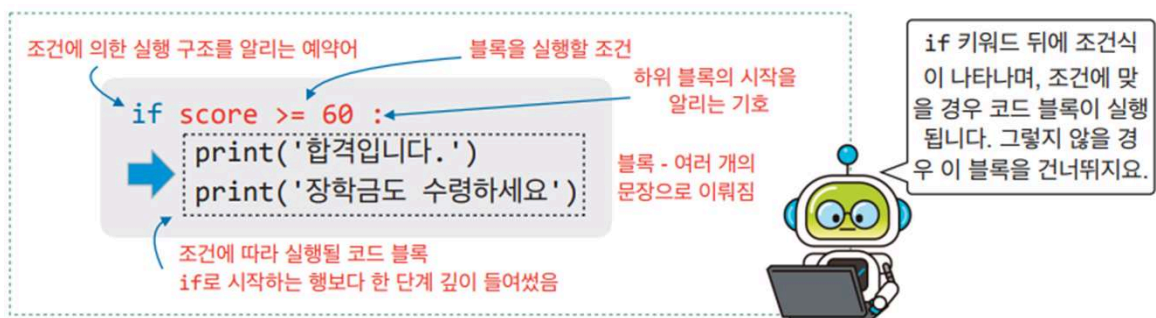
```
if 조건식 :  
    실행할 내용
```

<예시>

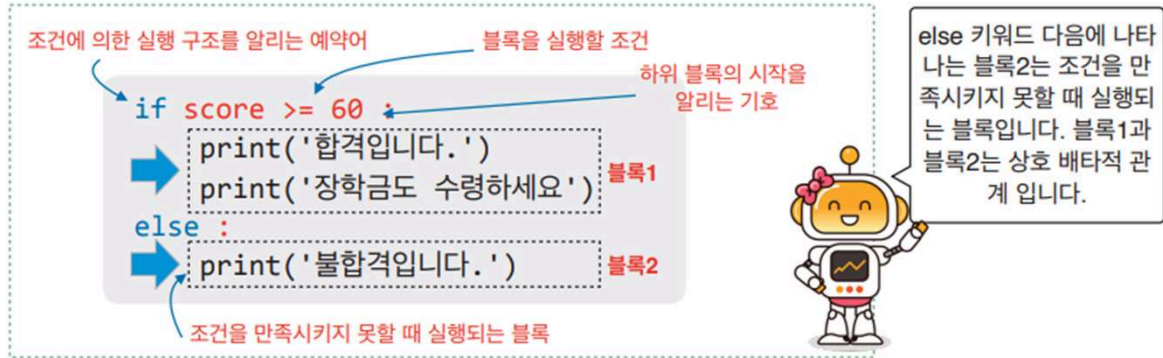
```
x = 100  
if x > 1:  
    print("x는 1보다 큼니다.")
```

x는 1보다 큼니다.

29



30



31

실습 시간



달력은 기본적으로 지구가 태양을 공전하는 시간을 기준으로 작성된다. 하지만 실제로 측정하여 보니 지구가 태양을 완전히 한 바퀴 도는데 걸리는 시간은 365일보다 약 1/4 만큼 더 걸린다. 따라서 매 4년마다 하루 정도 오차가 생기는 셈이다. 이것을 조정하기 위하여 윤년이 생겼다. 입력된 연도가 윤년인지 아닌지를 판단하는 프로그램을 만들어보자. 사용자로부터 연도를 입력받아서 다음과 같은 조건을 검사한다. 윤년은 다음의 조건을 만족해야 한다.

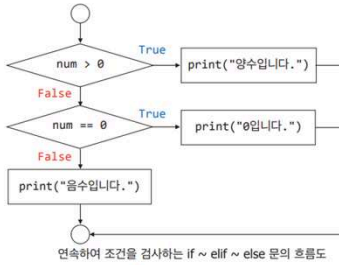
1. 연도가 4로 나누어 떨어지면 윤년이다.
2. 연수가 4로 나누어 떨어져도, 100으로는 나누어 떨어지지 않아야 한다.
3. 연수가 400으로 나누어 떨어지는 해는 무조건 윤년으로 한다.



원하는 결과

연도를 입력하시오: 2012
2012 년은 윤년입니다.

32



```

num = int(input("정수를 입력하시오: ")) # 사용자의 입력을 받아서 정수값으로 변환함

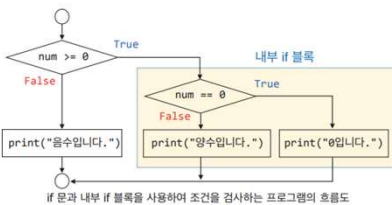
if num > 0: # 첫 번째 조건을 만족하면 두번째 elif 문의 조건을 검사하지 않음
    print("양수입니다.")
elif num == 0: # 첫 번째 조건을 만족하지 못할 경우 조건 검사를 수행함
    print("0입니다.")
else: # 위의 두 조건을 모두 만족시키지 못한 경우 수행되는 문장
    print("음수입니다.")
  
```

정수를 입력하시오: 0
0입니다.

정수를 입력하시오: 10
양수입니다.

정수를 입력하시오: -10
음수입니다.

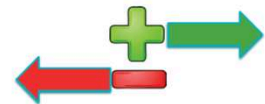
33



```

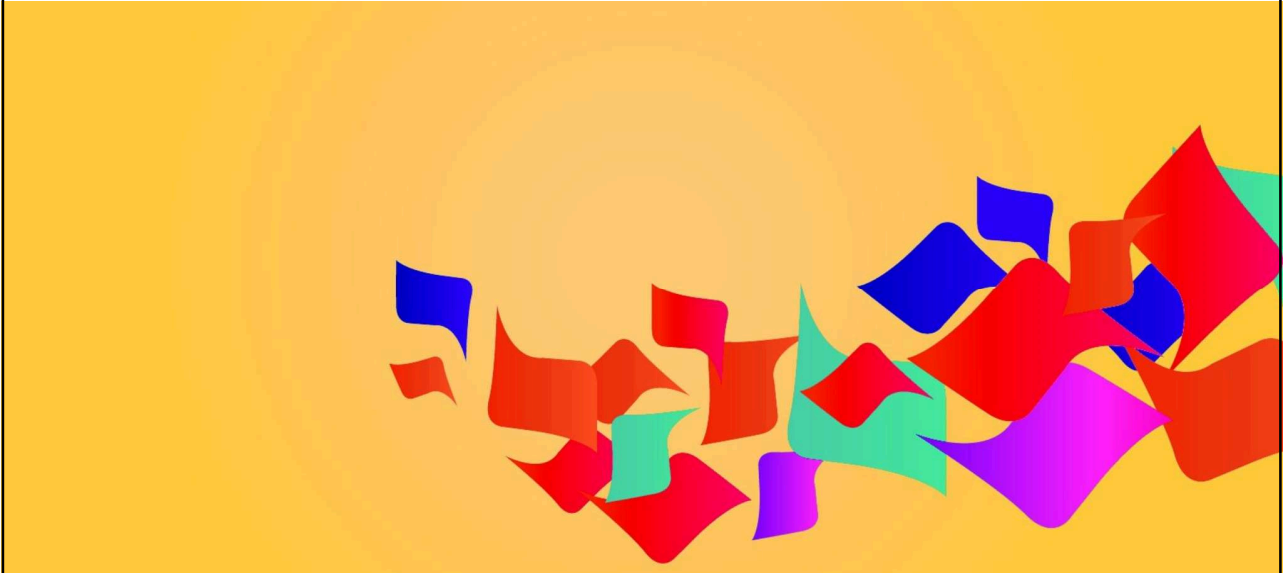
num = int(input("정수를 입력하시오: "))
if num >= 0: # if 문으로 조건을 검사하고 반드시 들여쓰기를 하여 블록을 생성
    if num == 0: # 블록 내에서 세부적인 조건을 한 번 더 검사
        print("0입니다.") # 또 다시 들여쓰기를 할 것
    else:
        print("양수입니다.")
else:
    print("음수입니다.")
  
```

정수를 입력하시오: 0
0입니다.
정수를 입력하시오: 10
양수입니다.
정수를 입력하시오: -10
음수입니다.



34

반복문



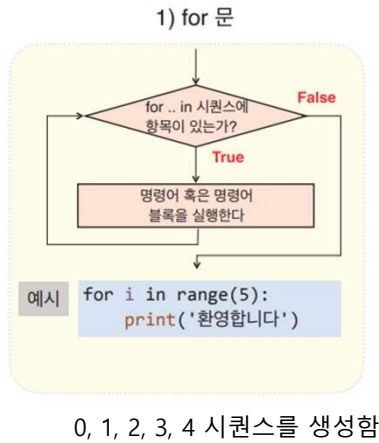
35

파이썬에서는 2가지 종류의 반복이 있다.

1. 횟수 제어 반복(**for 문**): 보통 횟수를 정해 놓고 반복한다.
 2. 조건 제어 반복(**while 문**): 특정한 조건이 만족되면 계속 반복한다.
- 횟수 제어 반복은 반복을 시작하기 전에 반복의 횟수를 미리 아는 경우에 사용한다.
 - 예를 들어서 "환영합니다" 문장을 5번 반복하여 출력한다면 **for 문**을 사용하면 자연스럽다.
 - 파이썬에서는 항목들을 모아 놓은 시퀀스라는 객체가 있고 여기에서 항목을 하나씩 가져와서 반복 할 때도 **for 문**이 적합하다.
 - 시퀀스에 항목이 더 이상 없으면 반복이 종료된다.

36

반복의 종류(for)



- 왼쪽의 그림에서 range(5)라는 함수는 0, 1, 2, 3, 4의 숫자 시퀀스를 자동으로 생성하는 함수이다.
- 따라서 이 문장은 0에서 4까지의 숫자를 반환하는 작업을 끝낼 때 까지 반복 실행된다.

37

for문

```

for i in [1, 2, 3, 4, 5]:      # 반복문의 끝에 :이 있어야 함
    print("방문을 환영합니다!") # if 문과 마찬가지로 들여쓰기를 하여야 함
    
```

- for 루프를 사용할 때는 다음과 같이 리스트를 이용하여 사용하는 방식을 먼저 연습
- 여기서 [...]은 **리스트list** 자료형
- 리스트는 여러 개의 값들을 담을 수 있는 장바구니와 같은 개념으로 이해
- 리스트 [1, 2, 3, 4, 5] 안에는 정수 1, 2, 3, 4, 5가 담겨있다고 생각

38

for문(계속)

- 반복해서 실행될 문장은 반드시 들여쓰기를 하여야 한다.
- for 문에 담긴 **블록**이라 생각하면 된다. 위의 코드를 실행하면 다음과 같은 출력을 얻을 수 있다.

```
방문을 환영합니다!
방문을 환영합니다!
방문을 환영합니다!
방문을 환영합니다!
방문을 환영합니다!
```

- 첫 번째 반복에서 변수 `i`의 값은 리스트의 첫 번째 숫자인 1이 되고 `print(...)` 문장이 실행된다.
- 두 번째 반복에서 변수 `i`의 값은 리스트의 두 번째 숫자인 2가 되고 `print(...)` 문장이 실행된다.
- 세 번째 반복에서 변수 `i`의 값은 리스트의 세 번째 숫자인 3이 되고 `print(...)` 문장이 실행된다.
- 네 번째 반복에서 변수 `i`의 값은 리스트의 네 번째 숫자인 4가 되고 `print(...)` 문장이 실행된다.
- 다섯 번째 반복에서 변수 `i`의 값은 리스트의 다섯 번째 숫자인 5가 되고 `print(...)` 문장이 실행된다.

39

for문-range()함수



- `range(3)` 함수는 0, 1, 2까지의 **숫자열** `sequence`을 반환한다.
- 반복할 때마다 변수 `i`에 이 값들을 대입하면서 문장을 반복한다.
즉, 첫 번째 반복에서는 `i`는 0이고 되고 두 번째 반복에서는 1이 된다.
마지막 반복에서 `i`는 2가 된다.
- 하나 이상의 반복되는 문장이 올 수 있는데,
반복되는 문장은 `if` 문과 같이 **반드시 동일한 간격의 들여쓰기를 해야 한다.**
이 때 들여쓰기가 있는 문장들만이 반복된다.

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



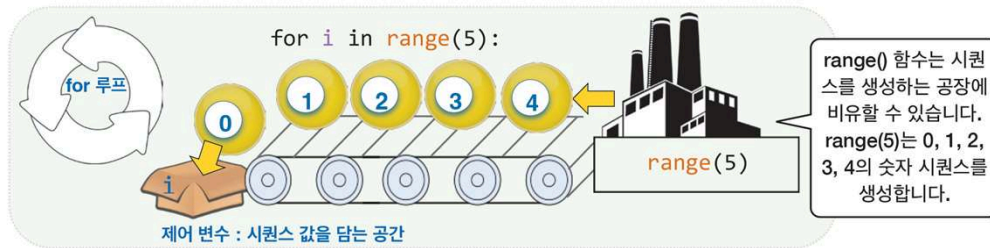
잠깐 - `range()` 함수가 반환하는 값은 `range` 형이다

파이썬의 `range()` 함수는 `range` 형의 자료형을 반환한다. `range` 형의 자료형은 호출이 발생할 때마다 매번 연속된 값을 생성하여 반환하는 특별한 일을 한다. 따라서 주로 `for` 문과 함께 사용된다.

40

range() 함수는 숫자를 생산하는 공장이다.

- range() 함수는 숫자들을 생산하는 공장으로 생각하면 된다.
- 그림과 같이 range(5)은 5개의 정수를 생성한다.
- 0, 1, 2, 3, 4가 바로 그것이다. 그리고 for 루프는 이 시퀀스의 갯 수만큼 반복 수행한다.



41

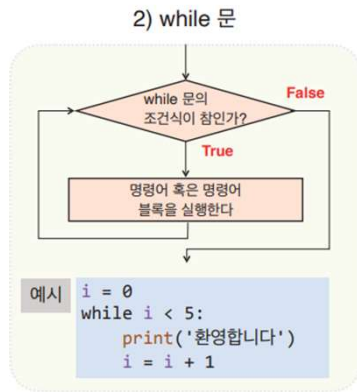
range() 함수는 숫자를 생산하는 공장이다.(계속)

- range() 함수는 여러 개의 인자를 이용하여 좀 더 다양한 정수 시퀀스를 생성할 수 있으며, 일반적인 형식은 다음과 같다.
- range(start, stop, step)이라고 호출하면 start에서 시작하여 (stop-1)까지 step 간격으로 정수들이 생성된다.
- 0, 1, 2, 3, 4가 바로 그것이다. 그리고 for 루프는 이 시퀀스의 갯 수만큼 반복 수행한다.
- 여기서 start와 step가 생략될 수 있는데 이 경우 start는 0으로 간주되고 step은 1로 간주된다.
- 하지만 **stop 값은 반드시 지정해야만 루프가 수행된다.**



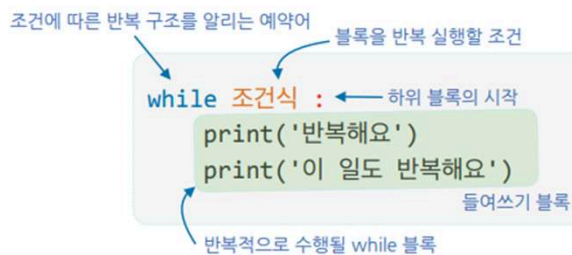
42

반복의 종류(while)



- i 를 0으로 초기화시킨다
- while문은 $i < 5$ 의 조건이 참일 경우 '환영합니다'를 출력하고 i 값을 1 증가시킨다.
- 최초의 i 값이 0이므로 이 반복문 역시 5회 반복 수행하게 된다.
- 조건 제어 반복은 **어떤 조건이 만족되는 동안 반복**하기 때문에 붙여진 이름이다.

43



while 반복문은 조건식이 참일 경우 블록의 내용을 반복해서 실행합니다. 들여쓰기 블록은 while로 시작하는 행보다 한 단계 깊이 들어가야 합니다.

위의 예를 프로그램으로 작성해보면 다음과 같다.

```
under_construction = True
while under_construction:
    response = input("공사가 완료 되었습니까?")
    if response == "예" :
        under_construction = False

print("루프를 탈출했습니다.")
```

44

무한 루프와 break로 빠져나가기

- 조건 제어 루프에서 가끔은 프로그램이 무한히 반복하는 일이 발생한다. 이것은 **무한 루프** infinite loop라고 한다.
- 무한 반복이 발생하면 프로그램은 빠져 나올 수 없기 때문에 문제가 된다.
- 하지만 가끔은 무한 루프가 사용되는데 예를 들면 신호등 제어 프로그램은 무한 반복하여야 하기 때문이다.
- 무한 반복 루프는 다음과 같은 형태를 가진다.



45

무한 루프와 break로 빠져나가기

- `while` 루프의 조건에 `True`가 있다. 따라서 조건이 항상 참이므로 무한히 반복된다.
 - 무한 루프라고 하더라도 어떤 조건이 성립하면 무한 루프를 빠져나와야 하는 경우도 많다.
 - 이런 경우는 `if` 문장을 사용하여 루프를 빠져나오게 된다. `break` 문장은 루프를 강제적으로 빠져 나올 때 사용하는 문장이다.

```
while True:
    light = input('신호등 색상을 입력하시오:')
    if light == 'green':
        break

print('녹색 신호입니다. 전진하세요!')
```

- `while` 루프의 조건에 `True`가 있다. 따라서 조건이 항상 참이므로 무한히 반복된다.
 - 무한 루프라고 하더라도 어떤 조건이 성립하면 무한 루프를 빠져나와야 하는 경우도 많다.
 - 이런 경우는 `if` 문장을 사용하여 루프를 빠져나오게 된다. `break` 문장은 루프를 강제적으로 빠져 나올 때 사용하는 문장이다.

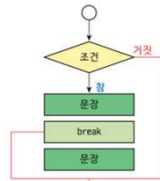
46

루프를 제어하는 고급 기법 : continue와 break

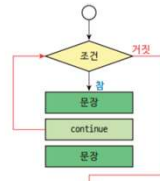
- 루프를 제어하는 첫번째 방법은 break 문으로 이 문장을 만나면 가장 가까운 블럭을 즉시 빠져나오게 된다.
- 그렇다면 continue는 어떻게 수행될까?
이 키워드는 루프를 빠져나오지 않고 *continue* 아래의 문장만을 건너뛰는 역할을 한다.
- 즉, 반복문이 종료되는 것은 조건이 거짓일 때에만 해당한다.

```
st = 'I love Python Programming' # 출력을 위한 문자열
for ch in st:
    if ch in ['a','e','i','o','u', 'A','E','I','O','U']:
        continue # 모음일 경우 아래 출력을 건너뛰다
    print(ch, end='')
```

```
lv Pythn Prgrmmng
```



break를 표현한 흐름도



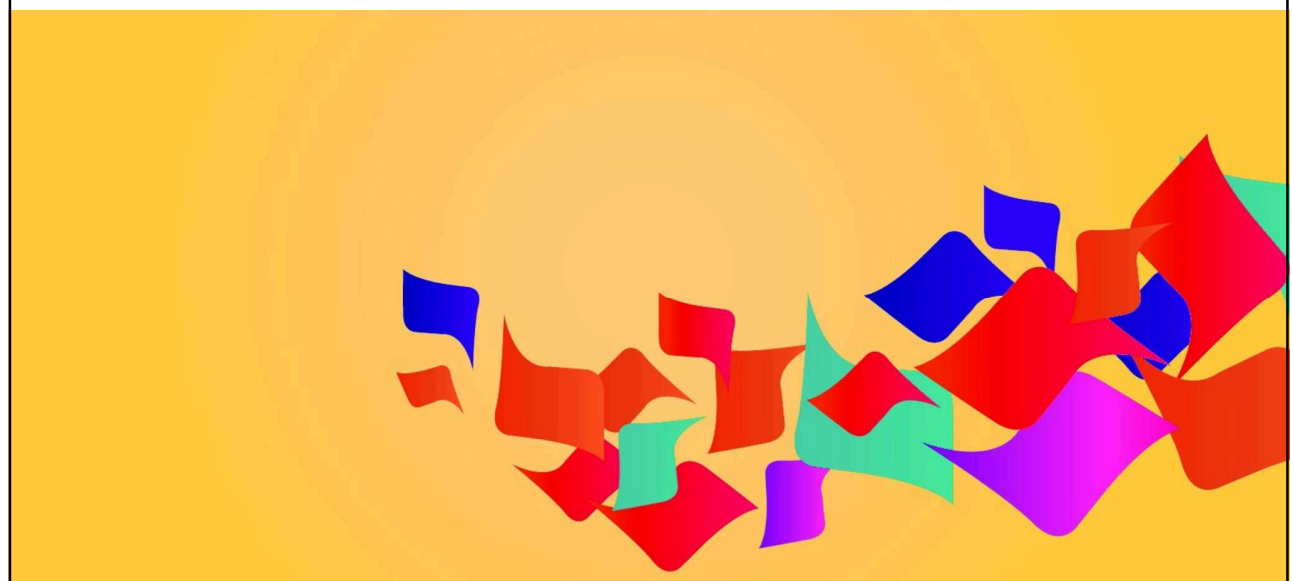
continue를 표현한 흐름도



break 문은 루프를 즉시 빠져나가지만, continue 문은 루프를 빠져나가지 않고 continue 아래쪽 문장을 건너뛰기만 합니다.

47

함수



48

- 파이썬에서는 프로그램을 쪼개는 **3가지의 방법**이 있다.
 - **함수function**는 우리가 반복적으로 사용하는 코드를 묶은 것으로 코드의 덩어리와 같다.
 - **객체object**는 코드 중에서 독립적인 단위로 분리할 수 있는 조각이다.
 - **모듈module**은 프로그램의 일부를 가지고 있는 독립적인 파일이다.
- **함수는 일을 수행하는 코드의 덩어리**이며, 더 큰 프로그램을 구축하는 데 사용할 수 있는 작은 조각이다.



49

함수 정의를 위한 예약어 함수의 이름 함수가 받아오는 데이터: 이 함수는 매개변수가 없음

```
def print_address():
    print('경상북도 울릉군 울릉읍')
    print('독도리 산 1-96번지')
```

하위 블록의 시작 함수 몸체: 들여쓰기 블록

하나의 블록을 이루는 코드 문치는 같은 깊이로 들여쓰여 함



함수를 정의하기 위해서는 **def** 라는 예약어를 사용합니다. 다음으로 소괄호와 콜론을 붙입니다. 함수의 몸체는 들여쓰기를 한 코드 블록으로 구성합니다.

50

함수에 하나의 값을 넘겨주고 일을 시키자

- 우리는 함수 외부에서 함수 안으로 값(정보)을 전달할 수 있다. 이 값을 **인자(argument)**라고 한다.



- 예를 들어서 앞의 주소를 인쇄하는 print_address() 함수에서 수신자의 이름은 외부에서 전달받는 것으로 해보자. 수신자의 이름을 변경할 수 있으면 같은 회사에서 근무하는 사람들도 사용할 수 있을 것이다.

51

함수에 하나의 값을 넘겨주고 일을 시키자

- 함수 호출시 전달되는 실제 값을 **인자(argument)**라고 하고, 함수 내부에서 전달받는 변수를 **매개변수(parameter)**라고 한다.

함수 내부에서 전달 받는 변수 : 매개변수

```
def print_address(name)
    print("서울 특별시 종로구 1번지")
    print("파이썬 빌딩 7층")
    print("수신 :", name)

print_address("홍길동") # "홍길동" 대신 다른 이름
print_address("넌넌한 교수")
```

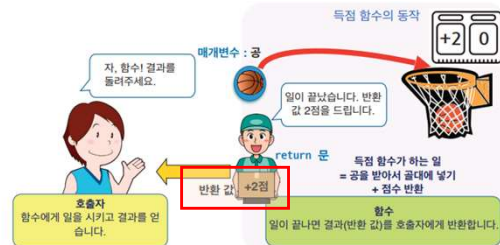
서울 특별시 종로구 1번지
파이썬 빌딩 7층
홍길동

서울 특별시 종로구 1번지
파이썬 빌딩 7층
넌넌한 교수

함수에 넘겨지는 값 : 인자

52

- 지금까지, 함수는 우리가 주는 값을 받아서 작업을 처리하였다. 그러나 함수가 매우 유용한 것은 함수가 주어진 일을 하고 우리에게 뭔가를 보낼 수 있다는 점이다. 이와 같이 함수로부터 되돌아오는 값을 **반환 값** **return value**이라고 한다.



- 함수가 함수 내부의 값을 외부에 보낼때는 **return 키워드**를 사용하면 된다. 원의 반지름을 보내면 원의 면적을 계산해서 반환하는 함수를 작성해보자.

53

```
def calculate_area(radius):
    area = 3.14 * radius**2    # 원의 면적을 구하는 근사식
    return area                # 이전 줄에서 구한 area 값을 호출문에 돌려준다
```

```
c_area = calculate_area(5.0)   # calculate_are() 함수가 계산한 값을 c_area에 저장
```

```
def get_sum(start, end):
    s = 0
    for i in range(start, end+1):
        s += i
    return s

x = get_sum(1, 10)             # 인자 1, 10을 넘겨주며 함수 get_sum()을 호출함
print('1에서 10까지의 합 :', x) # 1에서 10까지의 정수의 합 55를 출력한다
```

```
1에서 10까지의 합 : 55
```

54

- 지금까지 함수는 호출될 때 맡은 일을 수행하고 수행을 마치거나, 수행을 마칠 때 **하나의 값을 반환**하는 일을 하는 것을 살펴보았다.
- 파이썬의 큰 특징 중 하나는 함수가 **여러 개의 값을 반환**할 수 있다는 것으로, 아래와 같은 형식을 사용한다. 가장 큰 이유는 생산성이 뛰어나기 때문이다.
- 이와 같은 특징 때문에 파이썬을 이용하면 간결하면서도 효율적인 프로그램을 빠르게 작성할 수 있다.

```
def sort_num(n1, n2):          # 2개의 값을 받아오는 함수
    if n1 < n2:
        return n1, n2        # n1이 더 작으면 n1, n2 순서로 반환
    else:
        return n2, n1        # n2가 더 작으면 n2, n1 순서로 반환

print(sort_num(110, 210))     # 110과 210을 함수의 인자로 전달하고 반환되는 값을 출력
print(sort_num(2100, 80))

(110, 210)
(80, 2100)
```

55

```
def calc(n1, n2):
    return n1 + n2, n1 - n2, n1 * n2, n1 / n2 # 덧셈, 뺄셈, 곱셈, 나눗셈 결과를 반환

n1, n2 = 200, 100
t1, t2, t3, t4 = calc(n1, n2) # 네 개의 값을 반환받기 위해 4개의 변수를 사용함
print(n1, '+', n2, '=', t1)
print(n1, '-', n2, '=', t2)
print(n1, '*', n2, '=', t3)
print(n1, '/', n2, '=', t4)
```

```
200 + 100 = 300
200 - 100 = 100
200 * 100 = 20000
200 / 100 = 2.0
```

56

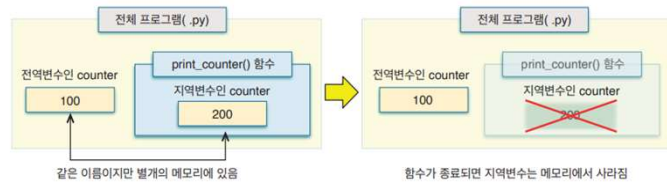
지역변수 local variable

```
def print_counter():
    counter = 200
    print('counter =', counter)
```

함수가 호출될 때 생성되어
호출이 완료되면 메모리에서
사라지는 지역변수

```
counter = 100
print_counter()
print('count =', counter)
```

print_counter() 함수의 호출에
관계없이 메모리에서 유지되는
전역변수



57

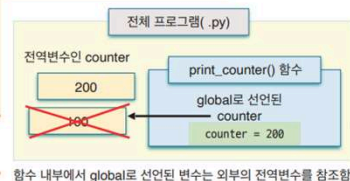
함수 안에서 전역변수 사용하기 : global 키워드

- 만일 함수 내부에서 새로 변수를 만들지 않고 전역변수인 counter를 불러서 사용하려면 어떻게 해야 할까?
- 아래와 같이 global이라는 키워드를 사용하는 것이 바람직하다.
- global counter는 함수 외부의 전역변수 counter를 사용하겠다는 선언**

```
def print_counter():
    global counter # 함수 외부의 전역변수 counter를 사용하겠다는 선언
    counter = 200
    print('counter =', counter) # 함수 내부의 counter 값
```

```
counter = 100
print_counter()
print('counter =', counter) # 함수 외부의 counter 값
```

```
counter = 200
counter = 200
```



함수의 내부에서 전역변수를 사용하기 위해서는 global 키워드를 사용합니다. 함수의 내부에서 counter 값을 변경하면 전역변수의 값이 변경됩니다.

- 이와 같은 선언을 할 경우 아래의 그림과 같이 print_counter()는 외부의 전역변수 counter를 사용하므로 **counter = 200을 적용**하면 전역변수의 값이 200으로 변경된다.

58

디폴트 인자 default argument

```
def order(num, pickle = True, onion = True) :  
    print('햄버거 {0} 개 - 피클 {1}, 양파 {2}'.format(num, pickle, onion))  
  
order(1, pickle = False, onion = True)  
order(2)      # 햄버거 2개를 주문, 디폴트로 pickle, onion 값은 True임  
  
햄버거 1 개 - 피클 False, 양파 True  
햄버거 2 개 - 피클 True, 양파 True
```

order(3) 만하면 자동으로 order(3, True, True) 가 된다

- 디폴트 값이 있는 매개 변수는 다음과 같이 순서를 바꾸어 인자를 넣을 수도 있다.

```
>>> order(3, onion = False, pickle = True)    # 인자의 순서를 바꾸었다.  
햄버거 3 개 - 피클 True, 양파 False
```

59

List



60

- 여러 개의 데이터를 하나로 묶어서 저장하는 것이 필요하다.
- 파이썬에서는 **리스트list**와 **딕셔너리dictionary**를 이용하여 여러 개의 데이터를 한꺼번에 저장하고 처리할 수 있다.

```
>>> height1 = 178.9    # float 타입의 데이터를 저장한다.
>>> height2 = 173.5    # float 타입의 데이터를 저장한다.
>>> height3 = 166.1    # float 타입의 데이터를 저장한다.
>>> height4 = 164.3    # float 타입의 데이터를 저장한다.
>>> height5 = 176.4    # float 타입의 데이터를 저장한다.
```

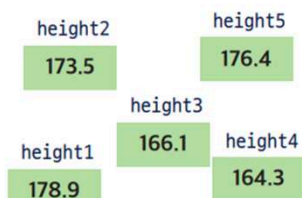
61

리스트(List)란?

- 이럴 때는 "리스트(list)"를 만들고 여기에 데이터를 저장하면 된다
- 파이썬에서 리스트는 대괄호를 이용해서 만들 수 있다.
- 예를 들어, 학생 5명의 키를 리스트에 저장하려면 다음과 같이 한다.

```
>>> heights = [178.9, 173.5, 166.1, 164.3, 176.4] # 리스트로 5명의 키를 저장
>>> heights
[178.9, 173.5, 166.1, 164.3, 176.4]
```

리스트를 사용하면 하나의 변수에 여러 개의 값을 담을 수 있다.



개별 변수로 표현된 데이터



리스트의 항목 또는 요소라고 함

리스트로 표현된 데이터와 인덱스

리스트 내의 개별 데이터를 항목 또는 Item이라고 한다.

62

리스트 – 최초 생성

- 파이썬에서 리스트는 다른 변수처럼 생성할 수 있다.
- 즉, 항목들을 [] 안에 적어주고 변수에 저장하면 리스트 변수가 생성된다.
- 다음과 같은 아이돌 그룹 BTS의 멤버 3명으로 리스트를 간단하게 만들어 보자.

```
>>> bts = ['V', 'Jungkook', 'Jimin']
```

- 만일 초기에 멤버가 없을 경우 다음과 같은 방식도 가능하다. 이때는 빈 리스트가 만들어진다.

```
>>> bts = []  
>>> bts = list()
```

63

리스트 – 아이템 값 삽입

```
>>> bts.append('Jin')  
>>> bts.append('Suga')  
>>> bts  
['V', 'Jin', 'Suga']
```

리스트 내에 새 항목을
추가한다.

```
>>> bts = ['V', 'Jin', 'Suga', 'Jungkook']  
>>> bts = bts + ['RM'] # 덧셈 연산자로 멤버 'RM'을 추가함  
>>> bts  
['V', 'Jin', 'Suga', 'Jungkook', 'RM']
```

리스트 내에 새 항목
'RM'을 추가한다.

```
>>> list(range(5))          # 0에서 5 미만 정수열을 생성(5는 포함 안됨)  
[0, 1, 2, 3, 4]  
>>> list(range(0, 5))      # list(range(5))와 동일한 결과  
[0, 1, 2, 3, 4]  
>>> list(range(0, 5, 1))    # list(range(0, 5))와 동일한 결과  
[0, 1, 2, 3, 4]  
>>> list(range(0, 5, 2))    # 생성하는 값을 2씩 증가시킴  
[0, 2, 4]  
>>> list(range(2, 5))       # 2에서 5미만의 연속된 수 2, 3, 4를 생성  
[2, 3, 4]  
>>> list(range(2, 6))       # 2에서 6미만의 연속된 수 2, 3, 4, 5를 생성  
[2, 3, 4, 5]  
>>> list('python')          # 'python' 문자열 시퀀스로부터 리스트를 생성  
['p', 'y', 't', 'h', 'o', 'n']
```

range() 함수를 사용해서
여러 항목을 한꺼번에
생성할 수 있다.

64

리스트 연산

- 파이썬에서는 리스트에 대해서도 다양한 연산이 가능하다.
- 예를 들어서 우리는 덧셈 연산으로 다음과 같이 두 개의 리스트를 합칠 수 있다.

```
>>> bts = ['V', 'J-Hope'] + ['RM', 'Jungkook', 'Jin']
>>> bts
['V', 'J-Hope', 'RM', 'Jungkook', 'Jin']
```

- 정수 리스트를 서로 더했을 때는 대응되는 원소끼리 더해지는 것이 아니라 리스트가 합쳐진다는 것을 명심하자.

두 리스트를 합치는 연산

```
>>> num_list = [0, 1, 2] * 3 # [0, 1, 2]가 3회 반복되어 저장됨
>>> num_list
[0, 1, 2, 0, 1, 2, 0, 1, 2]
```

```
>>> numbers = [10, 20, 30] + [40, 50, 60]
>>> numbers
[10, 20, 30, 40, 50, 60]
```

65

리스트 연산(계속)

- 대응되는 원소끼리 더하려면 뒤에서 설명하는 **넘파이NumPy**를 사용해야 한다.

```
>>> bts = ['V', 'J-Hope', 'RM', 'Jungkook', 'Jin', 'Jimin', 'Suga']
>>> 'V' in bts
True
>>> 'V' not in bts
False
```

'V'라는 멤버가 bts에 있는가?

66

리스트 – Index

- 파이썬 리스트에 저장된 데이터를 꺼내려면 어떻게 하면 될까? 예를 들어서 다음과 같이 알파벳 문자를 저장하고 있는 리스트가 있다고 하자.

```
>>> letters = ['A', 'B', 'C', 'D', 'E', 'F']
```

- 리스트에서 데이터를 추출하려면 **인덱스index**라는 정수를 사용한다. 리스트의 첫 번째 데이터는 인덱스 0을, 두 번째 요소는 인덱스 1을 가지게 된다. 나머지 요소들도 유사하게 인덱스가 할당된다.

```
>>> letters[0] # 리스트의 첫 번째 항목에 접근  
'A'
```

letters	'A'	'B'	'C'	'D'	'E'	'F'
인덱스	0	1	2	3	4	5

 letters[0]이 가리키는 객체

67

리스트 – Index(계속)

```
>>> letters[1] # 리스트의 두 번째 항목에 접근  
'B'  
>>> letters[2] # 리스트의 세 번째 항목에 접근  
'C'
```

파이썬에서는 인덱스를 음수로 줄 수도 있다!
리스트의 맨 끝에 있는 데이터를 얻고자 할 때 유용

```
>>> letters[-1] # 리스트의 마지막 항목에 접근  
'F'
```

letters	'A'	'B'	'C'	'D'	'E'	'F'
인덱스	0	1	2	3	4	5
음수 인덱스	-6	-5	-4	-3	-2	-1

 letters[-1]

68

리스트 – 아이템 값 변경

'Park'의 키를 175.0로 변경하고 싶으면
인덱스를 사용하여 다음과 같이 수정

```
>>> slist[3] = 175.0
```

```
>>> slist  
['Kim', 178.9, 'Park', 175.0, 'Lee', 176.1]
```

함수를 사용하여 동적으로 항목을 추가하려면
append()나 insert() 함수를 사용할 수 있다

```
>>> slist.insert(4, 'Hong')  
>>> slist.insert(5, 168.1)  
>>> slist  
['Kim', 178.9, 'Paik', 180.0, 'Hong', 168.1, 'Lee', 176.1,]
```

69

리스트 – 아이템 값 변경(계속)

- 리스트의 슬라이싱을 사용하여 한 번에 여러 원소의 값을 변경할 수도 있다.
- 예를 들어서 Park의 이름을 'Paik'으로, 키를 180.0으로 바꿔 주고 싶은 경우, slist[2:4]를 써준 뒤 이름과 키를 리스트로 적으면 된다.

```
>>> slist[2:4] = ['Paik', 180.0] # 슬라이싱 범위와 대체할 데이터  
>>> slist  
['Kim', 178.9, 'Paik', 180.0, 'Lee', 176.1]
```

- 함수를 사용하여 동적으로 항목을 추가하려면 append()나 insert() 함수를 사용할 수 있다. append()는 리스트의 뒤쪽에
만 추가할 수 있으나 insert(index, item)는 지정된 index 위
치에 항목을 추가한다.

70

리스트 – 아이템 값 삭제

- 리스트의 항목을 삭제하는 것도 가능할까? 물론이다.
- 삭제를 위해서는 `remove()`, `pop()` 메소드 그리고 `del` 명령을 사용할 수 있다.
- **`remove()` 메소드**는 다음과 같이 리스트에서 지정된 항목을 삭제한다.

```
>>> bts = [ 'V', 'J-Hope', 'Suga', 'Jungkook' ]
>>> bts.remove('Jungkook')
>>> bts
['V', 'J-Hope', 'Suga']
```

`remove()` 메소드는 다음과 같이 리스트에서 지정된 항목을 삭제

조건이 참인 경우에만 `bts` 리스트에서 이 항목을 삭제하면 안전한 프로그램이 된다

```
if 'Suga' in bts:      # 'Suga' 멤버가 bts 리스트에 있는가를 조회하는 기능
    bts.remove('Suga')
```

71

리스트 – 아이템 값 삭제(계속)

- `pop()` 역시 리스트에서 항목을 삭제하는 역할을 하는데
- `remove()`와는 달리 리스트의 마지막 항목을 삭제하여 그 항목값을 반환한다.

```
>>> bts = ['V', 'J-Hope', 'Suga', 'Jungkook']
>>> last_member = bts.pop() # 마지막 항목 'Jungkook'을 삭제하고 반환한다
>>> last_member # 삭제된 항목을 출력하자
'Jungkook'
>>> bts # bts 리스트에 마지막 항목이 삭제되었는가 확인하자
['V', 'J-Hope', 'Suga']
```

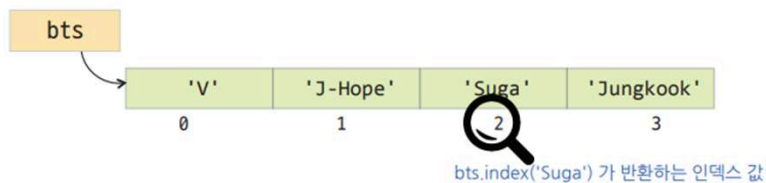
- `pop()` 메소드를 사용할 때 인덱스를 넣어서 항목을 삭제할 수 있다.
- 예를 들어 리스트에서 두 번째 원소를 제거하기 위해서는 `pop(1)`을 사용한다.

```
>>> bts = ['V', 'J-Hope', 'Suga', 'Jungkook']
>>> second_member = bts.pop(1) # 두 번째 항목을 삭제하고 반환한다
>>> second_member # 삭제된 항목을 출력하자
'J-Hope'
>>> bts # bts 리스트에 두 번째 항목이 삭제되었는가 확인하자
['V', 'Suga', 'Jungkook']
```

72

리스트 – 특정 아이템 위치 구하기

- 위의 코드를 살펴보면 bts의 멤버 4명의 이름이 들어 있는 리스트가 있다.
- 이 리스트에서 index() 라는 메소드를 사용하여 'Suga'를 인자로 사용하였다. 'Suga'는 bts 리스트의 세 번째 항목이므로 bts.index('Suga')는 2를 반환한다.



73

리스트 – 특정 아이템 위치 구하기(계속)

- 만약 리스트의 모든 항목을 한 번씩 방문하려면 어떻게 해야 할까?
- 아주 간단하다. 다음과 같은 구문을 사용하면 된다.

```
bts = ['V', 'J-Hope', 'Suga', 'Jungkook' ]  
for member in bts:  
    print(member)
```

```
V  
J-Hope  
Suga  
Jungkook
```

74

리스트 – 아이템 값으로 정렬

- 리스트는 정렬할 수도 있다. **정렬** `sorting`이란 리스트 안의 항목들을 크기 순으로 나열하는 것이다.
- 정렬을 위해서는 리스트의 `sort()` 메소드를 사용하면 된다.

```
>>> numbers = [ 9, 6, 7, 1, 8, 4, 5, 3, 2 ]
>>> numbers.sort()           # numbers 리스트를 크기 순으로 정렬하는 메소드
>>> print(numbers)
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```



- 기본정렬 방식은 **오름차순**이므로 위와 같이 1, 2, ..., 9의 결과로 정렬되었다.
- 만일 **내림차순** 정렬을 원한다면 다음과 같이 `reverse` 인자를 줄 수 있다.

```
>>> numbers = [ 9, 6, 7, 1, 8, 4, 5, 3, 2 ]
>>> numbers.sort(reverse=True) # numbers 리스트를 크기의 역순으로 정렬하는 메소드
>>> print(numbers)
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

75

리스트 – 아이템 값으로 정렬(계속)

- 리스트 안에 들어 있는 항목들이 문자열일 경우 사전 순서대로 정렬되는 것도 확인해 볼 수 있다.

```
>>> bts = [ 'V', 'J-Hope', 'Suga', 'Jungkook' ]
>>> bts.sort()           # bts 리스트를 알파벳 순으로 정렬하는 메소드
>>> print(bts)
['J-Hope', 'Jungkook', 'Suga', 'V']
```

- 위의 내용을 보면 `sort()` 메소드는 원래의 리스트의 내부 항목의 순서를 변경시키는 기능을 한다는 것을 알 수 있는데
- 만일 정렬된 새로운 리스트가 필요하다면 다음과 같이 내장함수 `sorted()`를 사용하여야 한다.

```
>>> numbers = [ 9, 6, 7, 1, 8, 4, 5, 3, 2 ]
>>> new_list = sorted(numbers) # numbers 리스트를 정렬하는 내장함수
>>> print(new_list)           # numbers 리스트를 항목의 크기 순으로 정렬
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> print(numbers)           # sorted() 함수는 정렬한 결과를 반환하므로 원래 리스트에는 변화가 없음
[9, 6, 7, 1, 8, 4, 5, 3, 2]
```

- 만약 리스트를 역으로 정렬하고 싶으면 `sorted()`를 호출할 때, `reverse=True`을 붙인다.

76

리스트에 사용 가능한 함수를 알아보자

- 리스트에 다음과 같은 수치값이 있을 경우 `len()`, `max()`, `min()`, `sum()`, `any()` 등의 내장 함수를 사용하여 편리하게 활용할 수 있다.
- `len()`은 리스트의 길이를 반환하며, `max()`는 리스트 항목 중 최대값, `min()`은 리스트 항목 중 최솟값을 반환한다.

```
>>> n_list = [200, 700, 500, 300, 400]
>>> len(n_list)      # 리스트의 길이를 반환한다
5
>>> max(n_list)      # 리스트 항목중 최대값을 반환한다
700
>>> min(n_list)      # 리스트 항목중 최솟값을 반환한다
200
>>> sum(n_list)       # 리스트 항목의 합을 반환한다
2100
>>> sum(n_list) / len(n_list) # 전체의 합을 원소의 개수로 나누면 평균값을 얻을 수 있다
420.0
>>> t_list = list((10, 20, 30)) # 튜플을 리스트로 변환
>>> t_list
[10, 20, 30]
>>> t_list.append(40)      # 새로운 값 40을 리스트 원소로 추가
>>> t_list
[10, 20, 30, 40]
```

77

리스트에 사용 가능한 함수를 알아보자(계속)

- 파이썬 내장함수인 `any()` 함수는 리스트에 참인 원소가 하나라도 있을 경우 `True`를 반환한다.
- 따라서 0과 ""(공백 문자열)만으로 이루어진 리스트는 `any()` 함수가 `False`를 반환한다. 0과 ""(공백 문자열)은 내부적으로 모두 `False`로 간주한다.
- 반면 `all()` 함수는 리스트의 모든 원소가 참인 경우에만 `True`를 반환한다.

```
>>> a_list = [10, 20, 30] # 0이 아닌 숫자로 된 리스트
>>> b_list = [0, 10, 20, 30] # 0과 0이 아닌 숫자로 된 리스트
>>> c_list = [0, ''] # 0과 빈 문자열을 가지는 리스트
>>> any(a_list), any(b_list), any(c_list) # 리스트내에 참인 원소 값이 존재하는가
(True, True, False)
>>> all(a_list), all(b_list), all(c_list) # 리스트내의 모든 원소가 참 값인가
(True, False, False)
```

78

리스트에 사용 가능한 함수를 알아보자(계속)

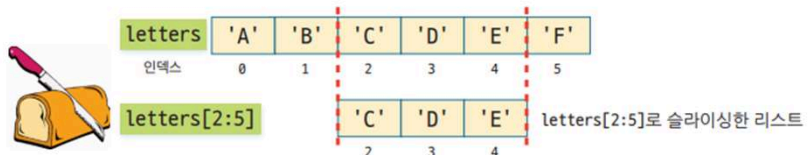
메소드	하는 일
<code>index(x)</code>	원소 <code>x</code> 를 이용하여 위치를 찾는다. 원소 <code>x</code> 의 인덱스 값을 반환한다.
<code>append(x)</code>	원소 <code>x</code> 를 리스트의 끝에 추가한다.
<code>count(x)</code>	리스트 내에서 <code>x</code> 원소의 개수를 반환한다.
<code>extend([x1, x2])</code>	<code>[x1, x2]</code> 리스트를 기존 리스트에 삽입한다.
<code>insert(index, x)</code>	원하는 <code>index</code> 위치에 <code>x</code> 를 추가한다.
<code>remove(x)</code>	<code>x</code> 원소를 리스트에서 삭제한다.
<code>pop(index)</code>	<code>index</code> 위치의 원소를 삭제한 후 반환한다. 이때 <code>index</code> 는 생략될 수 있으며 이 경우 리스트의 마지막 원소를 삭제하고 이를 반환한다.
<code>sort()</code>	값을 오름차순 순서대로 정렬한다. 키워드 인자 <code>reverse=True</code> 이면 내림차순으로 정렬한다.
<code>reverse()</code>	리스트를 원래 원소들의 역순으로 만들어 준다.

79

리스트를 원하는 모양으로 자르는 슬라이싱

- 리스트에서 여러 요소를 선택해서 새로운 리스트를 만들고 싶다면, **슬라이싱**(slicing)이라고 하는 기능을 사용할 수 있다.
- 리스트를 슬라이싱 하려면 다음과 같이 콜론을 이용하여 범위를 지정한다.

```
>>> letters = ['A', 'B', 'C', 'D', 'E', 'F']
>>> letters[2:5]
['C', 'D', 'E']
```



- 슬라이싱을 할 때 `letters[2:5]`와 같이 적으면 2부터 시작하여서 항목들을 추출하다가 5가 나오기 전에 중지한다.

80

리스트를 원하는 모양으로 자르는 슬라이싱(계속)

- 리스트를 슬라이싱하면 원래의 리스트는 손상되지 않으며, 새로운 리스트가 생성되어서 우리에게 반환된다.
- 즉, 슬라이스는 원래의 리스트의 부분 복사본이다.

```
>>> letters[:3]      # 리스트의 처음부터 세번째 항목까지 슬라이싱
['A', 'B', 'C']
```

- 또한 두 번째 인덱스가 생략되면 리스트의 끝까지라고 가정한다. 이것은 생각보다 편리한 기능이다.

```
>>> letters[3:]      # 리스트의 네번째 항목부터 끝까지 슬라이싱
['D', 'E', 'F']
```

- 극단적인 표현도 가능하다.
- 콜론만 있으면 리스트의 처음부터 끝까지라고 생각한다.

81

리스트를 원하는 모양으로 자르는 슬라이싱(계속)

```
>>> letters[:]
['A', 'B', 'C', 'D', 'E', 'F']
```

콜론만 있으면 리스트의 처음부터 끝까지

```
>>> letters[::2]
['A', 'C', 'E']
```

리스트를 처음부터 끝까지 읽어오되, 스텝(step) 만큼 건너뛰기

```
>>> letters[::-1]      # 뒤에서부터 앞으로 읽어오며 원소를 생성한다
['F', 'E', 'D', 'C', 'B', 'A']
```

82

문자열도 리스트의 메소드를 사용할 수 있다.

- 우리가 많이 사용하는 문자열 역시 리스트와 유사한 성질이 있다.
- 우선 다음과 같이 문자열을 list() 함수의 인자로 전달하고 그 출력 결과를 살펴보자.

```
>>> s = 'python'
>>> str_list = list(s) # s 문자열을 리스트로 변환시키기
>>> print(str_list)    # 리스트를 출력해보자
['p', 'y', 't', 'h', 'o', 'n']
```

- 마찬가지로 다음과 같이 인덱싱과 슬라이싱을 통해서 특정한 문자 한 개 또는 문자열을 추출할 수 도 있다.

```
>>> s = 'python'
>>> print('s[0] =', s[0]) # 첫 번째 문자를 추출해 보자
s[0] = p
>>> print('s[1] =', s[1]) # 두 번째 문자를 추출해 보자
s[1] = y
>>> print('s[-1] =', s[-1]) # 마지막 문자를 추출해보자
s[-1] = n
>>> print('s[0:2] =', s[0:2]) # 첫 두 문자열을 추출해보자
s[0:2] = py
>>> print('s[-2:] =', s[-2:]) # 마지막 두 문자열을 추출해보자
s[-2:] = on
>>> print('s[-1:-3:-1] =', s[-1:-3:-1]) # 마지막 두 문자열을 거꾸로 추출
s[-1:-3:-1] = no
```

83

문자열도 리스트의 메소드를 사용할 수 있다.(계속)

- 또한 문자열을 분리할 경우에도 다음과 같이 split() 메소드를 사용하여 이를 쉽게 리스트로 만들 수 있다.
- 만일 숫자들이 공백으로 구분된 다음과 같은 문자열들이 있을 경우 이는 쉽게 리스트로 바꿀 수 있다.

```
>>> s = '23 35 67 88 1'
>>> num_list = s.split() # s 문자열을 공백 문자로 분리하기
>>> print(num_list)
['23', '35', '67', '88', '1']
```

- 또한 문자열을 분리할 경우에도 split() 메소드의 인자로 ','를 사용하여 이를 쉽게 리스트로 만들 수도 있다.
- 만일 숫자들이 쉼표로 구분된 다음과 같은 문자열들이 있을 경우 이는 쉽게 리스트로 바꿀 수 있다.

```
>>> s = '23,35,67,88,1'
>>> num_list = s.split(',') # s 문자열을 쉼표 문자로 분리하기
>>> print(num_list)
['23', '35', '67', '88', '1']
```

84

List의 동작원리

- 우리는 파이썬 리스트가 내부적으로 어떻게 저장되고 작동하는 지를 이해하여야 한다.
- 왜냐하면 이것을 이해해야만 리스트를 올바르게 사용할 수 있기 때문이다.
- 우리가 alist라는 새로운 리스트를 만들면 파이썬은 컴퓨터 메모리에 리스트를 저장한 후에, 리스트의 주소를 alist 변수에 저장한다.
- 즉, alist는 리스트의 모든 원소를 담고있는 것이 아니라, 리스트의 원소들이 있는 곳의 저장 위치만을 가지고 있는 것이다.
- 이러한 특성으로 인해 alist가 리스트 객체를 **참조한다**reference는 표현을 사용한다

```
>>> alist = ['Kim', 'Park', 'Lee', 'Hong']
```



87

List의 동작원리 (계속)

- 여기서 blist의 인덱스 1의 원소를 변경하였음에도, alist 인덱스 1의 원소도 바뀌어 있는 것을 알 수 있다.

```
>>> blist = alist
>>> blist[1] = 'Choi' # blist의 두번째 항목값을 변경함
>>> alist
['Kim', 'Choi', 'Lee', 'Hong']
```



88

List의 동작원리 (계속)

- `id()` 함수를 통해서 메모리에 올라온 객체의 고유한 식별번호를 알아 낼 수 있다.
- 이제 `id(alist)`, `id(blist)`를 통해 `alist`와 `blist`의 아이디를 조회해보면 다음과 같이 두 객체의 아이디가 같다는 것을 알 수 있다. **(실습하는 컴퓨터 마다 다름)**



```
>>> id(alist) # 아이디 값은 이 책과 다른 값이 나올 수 있다
140050268419592
>>> id(blist) # alist의 아이디 값과 동일한 값이 나온다
140050268419592
```

89

List의 동작원리 – 기존 리스트 복사해서 새로운 리스트 생성

- 만약 동일한 내용을 가지는 새로운 리스트를 생성하고 싶은 경우에는, 단순히 등호를 써 주면 안되고 `'list()'` 함수를 사용해야 한다.
- `list()` 함수를 사용하면 원래 리스트의 복사본이 생성되며 `blist`는 이 복사본을 참조한다. 따라서 두 객체의 id는 다른 값을 가진다.
- 여기서는 140050268419592과 140050268535624라는 아이디 값이 나타나지만 이 값은 고정적이지 않으므로 여러분이 실행하면 다르게 나타날 수 있다. **(실습하는 컴퓨터 마다 다름)**

```
>>> alist = ['Kim', 'Park', 'Lee', 'Hong']
>>> blist = list(alist)
>>> id(alist) # 책의 값과 여러분이 실행한 값은 다를 수 있다
140050268419592
>>> id(blist) # blist의 아이디는 alist의 아이디와 다를 것이다.
140050268535624
```



90

Tuple

91

```
graph TD; A[파이썬 자료형] --> B[불변 속성]; A --> C[가변 속성]; B --> D["수치자료형<br/>int, float"]; B --> E[문자열<br/>str]; B --> F[튜플<br/>tuple]; C --> G[리스트<br/>list]; C --> H[집합<br/>set]; C --> I[딕셔너리<br/>dic];
```

파이썬은 여러가지 자료형이 있어서 목적에 맞게 선택해서 사용하면 됩니다.

- 파이썬의 데이터 형을 크게 분류하면 **불변속성** *immutable* 데이터 형, **가변속성** *mutable* 데이터 형으로 나눌 수 있다.
- **튜플** *tuple*은 리스트와 아주 유사하다.
- 하지만 튜플의 내용은 한 번 지정되면 변경될 수 없으며 대표적인 불변속성 자료형이다.
- 튜플은 한 번 그 내용이 정해지면 변경이 불가능하므로 구조가 단순하고 **리스트에 비하여 접근 속도가 빠르다**는 장점도 있다.
- 따라서 두 자료형은 목적에 따라서 적합한 경우에 선택하여 사용한다.

92

튜플 생성

```
>>> colors = ('red', 'green', 'blue')
>>> colors
('red', 'green', 'blue')
```

색상을 저장하는 튜플

```
>>> numbers = (1, 2, 3, 4, 5)
>>> numbers
(1, 2, 3, 4, 5)
```

다수의 정수를 저장하는 튜플

93

튜플의 속성 : Immutable

- 다시 한번 강조하면 튜플은 **변경될 수 없는 객체** `immutable`이므로 튜플의 요소는 변경될 수 없다.
- 따라서 다음과 같이 **변경을 시도하면 에러**를 출력한다.

```
>>> t1 = (1, 2, 3, 4, 5)
>>> t1[0] = 100      # 에러가 출력됨
...
t1[0] = 100
TypeError: 'tuple' object does not support item assignment
```

튜플 자료형은 한번 생성하면 그 값을
고칠 수 없다 : 오류 발생

- 튜플도 시퀀스의 일종이기 때문에 **인덱싱과 슬라이싱은 문자열이나 리스트와 동일하게 동작**한다.
- 또한 튜플 간의 결합연산인 **+**와 반복연산자 *****도 사용할 수 있다

```
>>> t1 = (1, 2, 3, 4, 5)
>>> t1[0]      # 튜플의 인덱싱-리스트 인덱싱과 동일한 방식
1
>>> t1[1:4]    # 튜플의 슬라이싱 결과로 튜플을 반환함
(2, 3, 4)
>>> t2 = t1 + t1 # 튜플의 결합 연산
>>> t2
(1, 2, 3, 4, 5, 1, 2, 3, 4, 5)
```

94

함수는 튜플을 돌려준다!

```
def sort_num(n1, n2):      # 2개의 값을 받아오는 함수
    if n1 < n2:
        return n1, n2     # n1이 더 작으면 n1, n2 순서로 반환
    else:
        return n2, n1     # n2가 더 작으면 n2, n1 순서로 반환

print(sort_num(110, 210))  # 110과 210을 함수의 인자로 전달하고 반환되는 값을 출력
print(sort_num(2100, 80))

(110, 210)
(80, 2100)
```

- 파이썬에서는 함수가 튜플을 반환하게 하면
함수가 여러 개의 값을 동시에 반환할 수 있다.

```
temp = a
a = b
b = temp
```

```
a, b = 100, 200
a, b = b, a      # 튜플을 이용하여 두 값을 교환하는 코드
print('a =', a, 'b =', b)

a = 200 b = 100
```

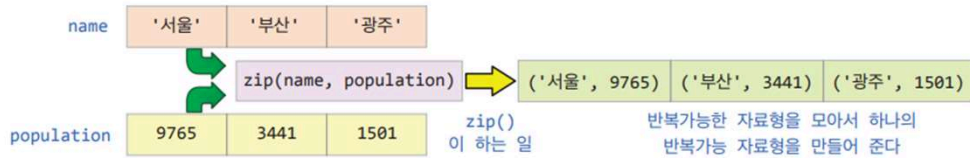
95

zip() 함수

- 파이썬에서는 리스트, 딕셔너리, 집합, 튜플과 같은 자료형이 기본적으로 제공된다.
- 이 자료형은 여러 개의 원소를 가질 수 있기 때문에 for - in 문에서 반복적으로 순환시키는 것이 가능하다.
- 이러한 이유로 이들 자료형을 **반복가능(iterable)** 자료형이라고 한다.
- 이러한 반복가능 자료형 여러 개를 넘겨주면, 이들을 합쳐서 튜플을 만들 수도 있는데 이러한 일을 하는 함수가 zip() 함수이다.
- 이 zip() 함수는 다음과 같이 여러 개의 반복가능 자료형을 받을 수 있는데, 이들을 모아서 하나의 반복가능 자료형을 만들 수 있다.
- 이것을 **집적화**라고 한다.

96

zip() 함수 : 집적화 예제



```
name = ['서울', '부산', '광주'] # 도시의 이름
population = ( 9765, 3441, 1501 ) # 도시의 인구(단위: 천명)

city_info = list(zip(name, population)) # 집적화한 자료를 리스트 자료형으로 만들기
print('city_info =', city_info)
```

```
city_info = [( '서울', 9765 ), ( '부산', 3441 ), ( '광주', 1501 )]
```

```
name = ['서울', '부산', '광주'] # 도시의 이름
population = ( 9765, 3441, 1501 ) # 도시의 인구(단위: 천명)
area = ( 605.2, 770, 501.2 ) # 도시의 면적( km^2 )

city_info = list(zip(name, population, area)) # 집적화한 자료를 리스트 자료형으로 만들기
print('city_info =', city_info)
```

```
city_info = [( '서울', 9765, 605.2 ), ( '부산', 3441, 770 ), ( '광주', 1501, 501.2 )]
```

97

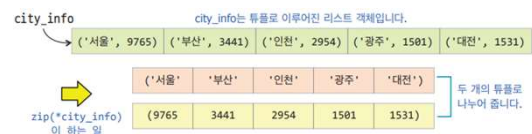
zip() 함수 : 집적화 풀기

```
city_info = [( '서울', 9765 ), ( '부산', 3441 ), ( '인천', 2954 ), ( '광주', 1501 ), ( '대전', 1531 )]
```

```
city_name, city_pop = zip(*city_info) # city_info의 튜플 원소를 나누어 새 일을 합니다.
print('city_name =', city_name)
print('city_pop =', city_pop)
```

```
city_name = ( '서울', '부산', '인천', '광주', '대전' )
city_pop = ( 9765, 3441, 2954, 1501, 1531 )
```

- 이러한 구조의 튜플은 `zip(*city_info)`라는 함수를 이용하여 (도시이름 리스트, 인구 리스트)의 쌍으로 쉽게 구분할 수 있다.
- 이때 `zip()` 함수의 내부에 `*city_info`와 같이 반드시 `*`를 사용하는 점에 유의하도록 하자.



98

zip() 함수 : 집적화 풀기 (계속)

```
max_pop = max(city_pop)
print('max_pop =', max_pop)
```

```
max_pop = 9765
```

```
n = city_pop.index(max_pop)
print('최대 인구 도시: {0}, 인구: {1} 천명'.format(city_name[n], max_pop))
```

```
최대 인구 도시: 서울, 인구: 9765 천명
```

```
min_pop = min(city_pop)
n = city_pop.index(min_pop)
print('최소 인구 도시: {0}, 인구: {1} 천명'.format(city_name[n], min_pop))
print('리스트 도시 평균 인구: {0} 천명'.format(sum(city_pop) / len(city_pop)))
```

```
최소 인구 도시: 광주, 인구: 1501 천명
리스트 도시 평균 인구: 3838.4 천명
```

99

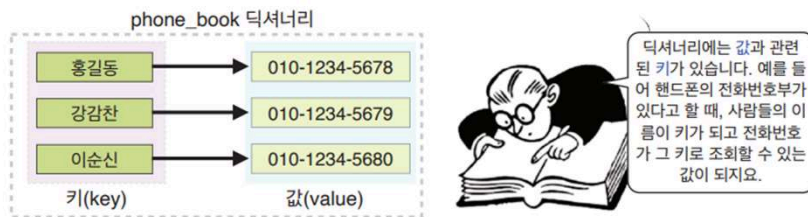
Dictionary



100

1 키와 값을 가진 딕셔너리로 자료를 저장하자

- 딕셔너리 `dictionary`도 리스트와 같이 값을 저장하는 자료구조로 파이썬에서는 기본 자료형으로 제공됨
- 하지만 딕셔너리에는 `값value`과 관련된 `키key`가 있다는 것이 큰 차이점임



- 파이썬의 딕셔너리에서는 서로 관련되어 있는 키와 값도 함께 저장되는데, 이것을 `키-값 쌍key-value pair` 이라고 함

101

1 키와 값을 가진 딕셔너리로 자료를 저장하자

- 딕셔너리를 만드는데는 몇 가지 방법이 있지만 일단 `{ }`를 이용해서 공백 딕셔너리를 생성하고 여기에 하나씩 전화번호를 추가

```
>>> phone_book = { }    # 공백 딕셔너리를 생성
```

- 공백 리스트는 대괄호 `[ ]`로 생성하고, 딕셔너리는 **중괄호 `{ }`**로 생성한다는 것에 유의

```
>>> phone_book["홍길동"] = "010-1234-5678"
```

102

1 키와 값을 가진 딕셔너리로 자료를 저장하자

- 지금까지 작성한 것을 출력

```
>>> print(phone_book)
{'홍길동': '010-1234-5678'}
```

- phone_book 딕셔너리를 출력하면 딕셔너리의 항목(item)이 쉼표로 구분되어 출력됨
- 딕셔너리를 생성하면서 동시에 초기화하는 방법도 있음

```
>>> phone_book = {"홍길동": "010-1234-5678"}
```

- 이제 이 딕셔너리에 몇 개의 다른 전화번호를 추가해서 출력해보면 다음과 같음

```
>>> phone_book["강감찬"] = "010-1234-5679"
>>> phone_book["이순신"] = "010-1234-5680"
>>> print(phone_book)
{'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}
```

103

1 키와 값을 가진 딕셔너리로 자료를 저장하자

딕셔너리의 기능을 알아보자

- 리스트와 마찬가지로 딕셔너리에는 어떤 유형의 값도 저장가능
- 아래 코드를 살펴보면 두 번째 항목에서 'Age'라는 키와 정수 27이 이 키의 값으로 사용되고 있음.

```
person_dic = {'Name': '홍길동', 'Age': 27, 'Class': '초급'}

print(person_dic['Name'])      # 딕셔너리의 'Name'이라는 키로 값을 조회함
print(person_dic['Age'])       # 딕셔너리의 'Age'이라는 키로 값을 조회함
```

```
홍길동
27
```

person_dic

Name	'홍길동'
Age	27
Class	'초급'

104

1 키와 값을 가진 딕셔너리로 자료를 저장하자

딕셔너리의 기능을 알아보자

- 딕셔너리에서 가장 중요한 연산은 **키를 가지고 연관된 값을 찾는 것**
- 주소록이 있을 경우 사람 이름을 가지고 전화번호를 찾을 수 있어야 할 것

```
>>> print(phone_book["강감찬"])  
010-1234-5679
```

- 리스트에서는 인덱스를 가지고 항목을 찾을 수 있지만 딕셔너리에서는 키가 있어야 값을 찾을 수 있음
- 딕셔너리에서 사용되는 모든 키를 출력하려면 다음과 같이 `keys()`라는 메소드를 사용함

```
>>> phone_book.keys()  
dict_keys(['홍길동', '강감찬', '이순신'])
```

`keys()` 메소드를 사용하면 키를 얻을 수 있다.

105

1 키와 값을 가진 딕셔너리로 자료를 저장하자

딕셔너리의 기능을 알아보자

- 반면 딕셔너리에서 사용되는 모든 값을 출력하려면 `values()`를 사용

```
>>> phone_book.values()  
dict_values(['010-1234-5678', '010-1234-5679', '010-1234-5680'])
```

- 그리고, 딕셔너리 내부의 모든 값을 출력하려면 `items()`를 사용할 수도 있음
- `for`문에서 `phone_book.items()`와 같은 함수를 호출하면 (키, 값) 튜플이 반환되므로 아래처럼 활용할 수 있음

```
>>> phone_book.items()  
dict_items([('홍길동', '010-1234-5678'), ('강감찬', '010-1234-5679'), ('이순신', '010-1234-5680')])  
>>> for name, phone_num in phone_book.items():  
...     print(name, ': ', phone_num)  
...  
홍길동 : 010-1234-5678  
강감찬 : 010-1234-5679  
이순신 : 010-1234-5680
```

딕셔너리의 항목들을 시퀀스로 추출할 수 있다.

106

2 딕셔너리와 리스트의 비교

- 다음의 표는 리스트와 딕셔너리를 비교하는 표
- 두 자료형이 가진 특성 중 가장 큰 차이를 보이는 인덱스와 키에 대해서 자세히 알아봄

리스트	딕셔너리
첫 번째 값의 인덱스는 0으로 시작하며 이 인덱스 값은 자동으로 1씩 증가하며 할당된다.	사용자가 지정한 키와 값이 쌍을 이룬다.
예시: lst = [11, 22, 33, 44, 55]	예: dic = {0:11, 1:22, 2:33, 3:44, 4:55}
index 01234 lst 1122334455 lst[0]lst[1]lst[2]lst[3]lst[4]	dic key01234 value1122334455 dic[0]dic[1]dic[2]dic[3]dic[4]

107

2 딕셔너리와 리스트의 비교

- 위의 lst를 생성한 후 pop(0) 메소드를 호출해 보자.
- 이 경우 아래 결과와 같이 가장 첫 번째 원소인 11이 반환되고 lst 객체의 목록에서 사라지는 것을 볼 수 있음.

```
>>> lst = [11, 22, 33, 44, 55]
>>> lst.pop(0)           # 리스트 항목의 첫 항목이 반환되고 삭제됨
>>> print('lst =', lst)
lst = [22, 33, 44, 55]
```

- 다음으로 dic이라는 딕셔너리를 생성한 후 pop(0) 메소드를 호출해 봄
- 이 경우 아래 결과와 같이 키가 0인 첫 번째 원소인 {0:11}이 반환되고 dic 객체의 목록에서 사라지는 것을 볼 수 있음

```
>>> dic = {0:11, 1:22, 2:33, 3:44, 4:55}
>>> dic.pop(0)           # 딕셔너리 항목중에서 키가 0인 항목 11이 반환되고 삭제됨
>>> print('dic =', dic)
dic = {1: 22, 2: 33, 3: 44, 4: 55}
```

108

2 딕셔너리와 리스트의 비교

- 위의 실행 코드를 통해서 리스트 객체인 lst와 딕셔너리 객체인 dic이 어떤 차이점이 있는지 다음의 표를 통해 한 번 살펴 봄

리스트	딕셔너리
<code>lst.pop(0)</code> 호출 후 <code>lst : [22, 33, 44, 55]</code>	<code>dic.pop(0)</code> 호출 후 <code>dic : {1:22, 2:33, 3:44, 4:55}</code>

109

2 딕셔너리와 리스트의 비교

- 이러한 내용을 확인하기 위하여 다음 코드를 만들고 실행시켜 봄

```
>>> print('lst[1] =', lst[1])
lst[1] = 33
>>> print('dic[1] =', dic[1])
dic[1] = 22
```

- 실행 결과는 예상한 바와 같이 나타나는 것을 볼 수 있음
- 다음으로 `lst[0]`를 출력하면 리스트의 첫 항목인 22가 출력되는 것을 볼 수 있음
- 반면 `dic[0]`를 출력하면 어떻게 될까?
이 경우 `dic[0]`에 해당하는 항목 11이 `dic.pop(0)`에 의해 삭제되었기 때문에, 오류가 출력되는 것을 볼 수 있을 것

```
>>> print('lst[0] =', lst[0]) # 리스트의 첫 항목을 출력
lst[0] = 22
>>> print('dic[0] =', dic[0]) # 주의 : 오류, 0을 키로 하는 항목이 없음
.....
KeyError: 0
```

110

2 딕셔너리와 리스트의 비교

항목의 수를 확인하고 삭제하기

- 리스트와 딕셔너리에서 모든 항목의 수를 구하는 방법, 그리고 특정한 요소 혹은 전체를 삭제하는 방법은 다음 표와 같이 큰 차이가 없음
- 각각 개수를 확인하는 함수인 len()을 사용하거나, 지정된 요소를 삭제하는 명령인 del과, 요소들을 모두 삭제하는 메소드인 clear()을 사용하면 됨
- del은 파이썬의 객체를 삭제하는 명령이라는 것을 잊지말자!

구분	리스트	딕셔너리
항목 수 확인	len(lst)	len(dic)
특정 항목 삭제	del(lst[0]) 또는 del lst[0]	del(dic[0]) 또는 del dic[0]
전부 삭제	lst.clear()	dic.clear()

111

3 딕셔너리의 다양한 메소드

메소드	하는 일
keys()	딕셔너리 내의 모든 키를 반환한다.
values()	딕셔너리 내의 모든 값을 반환한다.
items()	딕셔너리 내의 모든 항목을 [키]:[값] 쌍으로 반환한다.
get(key)	키에 대한 값을 반환한다. 키가 없으면 None을 반환한다.
pop(key)	키에 대한 값을 반환하고, 그 항목을 삭제한다. 키가 없으면 KeyError 예외를 발생시킨다.
popitem()	제일 마지막에 입력된 항목을 반환하고 그 항목을 삭제한다.
clear()	딕셔너리 내의 모든 항목을 삭제한다.

112

3 딕셔너리의 다양한 메소드

- 딕셔너리의 모든 항목을 하나씩 출력하려면 어떻게 할까?
- 리스트처럼 for 루프를 사용함
- 앞서 살펴본 바와 같이 keys() 메소드는 딕셔너리에 있는 키 항목을 시퀀스로 반환하므로, 이 값을 받아서 phone_book[key]로 접근하는 것이 가능함

```
phone_book = {'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}  
for key in phone_book.keys():  
    print(key, ': ', phone_book[key])
```

```
홍길동 : 010-1234-5678  
강감찬 : 010-1234-5679  
이순신 : 010-1234-5680
```

- 다음으로 values를 이용해서 value 값을 하나 하나 추출하여 출력

```
phone_book = {'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}  
for value in phone_book.values():  
    print('value : ', value)
```

```
value : 010-1234-5678  
value : 010-1234-5679  
value : 010-1234-5680
```

113

3 딕셔너리의 다양한 메소드

- 하지만 키와 값을 출력할 경우 다음과 같이 items()라는 메소드가 더 유용할 것
- 위의 방법과 아래의 방법이 모두 가능하지만 프로그래머의 입장에서는 보다 더 간결한 코드를 선택하는 것이 바람직함

```
phone_book = {'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}  
for name, phone_num in phone_book.items():  
    print(name, ': ', phone_num)
```

```
홍길동 : 010-1234-5678  
강감찬 : 010-1234-5679  
이순신 : 010-1234-5680
```

114

3 딕셔너리의 다양한 메소드

- get(key) 메소드는 키에 대한 값을 반환하는데, 키가 없으면 None을 반환함
- dic[key]와의 차이점을 다음 코드를 통해서 살펴보자.

```
>>> phone_book.get('홍길동')
'010-1234-5678'
>>> phone_book['홍길동']
'010-1234-5678'
>>> phone_book.get('임꺽정')      # '임꺽정' 키가 없으므로 None이 반환됨
>>> phone_book['임꺽정']          # 주의 : '임꺽정' 키가 없으므로 오류가 출력
...
KeyError: '임꺽정'
```

115

3 딕셔너리의 다양한 메소드

- 출력결과를 살펴보면 phone_book.get('임꺽정')의 경우 키인 '임꺽정'을 가지는 항목이 없기 때문에 None이 반환되지만 phone_book['임꺽정']은 오류를 출력함
- 이 때문에 phone_book.get(' 임꺽정 ')이 조금 더 안전한 코드라고 볼 수 있음
- 그리고 pop(key)는 다음과 같이 키에 대한 값을 반환하고, 그 항목을 삭제한다
- 키가 없으면 KeyError 예외를 발생시킴

```
phone_book = {'홍길동':'010-1234-5678','강감찬':'010-1234-5679','이순신':'010-1234-5680'}
phone_num = phone_book.pop('홍길동')      # '홍길동'을 키로하는 값이 반환되며 이 항목이 삭제됨
print('phone_book =', phone_book)
print('phone_num =', phone_num)
```

```
phone_book = {'강감찬': '010-1234-5679', '이순신': '010-1234-5680'}
phone_num = 010-1234-5678
```

116

3 딕셔너리의 다양한 메소드

- pop()과 달리 popitem()은 가장 마지막 항목인 이순신 항목을 (키, 값)의 튜플로 반환한 후 이 항목을 삭제함

```
phone_book = {'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}
phone_num = phone_book.popitem() # '홍길동'을 키로하는 값이 반환되며 이 항목이 삭제됨
print('phone_book =', phone_book)
print('phone_num =', phone_num)
```

```
phone_book = {'홍길동': '010-1234-5678', '강감찬': '010-1234-5679'}
phone_num = ('이순신', '010-1234-5680')
```

117

3 딕셔너리의 다양한 메소드

- sorted() 함수는 phone_book.items()와 같이 키, 값의 튜플 쌍을 받아 이를 정렬할 수 있음
- 키, 값의 튜플 쌍 중에서 첫 번째 항목인 키에 대하여 정렬할 것인지, 두 번째 항목인 값에 대하여 정렬할 것인가를 key 매개변수를 사용하여 결정한다.
- 이를 위하여 key = lambda x: x[0]와 같은 람다 표현식을 사용한다.

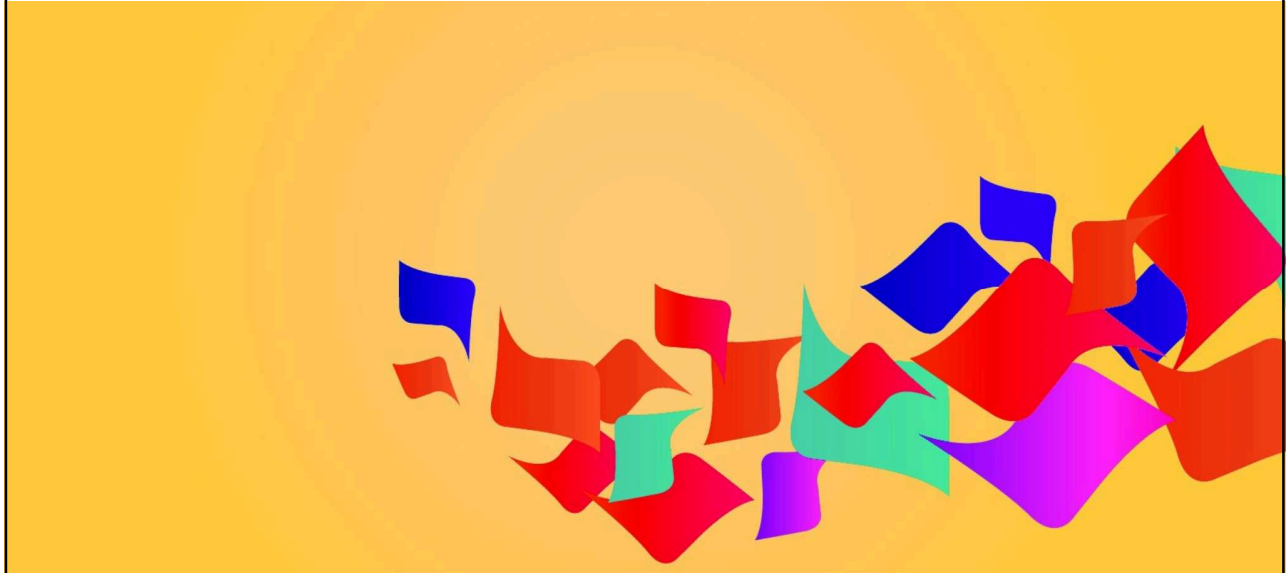
```
>>> phone_book={'홍길동': '010-1234-5678', '강감찬': '010-1234-5679', '이순신': '010-1234-5680'}
>>> sorted(phone_book) # 딕셔너리를 키를 기준으로 정렬하여 리스트를 반환
['강감찬', '이순신', '홍길동']
>>> sorted_phone_book = sorted(phone_book.items(), key=lambda x: x[0])
>>> print(sorted_phone_book)
[('강감찬', '010-1234-5679'), ('이순신', '010-1234-5680'), ('홍길동', '010-1234-5678')]
```

- 그리고 딕셔너리의 모든 항목을 삭제하려면 다음과 같이 clear() 메소드를 사용함

```
>>> phone_book.clear()
>>> print(phone_book)
{}
```

118

Set

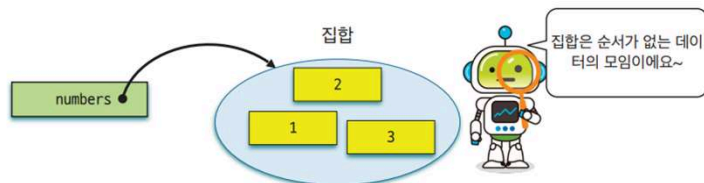


119

1 순서가 중요하지 않은 대상들이 모이면 : 집합

- 파이썬에서 사용하는 **집합set** 역시 수학의 집합과 비슷하며, 튜플과 달리 순서가 없는 자료형임
- 특히 동일한 값을 가지는 항목의 중복이 허용되지 않는다는 특징이 있으며, *교집합, 합집합, 차집합, 대칭차집합* 등의 다양한 집합 연산을 수행 가능

```
>>> numbers = {2, 1, 3} # 숫자 3개로 이루어진 집합 자료형
>>> numbers
{1, 2, 3}
```



120

1 순서가 중요하지 않은 대상들이 모이면 : 집합

- 다음과 같이 리스트로부터 집합을 생성하는 것도 가능
- 하지만 집합자료형에서는 **요소가 중복되면 자동으로 중복된 요소를 제거**

```
>>> set([1, 2, 3, 1, 2]) # 리스트로부터 집합을 생성함
{1, 2, 3}
```

- 그리고 문자열로부터 집합을 생성하는 것도 가능한데,
이때는 각 문자들이 하나의 요소가 됨

```
>>> set("abcdefa") # 문자열도 시퀀스형이라서 집합형으로 변환이 가능하다
{'f', 'a', 'b', 'e', 'c', 'd'}
```

- 비어 있는 집합을 생성하려면 다음과 같이 set() 함수를 사용

```
>>> numbers = set() # 비어있는 집합 생성
```

121

1 순서가 중요하지 않은 대상들이 모이면 : 집합

- 리스트의 경우 + 연산으로 두 리스트를 결합시키는 것이 가능하지만
집합은 이 연산을 지원하지 않는다. 따라서
- {1, 2} + {1, 2, 3} 연산은 아래와 같이 TypeError를 출력

```
>>> n_list = [1, 2] + [1, 2, 3]
>>> print('n_list =', n_list)
n_list = [1, 2, 1, 2, 3]
>>> n_set = {1, 2} + {1, 2, 3}
....
TypeError: unsupported operand type(s) for +: 'set' and 'set'
```

122

1 순서가 중요하지 않은 대상들이 모이면 : 집합

집합의 항목에 접근하는 연산

- 어떤 항목이 집합 안에 있는지를 검사하려면 리스트와 마찬가지로 in 연산자를 사용

```
numbers = {2, 1, 3}
if 1 in numbers:    # 1이라는 항목이 numbers 집합에 있는가 검사
    print("집합 안에 1이 있습니다.")
else:
    print("집합 안에 1이 없습니다.")
```

```
집합 안에 1이 있습니다.
```

- 집합의 항목은 순서가 없기 때문에 인덱스를 가지고 접근할 수는 없음
- 하지만 for 반복문을 이용하여 각 항목들에 접근할 수 있음

```
numbers = {2, 1, 3}
for x in numbers:
    print(x, end=" ")
```

```
1 2 3
```

123

1 순서가 중요하지 않은 대상들이 모이면 : 집합

집합의 항목에 접근하는 연산 (계속)

- 여기서 주의할 점은 항목들이 출력되는 순서는 입력된 순서와 다를 수도 있다는 점
- 만약 정렬된 순서로 항목을 출력하기를 원한다면 다음과 같이 sorted() 함수를 사용하면 됨

```
for x in sorted(numbers):
    print(x, end=" ")
```

```
1 2 3
```

124

1 순서가 중요하지 않은 대상들이 모이면 : 집합

집합의 항목에 접근하는 연산 (계속)

- 집합의 요소에는 인덱스가 없기 때문에 인덱싱이나 슬라이싱 연산은 의미가 없음
- 우리는 add() 메소드를 이용하여서 하나의 요소를 추가할 수 있음

```
>>> numbers = {1, 2, 3}
>>> numbers.add(4)      # numbers 집합에 원소 4를 추가
>>> numbers
{1, 2, 3, 4}
```

- 또한 집합의 요소를 삭제할 때는 remove() 메소드를 사용할 수 있음

```
>>> numbers.remove(4)   # numbers 집합에 원소 4를 삭제
>>> numbers
{1, 2, 3}
```

125



잠깐 - in 연산자는 집합에만 사용하는 것은 아니다

이 절에서 사용한 in 연산자는 집합에만 사용할 수 있는 연산자가 아니라 여러 항목을 가진 다양한 데이터에 적용할 수 있다. 대표적인 예로 리스트에 특정한 항목이 있는지도 다음과 같이 검사할 수 있다.

```
>>> a_list = ['hello', 'world', 'welcome', 'to', 'python']
>>> 'python' in a_list    # 'python' 문자열 항목이 a_list에 있는가 조사한다
True
```

126

2 집합에 적용할 수 있는 다양한 연산들을 살펴보자

집합 비교 연산

- 2개의 집합이 서로 같은지도 검사가능
- 이것은 ==과 != 연산자를 사용하는 것이 가장 쉬움

```
>>> A = {1, 2, 3}
>>> B = {1, 2, 3}
>>> A == B          # A가 B와 같은지 검사
True
```

- < 연산자와 <= 연산자를 사용하면 집합이 진부분집합인지, 부분집합인지를 검사 가능

```
>>> A = {1, 2, 3, 4, 5}
>>> B = {1, 2, 3}
>>> B < A           # B가 A의 진부분집합인가 검사
True
```

127

2 집합에 적용할 수 있는 다양한 연산들을 살펴보자

항목들에 대한 정보 처리

- 집합에 대해서도 len(), max(), min(), stored(), sum() 등의 메소드는 사용할 수 있음
- 아래와 같이 6개의 항목을 가지고 집합을 만들수 있음

```
>>> a_set = {1, 5, 4, 3, 7, 4 }   # 6개 항목으로 집합 생성
>>> len(a_set)                   # 항목의 개수는 중복을 제외하면 5개이다
5
>>> max(a_set)                   # 항목 가운데 가장 큰 수는 7
7
>>> min(a_set)                   # 항목 가운데 가장 작은 수는 1
1
>>> sorted(a_set)                # 항목을 정렬하여 리스트 만든다. 집합이므로 중복은 제거한다
[1, 3, 4, 5, 7]
>>> sum(a_set)                   # 중복 원소는 하나만 사용되므로 전체 합은 20
20
```

128

2 집합에 적용할 수 있는 다양한 연산들을 살펴보자

집합에 적용할 수 있는 논리 연산

- 다수의 항목을 가진 데이터에 대해 적용할 수 있는 논리 연산으로 `all()`과 `any()` 함수가 있음
- 이 함수는 인자로 주어진 데이터의 항목들 각각에 대해 부울형 평가를 한 결과를 종합적으로 반환함

```
>>> a_set = { 1, 0, 2, 3, 3}
>>> all(a_set)    # a_set이 모두 True인가를 검사한다
False
>>> any(a_set)    # a_set에 True인 것이 하나라도 있는가를 검사한다
True
```

129

2 집합에 적용할 수 있는 다양한 연산들을 살펴보자

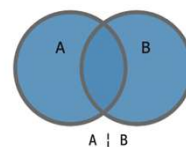
합집합, 교집합, 차집합, 대칭차집합 연산 (계속)

- 집합이 유용한 이유는 교집합이나 합집합과 같은 여러가지 집합 연산을 지원하기 때문
- 이것은 연산자나 메소드로 수행할 수 있음

```
>>> A = {1, 2, 3}
>>> B = {3, 4, 5}
```

- **합집합**은 다음과 같이 2개의 집합을 합하는 연산으로 `|` 연산자나 `union()` 메소드를 사용한

```
>>> A | B    # 합집합 연산
{1, 2, 3, 4, 5}
>>> A.union(B)  # 합집합 메소드
{1, 2, 3, 4, 5}
```



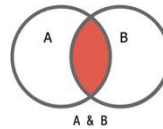
130

2 집합에 적용할 수 있는 다양한 연산들을 살펴보자

합집합, 교집합, 차집합, 대칭차집합 연산 (계속)

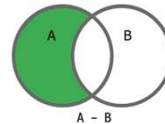
- **교집합**은 2개의 집합에서 겹치는 요소를 구하는 연산임
- 교집합은 `&` 연산자나 `intersection()` 메소드를 사용함

```
>>> A & B      # 교집합 연산
{3}
>>> A.intersection(B) # 교집합 메소드
{3}
```



- **차집합**은 하나의 집합에서 다른 집합의 요소를 빼는 것
- 차집합은 `-` 연산자나 `difference()` 메소드를 사용

```
>>> A - B      # 차집합 연산
{1, 2}
>>> A.difference(B) # 차집합 메소드
{1, 2}
```



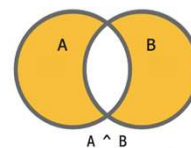
131

2 집합에 적용할 수 있는 다양한 연산들을 살펴보자

합집합, 교집합, 차집합, 대칭차집합 연산 (계속)

- **대칭차집합**은 두 집합의 합집합에서 교집합을 뺀 요소를 구하는 연산
- 대칭차집합은 `^` 연산자나 `symmetric_difference()` 메소드를 사용함

```
>>> A ^ B      # 대칭 차집합 연산
{1, 2, 4, 5}
>>> A.symmetric_difference(B)
{1, 2, 4, 5}
```



132

두 수의 약수와 최대공약수 그리고 프로그래밍적인 사고

- 두 정수의 약수를 구하는 방법을 생각해 보자.
- 10의 약수는 1, 2, 5, 10 이 될 수 있는데 이 중에서 1과 자기 자신인 10을 제외한 2와 5를 진약수라고 한다.
- 다음과 같이 리스트를 만들고 2, 3, 4, 5..9까지의 모든 수를 10으로 나누어 그 나머지 값이 0이 되는지를 확인해보고 그렇다면 리스트에 담아 두면 된다.

```
num = 10
divisors = []

for i in range(2, num):
    if num % i == 0:
        divisors.append(i)

print(num, '의 진약수 :', divisors)
```

```
10 의 진약수 : [2, 5]
```

133

두 수의 약수와 최대공약수 그리고 프로그래밍적인 사고

- 이 코드를 활용하여 48과 60의 최대공약수를 구하려면 48과 60의 진약수를 구한 다음 공통된 값을 구하고 그 중에서 가장 큰 값을 찾는 방식으로 최대공약수를 구해보자.
- 이를 위하여 get_divisors() 함수 내에 divisors라는 빈 집합 자료형을 선언하고 num % i 가 0이 되는 값을 이 집합에 추가하도록 하자.

```
def get_divisors(num):    # num의 약수를 집합형으로 반환함
    divisors = set()
    for i in range(2, num):
        if num % i == 0:
            divisors.add(i)
    return divisors

x = 48
print(x, '의 진약수 :', get_divisors(x))
y = 60
print(y, '의 진약수 :', get_divisors(y))
```

```
48 의 진약수 : {2, 3, 4, 6, 8, 12, 16, 24}
60 의 진약수 : {2, 3, 4, 5, 6, 10, 12, 15, 20, 30}
```

134

두 수의 약수와 최대공약수 그리고 프로그래밍적인 사고

- 이제 이 약수 중에서 공통된 약수를 구해보자.
- 그리고 이 값 중 가장 큰 값을 max() 함수를 이용해서 구해보자.

```
A = get_divisors(x)
B = get_divisors(y)

print(A.intersection(B))
print(x, y, '의 최대공약수 :', max(A.intersection(B)))
```

```
{2, 3, 4, 6, 12}
48 60 의 최대공약수 : 12
```

- 물론 이 방법보다 더 빠르게 두 수의 최대공약수를 구하는 유클리드의 방법이 있다.
- 이와 같이 문제를 해결하는 단계적 과정을 통해서 해를 구하는 것이 바로 **프로그래밍적인 사고 방식**일 것이다.

135

리스트, 튜플, 집합, 딕셔너리를 비교하자

파이썬의 중요한 자료구조들

- **리스트**: 리스트는 여러 가지 유형의 자료값을 한꺼번에 담을 수 있으며 순서를 가지고 있기 때문에 0,1,2,...와 같은 인덱스를 이용해서 각각의 항목에 접근가능
- 이러한 특징을 가지는 자료구조를 **순서형** *ordered* 자료구조라고 함
- 리스트 내부의 자료값은 인덱스를 이용하여 참조할 수 있으며 변경하는 것이 가능한데 이러한 유형의 자료형을 **변경가능** *mutable* 자료형이라고 함
- **튜플**: 튜플 역시 여러 가지 유형의 자료값을 한꺼번에 담을 수 있으며 순서를 가지고 있는 순서형 자료구조.
- 하지만 리스트와는 달리 그 자료값을 변경하는 것이 불가능
- 이러한 특징을 가지는 자료형을 **변경불가능** *immutable* 자료형이라고 함

136

리스트, 튜플, 집합, 딕셔너리를 비교하자

파이썬의 중요한 자료구조들 (계속)

- **집합**: 집합은 여러 가지 자료값을 한 꺼번에 담을 수 있으나 순서가 존재하지 않고 중복된 자료값을 허용하지 않음
- 순서가 존재하지 않기 때문에 **비순서형** `unordered` 자료구조라고 함
- 리스트의 원소들 중에서 중복된 원소들을 빠르게 제거하기 위해서는 리스트 자료형을 집합 자료형으로 변경한 후 다시 리스트 형으로 변경하면 됨
- **딕셔너리**: 딕셔너리는 키-값 쌍으로 자료값이 저장되는 특징이 있다. 딕셔너리는 유일한 키를 통해서 값에 접근 가능하다. 딕셔너리의 키는 문자열, 정수, 튜플 등이 될 수 있음

137

리스트, 튜플, 집합, 딕셔너리를 비교하자

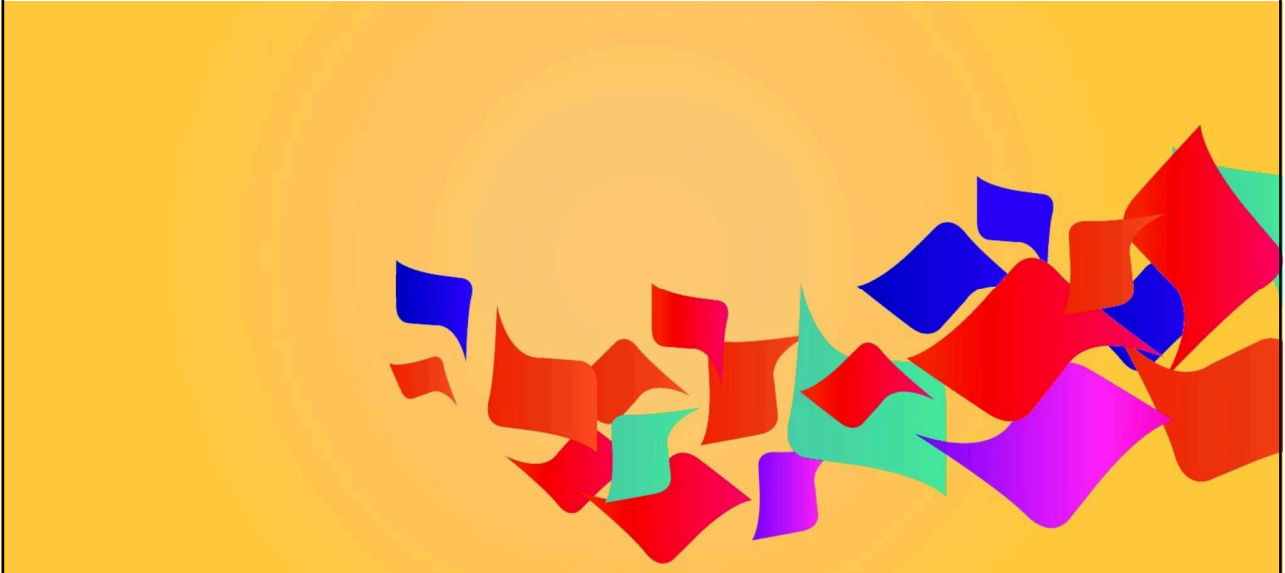
파이썬의 중요한 자료구조들 (계속)

자료구조	순서형	변경가능형	생성하기	예제
리스트	O	O	[] 또는 <code>list()</code>	[3, 4.5, 'hello', (4, 5, 6), 0]
튜플	O	X	() 또는 <code>tuple()</code>	(3, 4.5, 'hello', (4, 5, 6), 0)
집합	X	O	{ } 또는 <code>set()</code>	{ 3, 4.5, 'hello' }
딕셔너리	X	O	{ } 또는 <code>dict()</code>	{ 'Jan': 31, 'Feb': 28, 'March': 31 }

- 주의할 사항: 집합과 딕셔너리의 생성 방법은 { }로 동일함
- 일반적으로 $a = \{1, 2, 3\}$ 과 같이 항목값이 있을 경우 이 자료형은 집합이지만, $a = \{\}$ 와 같이 항목값이 없을 경우 디폴트로 원소가 없는 딕셔너리 형이 됨.
- 순서형 자료구조는 순서가 있는 인덱스를 이용하여 개별 항목값에 접근하거나 슬라이싱하는 것이 가능
- 변경가능형 자료구조는 내부의 항목값을 변경시키는 것이 가능함

138

Object oriented programming



139

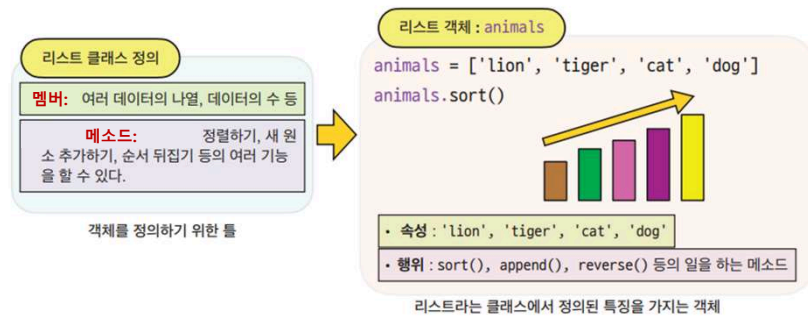
파이썬은 '객체지향 프로그래밍 언어'이다

- 프로그램 코드가 복잡해지면 이를 관리하기가 쉽지 않다.
- 이러한 어려움을 해결하기 위해 소프트웨어 엔지니어들은 수십년간 다양한 방법으로 해결책을 찾았으며 그 중에서 가장 뛰어난 해결책으로 인정 받고 있는 방법이 바로 **객체지향 프로그래밍** object oriented programming 방식이다
- 객체지향 프로그래밍은 다양한 객체를 **클래스** class로 미리 정의해두고 이 객체들이 프로그램 상에서 상호작용하면서 원하는 작업을 수행하는 문제해결 방식이다.
- 앞서 다루어 본 자료형인 **리스트**라는 것도 일종의 **클래스**이다.
- 우리는 이 리스트라는 클래스의 특징을 가지는 객체를 만들 수 있다.
- 이 각각의 **객체**들은 **클래스**의 틀을 지키면서 **각자 서로 다른 실제 값을 담고 있다**.
- 이런 객체를 **인스턴스** instance 객체라고 한다.
- 파이썬은 '**객체지향 프로그래밍 언어**'이다.
- 그리고 이미 살펴본 **자료형, 함수, 모듈**은 모두 **객체**이다.

140

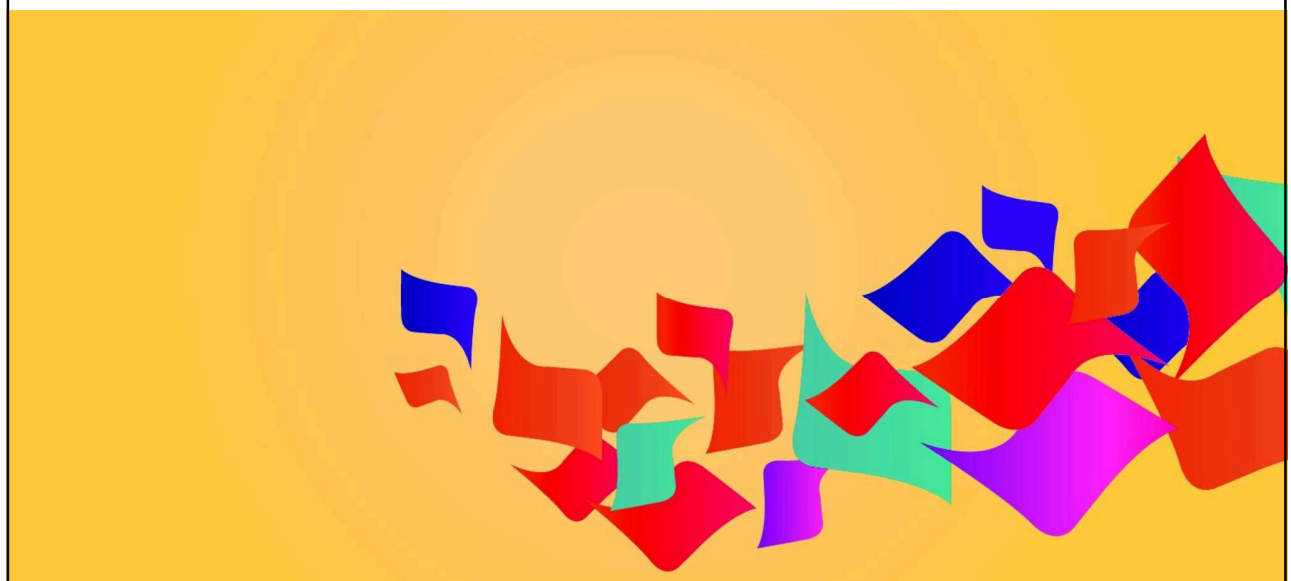
파이썬은 '객체지향 프로그래밍 언어'이다.

- 이렇게 특정한 클래스에 속한 객체들이 사용할 수 있는 함수들을 해당 클래스의 **메소드 method**라고 부른다.
- 객체 지향 언어는 리스트와 같은 클래스를 정의할 수 있고
- 해당 클래스의 인스턴스 객체를 생성할 수 있으며
- 이들이 메소드를 이용하여 다양한 일을 할 수 있도록 지원하는 언어를 의미한다.



141

파일처리



142

파일로부터 자료를 읽고 저장해보자

- 컴퓨터 **파일**file이란 컴퓨터의 저장 장치 내에 데이터를 저장하기 위해 사용하는 논리적인 단위를 말한다.
- 파일은 **하드 디스크**hard disk나 **외장 디스크**external disk 같은 저장 장치에 저장한 후 필요할 때 다시 불러서 사용하는 것이 가능하며, 필요에 따라서 수정하는 것도 가능하다.
- 파일에는 여러 종류가 있으며 일반적으로 마침표(.)문자 뒤에 py, txt, doc, hwp, pdf와 같은 확장자를 붙여서 파일의 종류를 구분한다.

143

파일로부터 자료를 읽고 저장해보자

• 파일 입출력을 위한 절차

- ① open() 함수를 이용하여 지정된 파일의 객체를 생성
- ② 생성된 파일 객체로부터 데이터를 읽어 들이거나 출력합니다.
- ③ close() 함수로 지정된 파일 객체의 사용을 종료

144

파일로부터 자료를 읽고 저장해보자

• `open()` 함수

- 파일을 열어 파일 객체를 반환하는 함수
- 매개변수로 파일을 지정
- 지정된 파일이 없을 경우 자동으로 새로운 파일을 생성

▶ 형식 파일객체 = `open(파일이름, 이용형태)`

또는

`with open(파일이름, 이용형태) as 파일객체 :`
 **# 파일 처리 블록**

[표 7-1] `open()` 함수의 이용형태 모드

지정 값	모드	설명
r	읽기 모드	기존의 파일로 부터 읽어 들임. 파일이 없는 경우 오류 발생. 모드를 지정하지 않는 경우 묵시적으로 읽기모드로 설정됨.
w	쓰기 모드	파일에 데이터를 저장하기 위한 모드. 지정된 파일을 생성하여 데이터를 저장. 지정된 파일이 이미 존재하는 경우 그 파일의 내용을 모두 삭제하고 새로 저장.
a	추가 모드	지정된 파일의 끝에 데이터를 추가하여 저장. 지정된 파일이 없는 경우 새로 생성하여 데이터를 추가.

145

파일로부터 자료를 읽고 저장해보자

• 입출력을 수행하기 위해 파일을 `open()`하고 `close()`하는 예

```
# open()을 사용하는 예
f=open("test.txt", 'w')      # open()을 사용하여 파일 객체 생성
.....
.....                        # 입출력 수행
.....
f.close()                    # 파일 close()

# with open()을 사용하는 예
with open("test.txt", 'w') as f :    # with open()을 사용하여 파일 객체 생성
    .....
    .....                        # with 블록 내에서 입출력 수행
    .....                        # 파일 close() 생략 가능
```

146

파일로부터 자료를 읽고 저장해보자

- 파일에 텍스트를 입출력하기 위한 메소드
 - write() 메소드, read() 메소드

147

파일로부터 자료를 읽고 저장해보자



```
*file_write_hello.py - C:/workspace/file_write_hello.py (3.11.3)*
File Edit Format Run Options Window Help
f = open('hello.txt', 'w')      # 파일을 쓰기 모드로 열기
f.write('Hello World!')        # hello.txt 파일에 쓰기
f.close()                      # 파일을 닫는다
Ln: 5 Col: 0
```

- 이 코드는 open() 이라는 명령을 통해서 'hello.txt' 파일을 열게 되는데 뒤에 나타나 는 'w' 인자에 의해서 파일을 쓰기 모드로 열게 된다.
- 이렇게 만든 파일 객체 f 는 f.write() 명령을 통해서 'hello world!' 라는 문자열을 현재 디렉토리의 hello.txt 라는 파일에 쓰고 모든 작업을 마친 후 f.close() 를 통해서 파일 쓰기 작업을 종료한다.

148

파일로부터 자료를 읽고 저장해보자

- 역시 `open()` 함수를 사용하여야 하지만 'w' 인자가 아닌 'r' 인자를 사용해야 읽기 모드 (read mode)로 파일을 읽을 수 있다.
- 성공적으로 파일 읽기가 완료되면 다음과 같이 파일의 내용을 화면에서 볼 수 있다.

```
f = open('hello.txt', 'r') # 파일을 연다.  
s = f.read()              # hello.txt 파일을 읽는다.  
print(s)                  # 파일의 내용을 출력한다.  
f.close()                  # 파일을 닫는다.  
  
Hello World!
```

- 파일에 여러 줄의 내용이 있을 경우 `for`문을 사용할 수 있다.

149

String(문자열 처리)



150

파이썬을 이용한 텍스트 처리 방법을 알아보자

- 우리는 텍스트 처리를 통하여 문서에서 어떤 특정한 정보를 추출한다거나 글을 읽고 감정을 분석할 수 있다.
- ChatGPT와 같은 챗봇을 훈련시키려면 우리가 일상 생활에서 접하는 텍스트 데이터를 학습에 적합하도록 가공하고 처리하여야 한다.
- 수많은 원천 데이터들은 텍스트 형태로 저장되어 있다.
- 따라서 텍스트를 읽어서 처리하는 것은 아주 중요하다.
- 파이썬을 사용한 텍스트 처리에는 텍스트 데이터를 정리, 전처리 및 분석하는 일련의 작업이 포함된다.

151

파이썬을 이용한 텍스트 처리 방법을 알아보자(계획)

- 다음은 파이썬에서 쉽게 할 수 있는 몇 가지 기본 텍스트 처리 작업이다.
- **문자열 다루기:** 문자열은 파이썬에서 기본적인 데이터 타입 중 하나이며, 문자열 다루기는 텍스트 처리에서 가장 기본적인 기능 중 하나이다. 문자열 다루기에는 문자열 자르기, 합치기, 변환하기, 포맷팅하기 등이 포함된다.
- **데이터 전처리:** 텍스트 데이터를 분석하려면 데이터에 대한 전처리가 필요하다. 이 전처리 작업에는 토큰화, 어간 추출(Stemming/Lemmatization), 불용어 제거 등이 포함된다. 파이썬에서는 NLTK와 같은 라이브러리를 사용하여 데이터 전처리를 수행할 수 있다.
- **정규식:** 정규식은 문자열 패턴을 검색하고 추출하는 데 사용되는 강력한 도구이다. 파이썬에서는 re 모듈을 사용하여 정규식을 지원한다.
- 파이썬의 기본 라이브러리에서 제공하는 문자열 함수들만 이용하여도 텍스트 데이터를 어느 정도 처리할 수 있지만 BeautifulSoup, csv, json, nltk와 같은 **우수한 외부 모듈**을 사용하면 비교적 쉽게 텍스트를 처리하고 분석할 수 있다.

152

기본적인 텍스트 처리

문자열을 문자로 분해해보자

- 문자열에서 사용할 수 있는 가장 기본적인 작업은 아마도 문자열 안에 있는 개별 문자들을 추출하는 작업일 것임
- 문자열 안에 저장된 문자들은 리스트와 마찬가지로 0부터 시작하는 수치 값으로 접근가능하며, 이를 이용하여 원하는 문자를 추출할 수 있음. 이 번호를 **인덱스**라고 부름
- 이 인덱스는 0과 양의 정수를 사용할 수도 있으나 그림과 같이 음의 정수도 사용가능 함

	[6:10] : 슬라이싱											
인덱스	0	1	2	3	4	5	6	7	8	9	10	11
원본 문자열	M	o	n	t	y		P	y	t	h	o	n
음수 인덱스	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
	[-12:-7] : 슬라이싱											

153

기본적인 텍스트 처리

문자열을 문자로 분해해보자 (계속)

- 예를 들어 'Monty Python'이라는 문자열이 변수 s에 저장되어 있다고 하자.

```
>>> s = 'Monty Python'
>>> s[0]          # 인덱싱
'M'
```

- 위의 예제와 같이 s[0]과 같이 인덱스 0을 이용해서 문자 'M'에 접근 가능하며, s[11]은 마지막 문자 'n'이 된다.

154

기본적인 텍스트 처리

문자열을 문자로 분해해보자 (계속)

- 그리고 여러 개의 문자를 동시에 선택하려면 `s[6:10]`과 같은 콜론(:) 표기법을 사용함.
- 이 표기법은 `s[6]`에서 `s[10-1]`까지의 문자를 모두 선택한다는 의미인데 이를 **슬라이싱**이라고 한다.

```
>>> t = s[6:10]    # 슬라이싱
>>> t
'Pyth'
```

- 문자열의 각 문자를 리스트 요소로 분리하려면 다음과 같이 `list()` 함수를 사용한다.

```
>>> my_string = "Hello"
>>> characters = list(my_string)    # 문자열을 리스트로 변환시킴
>>> print(characters)               # ['H', 'e', 'l', 'l', 'o']
['H', 'e', 'l', 'l', 'o']
```

155

기본적인 텍스트 처리

`split()` 메소드로 문자열을 단어로 나누어보자

- 문자열을 단어들의 리스트로 바꾸는 것은 문자열 처리에서 가장 중요한 작업 중의 하나이다.
- 문자열을 단어들의 리스트로 바꾸는 메소드는 `split()`이다.
- `split()`는 **주어진 분리자 문자를 이용하여 문자열을 단어들로 분리**한다.
- 별도의 분리자 문자를 지정하지 않을 경우 공백 문자를 이용하여 분리한다.
- 예를 들어 다음과 같은 문자열 `s`에 `split()`이라는 메소드를 사용하면 공백을 구분자로 사용하여 하나의 문자열을 3개의 문자열로 분리할 수 있다.

```
>>> s = 'Welcome to Python'
>>> s.split()
['Welcome', 'to', 'Python']
```

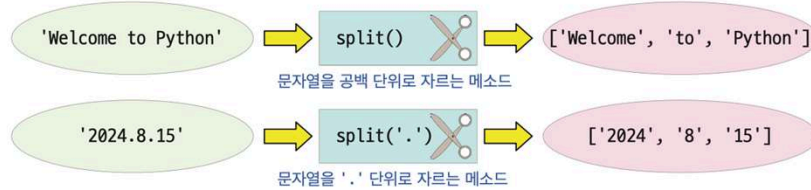
156

기본적인 텍스트 처리

split() 메소드로 문자열을 단어로 나누어보자 (계속)

- 반면 다음과 같이 마침표로 구분된 연, 월, 일이 있을 경우 마침표(.)를 split() 메소드의 인자로 주면 마침표 단위로 문자를 구분할 수 있다.

```
>>> s = '2024.8.15'
>>> s.split('.') # 분리문자를 명시하여 분리함
['2024', '8', '15']
```



157

기본적인 텍스트 처리

join() 메소드로 단어들을 붙여서 문자열을 만들어보자

- `split()`가 문자열을 부분 문자열들로 분리하는 함수라면
- `join()`은 반대로 **부분 문자열들을 모아서 하나의 문자열로 만드는 역할**을 하는 함수이다. `join()`을 호출할 때는 접착제 역할을 하는 문자를 지정할 수 있다.



- 다음 코드로 이 내용을 확인해 보자.

```
>>> ','.join(['apple', 'grape', 'banana'])
'apple,grape,banana'
```

쉼표를 이용하여 세 단어를 연결함.

```
>>> '-'.join('010.1234.5678'.split('.')) # .으로 구분된 전화번호를 하이픈(-)으로 고치기
'010-1234-5678'
```

158

기본적인 텍스트 처리

부분 문자열의 출현 횟수 계산

- 문자열의 `count()` 메소드는 문자열 중에서 특정 문자열이 등장하는 횟수를 반환한다.

```
>>> s = 'www.booksr.co.kr'      # 생능출판사의 홈페이지
>>> s.count('.')                # . 이 몇 번 나타나는가를 알려준다
3
```

159

기본적인 텍스트 처리

`startswith()`와 `endswith()`

- `startswith()`, `endswith()`은 문자열이 특정 문자열로 시작하는지 또는 끝나는지 여부를 확인한다.

```
>>> s = "Hello, World!"
>>> s.startswith("Hello")      # s 문자열이 "Hello"로 시작하는가
True
```

160

텍스트를 변경하고 포매팅하는 방법

- 문자열에서 대문자를 소문자로 변경하는 함수는 `lower()`이고
- 반대는 대문자로 변경하는 메소드는 `upper()` 함수이다.
- 첫 번째 문자만 대문자로 변환하는 함수는 `capitalize()`이다.

```
>>> s = 'Hello, World!'
>>> s.lower()    # s 문자열을 소문자로 변환
'hello, world!'
>>> s.upper()    # s 문자열을 대문자로 변환
'HELLO, WORLD!'
```

- 문자열에서 특정 문자열을 다른 문자열로 치환하려면 `replace()` 메소드를 사용한다.

```
>>> s = "Hello, World!"
>>> s.replace("World", "Python")
'Hello, Python!'
```

161

텍스트를 변경하고 포매팅하는 방법

- 텍스트 데이터를 처리하는 동안 마침표나 쉼표와 같은 구두점들은 별 도움이 되지 않는 경우도 많다.
- 구두점을 제거하면 훈련 데이터의 크기를 줄이는 데 도움이 된다.
- 이 경우 `replace()` 메소드를 사용하면 좋다. 예를 들어 느낌표를 제거하는 코드는 다음과 같다.

```
>>> s = 'Hello, World!'
>>> s.replace('!', '')
'Hello, World'
```

162

텍스트를 변경하고 포매팅하는 방법

공백 삭제

- 문자열 데이터를 처리할 때 문자열에서 원치 않는 공백을 제거하는 작업은 `strip()` 함수가 담당한다.
- `strip()`은 문자열의 첫 부분과 끝부분에서 공백 문자만을 제거하며, 문자 사이의 공백은 제거하지 않는다.
- 반면 `lstrip()`, `rstrip()`은 각각 문자열의 **왼쪽**, **오른쪽**의 공백 문자를 제거하여 다음과 같은 결과를 만들어낸다.



163

텍스트를 변경하고 포매팅하는 방법

공백 삭제 (계속)

- 만일 문자열의 앞과 뒤에 있는 특정한 문자를 삭제하려면 문자를 `strip()`의 인자로 이 문자를 전달한다.

```
>>> s = '#####this is an example#####'
>>> s.strip('#') # 문자열의 앞 뒤에 있는 해시 문자를 제거한다
'this is an example'
```

- `rstrip()`과 `lstrip()`에도 특정한 문자를 인자로 넣어줄 수 있다.

```
>>> s = '#####this is an example#####'
>>> s.lstrip('#')
'this is an example#####'
>>> s.rstrip('#')
'#####this is an example'
```

164

텍스트를 변경하고 포매팅하는 방법

부분 문자열 찾기, 그리고 다양한 포매팅

- `find()` 메소드는 문자열에서 지정된 부분 문자열을 찾아서 그 인덱스를 반환한다.
- 지정된 문자를 찾지 못했을 경우에는 `-1`을 반환한다.
- 문자열 중에서 관심 있는 부분을 찾을 때 `find()` 함수를 사용하면 좋다.

```
>>> s = 'www.booksr.co.kr'
>>> s.find('.kr')
13
>>> s.find('x')      # 'x' 문자열이 없을 경우 -1을 반환함
-1
```

165

텍스트를 변경하고 포매팅하는 방법

부분 문자열 찾기, 그리고 다양한 포매팅 (계속)

- `index()` 메소드도 문자열 내부의 부분 문자열 또는 문자의 인덱스를 반환한다.
- 부분 문자열이나 문자를 찾을 수 없으면 예외가 발생한다.
- 예를 들어 문자 `b`가 `s`의 몇 번째에 있는가는 다음과 같은 `index()` 메소드로 조회할 수 있다.

```
>>> s = 'www.booksr.co.kr'
>>> n = s.index('b')
>>> print('s의 b 인덱스 =', n)      # 'b' 문자열이 없을 경우 -1을 반환함
s의 b 인덱스 = 4
```

166

불용어 제거

실습 시간



불용어(stop words)는 텍스트 마이닝, 자연어 처리 등의 작업에서 주로 이용하는 단어이다. 이 단어란 문장에서 특별히 고려할 필요가 없는 단어들을 말한다. 예를 들어, "is", "and", "the" 등과 같은 매우 빈번하게 등장하는 조사, 접속사, 관사 등이 불용어에 해당한다. 불용어를 제거하는 이유는 불용어들이 텍스트에서 특정 의미를 가지지 않기 때문이다. 이를 제거하여 문장에서 특정한 의미를 가진 단어들만 남겨두는 것이 좋기 때문이다. 예를 들어 "This is an example of removing stop words from a string."라는 문자열이 있다면 이 문자열을 다음과 같이 변경하는 것이다.

원하는 결과

입력=This is an example of removing stop words from a string.
출력=This example removing stop words string.

힌트

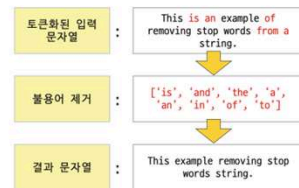


해답 코드



```
stop_words = ['is', 'and', 'the', 'a', 'an', 'in', 'of', 'to']

text = "This is an example of removing stop words from a string."
words = text.split()
filtered_sentence = [word for word in words if word.lower() not in stop_words]
print(f"입력={text}")
print(f"출력='{"".join(filtered_sentence)}'")
```



순수한 파이썬 코드만으로 이 기능을 구현한다면 불용어들을 모아놓은 리스트를 만들고 반복문과 in이라는 키워드를 사용해서 하나하나 검사하면 될 것이다. 구현을 위하여 split() 메소드를 사용하여 입력 문자열을 단어로 분할하고 각 단어의 소문자 버전이 stop_words 목록에 있는지 확인하여 불용어를 제거한다. 그런 다음 필터링된 단어는 join() 메소드를 사용하여 다시 문자열로 결합한다.

167

불용어를 쉽게 처리하자

- 가장 유명한 자연어 처리 라이브러리는 nltk이다.
- nltk는 시스템 명령으로 설치해야 하는데 다음과 같은 운영체제의 프롬프트 상에서 pip 명령을 사용한다.

```
> pip install nltk
```

168

불용어를 쉽게 처리하자

- nltk를 이용한 불용어 처리 프로그램은 다음과 같다.

```
import nltk
from nltk.corpus import stopwords # 불용어 패키지를 사용하기 위해 가져온다
nltk.download('stopwords') # nltk 패키지로부터 불용어를 다운받는다
nltk.download('punkt') # nltk 패키지로부터 토큰화 작업을 위하여 punkt를 다운받는다
text = "This is an example of removing stop words from a string."
stop_words = set(stopwords.words("english"))
words = nltk.word_tokenize(text)

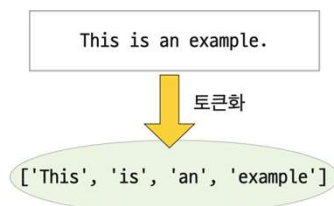
filtered_sentence = [w for w in words if not w in stop_words]
print(f"입력={text}")
print(f'출력="{ ".join(filtered_sentence)}"')

입력=This is an example of removing stop words from a string.
출력=This example removing stop words string
```

169

불용어를 쉽게 처리하자

- 이 프로그램은 nltk 라이브러리를 사용하여 입력 문자열을 단어로 토큰화하고 불용어를 제거하였다.
 - 이때 불용어 단어 목록은 nltk **말뭉치** corpus에서 가져온다.
 - 이를 위하여 nltk.download('stopwords')를 사용하였다.
 - 문장을 문자나 단어 또는 어휘 집합 단위로 나누는 작업을 **토큰화** tokenization라고 한다.
- nltk.download('punkt')는 nltk 패키지로 부터 토큰화 작업을 수행하기 위하여 필요하다.



170

불용어를 쉽게 처리하자



잠깐 - 말뭉치란

영어 corpus는 한글로 말뭉치라는 단어로 번역된다. 말뭉치란 평소에 우리가 쓰는 말이나 글에서 표본이 될 만한 것을 컴퓨터가 읽을 수 있는 형태로 모아 놓은 언어 자료를 지칭한다. 말뭉치는 인공지능과 빅데이터 시대에 매우 중요한 역할을 하고 있다.

171

실습 시간



감정 분석은 자연어 처리의 중요한 작업이다. 텍스트가 주어지면 이 텍스트에 실린 감정이 긍정적(positive)인지, 부정적(negative)인지, 중립(neutral)인지를 판단하는 작업을 해보자. 이 작업은 딥러닝을 이용하여 훈련 데이터로 학습시킬 수도 있으나 우리는 TextBlob 라이브러리를 사용하여 간단히 감정을 분석해볼 것이다. TextBlob는 다음과 같은 명령으로 설치하여야 한다.

```
> pip install textblob
```

원하는 결과

입력=This movie was terrible! The acting was poor and the story was boring.

결과=부정적

힌트



이 프로그램은 TextBlob에 내장된 감정 분석 기능을 사용하여 -1(부정)과 1(긍정) 사이의 값을 출력한다. TextBlob의 감정 분석은 규칙 기반 방법이며 항상 정확한 결과를 생성하는 것은 아니다. 딥러닝 기반의 감성 분류기를 이용하면 보다 발전된 감성 분석 모델을 사용할 수 있다. TextBlob이 설치되면 TextBlob(입력문자).sentiment.polarity로 감정값을 추출할 수 있다. 이 값이 0보다 크면 긍정적, 0이면 중립적, 그렇지 않으면 부정적 감정으로 볼 수 있다.

172

해답 코드



```
from textblob import TextBlob
review = "This movie was terrible! The acting was poor and the story was boring."
sentiment = TextBlob(review).sentiment.polarity
print(f"입력={review}\n")

if sentiment > 0:
    print("결과=긍정적")
elif sentiment == 0:
    print("결과=중립")
else:
    print("결과=부정적")
```

173

정규식을 알아보자

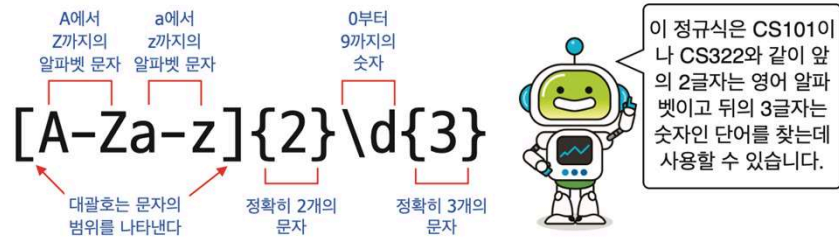
- 정규식 Regular Expression은 문자열 패턴을 나타내는 식
- 문자열을 처리하고 분석하는 데 매우 유용하다.
- 정규식을 사용하면 특정 패턴의 문자열을 검색하거나 추출하는 등 다양한 문자열 처리 작업을 보다 쉽게 수행할 수 있다.
- 정규식은 다양한 프로그래밍 언어에서 지원되며, 파이썬에서도 re 모듈을 통해 사용할 수 있다.
- 정규식은 기본적으로 문자열 패턴을 표현하는데 사용되는 특수한 문자열이다.
- 정규식은 일반적으로 패턴을 나타내는 특수한 의미를 가지는 문자인 메타 문자 meta character로 이루어져 있다.



174

정규식을 알아보자

- 예를 들어서 다음과 같은 정규식은 대학교에서 수강신청 과목 코드를 나타낸다.
- 즉, CS101이나 CS322와 같이 앞의 2글자는 영어 알파벳이고 뒤의 3글자는 숫자인 단어를 찾는데 사용할 수 있다.



- 위의 정규식을 사용해서 문자열 안에서 정규식에 해당되는 단어를 찾을 때
 - 파이썬에서 정규식을 사용하려면 re 모듈을 포함시켜야 한다.
 - re.search()를 호출하면 정규식에 매치되는 문자열을 찾을 수 있다.

175

정규식을 알아보자

```
import re          # 이 코드 이후의 모든 정규식 코드 예제에서는 import re 문을 사용합니다
text = "이번 학기 개설 과목은 CS101입니다."
match = re.search('[A-Za-z]{2}\d{3}', text)
if match:
    print("과목 코드가 발견됨:", match)
else:
    print("과목 코드가 발견되지 않음:", match)
```

과목 코드가 발견됨: <re.Match object; span=(13, 18), match='CS101'>

- 결과를 해석해보자면 re라는 정규식 모듈 내의 Match라는 객체가 반납한 값이 match이며, 'CS101'이 전체 텍스트의 13번째 위치에서 발견되었음을 알 수 있다.
- 발견된 위치는 match.start()와 match.end()를 호출하면 알 수 있다. 만약 정규식을 사용하지 않았다면 각 글자들을 검사해야 하고 또한 3개의 숫자가 연속되어 있는지도 검사하여야 하므로 상당히 복잡한 프로그램이 되었을 것이다.

176

정규식을 알아보자

메타 문자

- 앞의 코드에서 살펴본 '\d'와 같은 문자들을 메타 문자라고 하는데, 정규식에서는 다음 표와 같은 많은 메타 문자가 사용되고 있다.
- 메타 문자의 의미를 요약하면 다음과 같다.

메타문자	기능	설명
^	시작	시작하는 문자와 일치하는 가를 검사
\$	끝	종료 문자와 일치하는 가를 검사
.	문자	어떤 한 개의 문자와 일치하는 가를 검사
\d	숫자	어떤 한 개의 숫자와 일치하는 가를 검사
\D	숫자 제외	숫자가 아닌 모든 문자와 일치하는 가를 검사
\w	문자와 숫자	한 개의 문자나 숫자와 일치하는 가를 검사
\s	공백문자	공백 문자 (스페이스, 탭, 줄바꿈 등)
\S	공백제외 문자	공백 문자를 제외한 모든 문자가 될 수 있음
*	반복	앞 문자가 0번 이상 반복하는 가를 검사
+	반복	앞 문자가 1번 이상 반복하는 가를 검사
[abc], [a-c]	문자 선택 범위	a, b, c 중의 하나인 가를 검사
[^abc]	문자 제외 범위	a, b, c가 아닌 임의의 문자를 의미함
	또는	앞의 문자 또는 뒤의 문자를 의미함

- 메타 문자 중에서 가장 중요한 문자는 점(.)과 별표(*)이다.
- 점은 어떤 문자가 와도 상관없다는 의미이고
- 별표는 몇 번 반복되어도 상관없다는 것을 의미한다.

177

정규식을 알아보자

메타 문자 (계속)

- 점(.) 메타문자를 먼저 살펴보자.
 - 이 문자는 임의의 문자 한 개를 의미하는 메타 문자이다.
 - 예를 들어 주어진 텍스트에서 A로 시작해서 A로 끝나는 4글자의 단어를 모두 찾고자 한다. 이 패턴은 정규식으로 'A..A'로 나타낼 수 있다.
 - ABBA, AAAA, ACCA 등은 모두 발견된다.
 - 하지만 ABA, ABBBA와는 일치하지 않는다.

```
>>> re.search('A..A', 'ABBA')      # 조건에 맞춤
      <_sre.SRE_Match object; span=(0, 4), match='ABBA'>
>>> re.search('A..A', 'ABBBA')     # 조건에 맞지 않음
```

178

정규식을 알아보자

메타 문자 : *, +

- 메타 문자 별표(*)는 직전에 있는 문자를 0회 이상 반복하는 패턴과 매치된다.
- 예를 들어 AB*는 A, AA, AB, ABB 등의 어떤 문자열과도 일치된다.
- 모두 A로 시작하고 B가 0회 이상 반복되기 때문이다.

```
>>> re.search('AB*', 'A')           # 'A'라는 문자열이 조건에 맞음
<re.Match object; span=(0, 1), match='A'>
>>> re.search('AB*', 'ABB')        # 'ABB'라는 문자열이 조건에 맞음
<re.Match object; span=(0, 3), match='ABB'>
```

- 메타문자 +는 직전에 있는 임의의 패턴을 1회 또는 그 이상의 수로 가급적 많이 반복하는 패턴에 대해서 매치되는 표현식이다.

179

정규식을 알아보자

메타 문자 : *, + (계속)

- 따라서 AB+는 A와는 매치되지 않으며 AB, ABB와 같은 패턴의 문자열과 일치한다.

```
>>> re.search('AB+', 'A')           # 조건에 맞지 않음
>>> re.search('AB+', 'AB')
<re.Match object; span=(0, 2), match='AB'>
```

180

정규식을 알아보자

메타 문자 (계속)

- 정규 표현식의 `findall()`을 사용하면 정규식을 만족하는 모든 문자열들을 추출할 수 있다.
- 다음과 같이 `findall()`을 사용하여 `txt3`에 나타나는 모든 `My`를 뽑아 볼 수 있다.

```
>>> txt3 = 'My life my life my life in the sunshine'
>>> re.findall('[Mm]y', txt3)
['My', 'my', 'my']
```

181

정규식을 이용하여 특정한 패턴 찾기

문자열에서 전화번호를 찾는 예제

```
# 문자열에서 전화번호를 찾는 예제
string = "My phone number is 010-1234-5678. Call me anytime."
result = re.search('\d{3}-\d{4}-\d{4}', string)
print(result.group())    # 010-1234-5678
```

010-1234-5678

- 정규식 `\d{3}-\d{4}-\d{4}`은 숫자 3번 반복 후 - 글자, 숫자 4번 반복 후 - 글자, 숫자 4번 반복을 의미한다. 즉, 이 정규식은 "010-1234-5678"와 같이 세 자리 숫자, 하이픈, 네 자리 숫자, 하이픈, 네 자리 숫자의 패턴을 표현한다.

182

정규식을 이용하여 특정한 패턴 찾기

문자열에서 숫자를 찾는 예제

```
text = 'There are 12 apples and 3 oranges'
numbers = re.findall('\d+', text)
print(numbers)
```

```
['12', '3']
```

- re.findall() 함수를 사용하여 정규식 패턴 `\d+`을 이용해 문자열 `text`에서 연속된 숫자를 찾아 리스트 `numbers`에 저장한다. `\d`는 숫자를 의미하며, `+`는 바로 앞에 있는 패턴이 하나 이상 반복되는 경우에 일치된다.

183

정규식을 이용하여 특정한 패턴 찾기

이메일 주소에서 도메인 추출하기

```
email = 'user@example.com'
domain = re.search('@\w+\.\w+', email)
print(domain.group())
```

```
@example.com
```

- 위의 코드에서 사용된 정규식의 의미는 다음과 같다.
 - `@`는 문자열에서 `@` 문자를 찾는다.
 - `\w`는 알파벳, 숫자, 언더스코어(_)를 나타내는 메타 문자이다.
 - `+`는 앞에 나온 문자나 메타 문자가 1번 이상 반복됨을 의미한다.
 - `\.`는 이스케이프 문자로서, `.` 문자를 나타낸다. 이는 이메일 주소에서 도메인을 구분하는 `.` 문자를 나타낸다.
 - 마지막으로 `\w`가 나오는데, 이는 앞에서와 같이 알파벳, 숫자, 언더스코어를 나타내는 메타 문자인 `\w`가 1번 이상 반복됨을 나타낸다.
 - 따라서 `'@\w+\.\w+'`은 `@` 문자를 포함한 문자열에서, `.`으로 구분된 이메일 주소에서 도메인을 나타내는 정규식이다. 예를 들어, `@example.com`과 같은 도메인 이름은 이 정규식과 일치하게 된다.
- `group()` 메소드는 정규식에서 찾은 패턴을 반환한다.

184

정규식을 이용하여 특정한 패턴 찾기

주민등록번호 검사하기

- 우리의 일상생활에서 많이 사용되는 주민등록번호는 060101-4234567과 같이 출생년도 6자리와 하이픈 그리고 7자리의 숫자로 이루어져 있다. 입력한 주민등록 번호가 올바른가를 판단하는 코드를 만들어보자.

```
def is_valid_jumin(st):  
    pattern = '^d{6}\-d{7}' # 출생년도-7자리 숫자로 주민등록번호를 검사하는 정규식  
    match = re.match(pattern, st)  
    return match  
  
jumin = input('당신의 주민등록번호는: ') # 주민등록번호를 입력받는 코드  
if is_valid_jumin(jumin):  
    print('주민등록번호가 유효합니다.')  
else:  
    print('주민등록번호가 유효하지 않습니다.')
```

```
당신의 주민등록번호는: 060101-4234567  
주민등록번호가 유효합니다.  
당신의 주민등록번호는: 06011-1234567  
주민등록번호가 유효하지 않습니다.
```

185

정규식을 이용하여 특정한 패턴 찾기

주민등록번호 검사하기 (계속)

- 이 코드에서 사용된 '^wd{6}w-wd{7}'는 숫자 6개, 하이픈, 숫자 7개로 이루어진 문자열을 나타내는 정규식이다.
 - ^는 문자열의 시작을 나타낸다.
 - wd는 숫자를 나타내는 메타 문자이다.
 - {6}는 바로 앞에 있는 문자 또는 메타 문자가 6번 반복됨을 의미한다.
 - w는 이스케이프 문자로, 다음 문자를 메타 문자가 아닌 일반 문자로 해석하도록 한다.
 - -는 하이픈 문자를 나타내며, 이스케이프 문자인 w와 함께 사용된다.
 - {7}은 바로 앞에 있는 문자 또는 메타 문자가 7번 반복됨을 의미한다.



'^d{6}\-d{7}'는
숫자 6개, 하이픈, 숫자
7개로 이루어진 문자열
을 나타내는 정규식입니
다. 주민등록번호를 식
별하는데 유용하겠네요.

186

정규식을 이용하여 특정한 패턴 찾기

- 정규식을 사용하면 문자열에서 특정한 문자열을 찾아 다른 문자열로 대체할 수 있다. 파이썬에서는 re 모듈의 sub() 함수를 사용하여 문자열 대체를 수행한다. sub() 함수의 매개변수는 다음과 같다.

```
re.sub(pattern, repl, string, count=0, flags=0)
```

↑ ↑ ↑ ↑ ↑
찾고자 하는 대체할 검색 대상 대체를 수행할 정규식의 동작 방식을
문자열 패턴 문자열 문자열 횟수 제어하는 플래그

- 위의 매개변수에서 pattern은 찾고자 하는 문자열 패턴을 나타내는 정규식이며, repl은 대체할 문자열을 나타낸다. 또한 string은 검색 대상이 되는 문자열이다. 다음으로 나타나는 count는 대체를 수행할 최대 횟수를 나타내며, flags는 정규식의 동작 방식을 제어하는 플래그이다.

187

정규식을 이용하여 특정한 패턴 찾기

- 예를 들어, 아래의 text라는 문자열에서 숫자를 모두 제거하고 싶다면 다음과 같은 코드를 사용할 수 있다.

```
text = 'abc123def456'
result = re.sub('\d', '', text) # text 문자열에서 숫자를 모두 제거함
print(result)                    # 'abcdef'
```

abcdef

- 위 코드에서 'wd'는 정규식으로서, 숫자(wd)를 나타낸다. 그리고 re.sub() 함수의 마지막 인자인 text는 원본 문자열을 나타내며, 이 문자열에서 정규식과 일치하는 문자열을 모두 찾아 공백 문자열 (")로 대체한다. 따라서 결과는 abcdef가 된다.

188

정규식을 이용하여 특정한 패턴 찾기

- 또 다른 예로, 문자열에서 이메일 주소를 모두 마스킹 처리하고 싶다면 다음과 같은 코드를 사용할 수 있다.

```
text = 'My email is john@example.com'
result = re.sub('\w+@\w+\.\w+', '****', text)
print(result) # 'My email is ****'
```

```
My email is ****
```

- 위 코드에서 '\w+@\w+\.\w+'는 정규식으로서, 이메일 주소를 나타내는 패턴을 찾는다. 따라서, re.sub() 함수는 이메일 주소를 찾아 ****로 대체한다.
- 결과는 'My email is ****'가 된다. 또한 앞에서 찾은 패턴의 순서를 변경할 수도 있다.

189

정규식을 이용하여 특정한 패턴 찾기

- 예를 들어서 문자열에서 날짜 형식 mm/dd/yyyy를 찾아 yyyy-mm-dd 형식으로 변경하는 코드를 살펴보자.

```
text = '03/17/2024'
new_text = re.sub('(\d{2})/(\d{2})/(\d{4})', r'\3-\1-\2', text)
print(new_text) # 2024-03-17
```

```
2024-03-17
```

- 정규식 '(\d{2})/(\d{2})/(\d{4})'는 날짜 형식인 mm/dd/yyyy를 찾기 위한 정규식이다. 정규식에서 ()는 하나의 그룹을 나타낸다. (와) 사이에 있는 패턴은 하나의 그룹으로 취급된다.
- r'\3-\1-\2'는 교체될 문자열이다. 이 패턴에서 \3은 세 번째 그룹을 나타내고, \1, \2는 각각 첫 번째 패턴과 두 번째 패턴을 나타낸다.

190

HTML 태그를 제거하자

실습 시간



다음과 같이 문자열을 입력으로 받아서 HTML 태그만 제거하는 프로그램을 작성해보자.

원하는 결과

```
HTML 문장을 입력하세요: <h1>Hello, world!</h1><p>This is an example.</p>
Hello, world!This is an example.
```

힌트



다음은 사용자로부터 입력 받은 HTML 문장에서 HTML 태그를 제거하는 파이썬 프로그램이다. 이 프로그램은 `re.sub()` 함수를 사용하여 정규식으로 HTML 태그를 찾아 제거한다.

191

HTML 태그를 제거하자

해답 코드



```
import re

html = input("HTML 문장을 입력하세요: ")
# HTML 태그를 제거하는 sub()함수와 정규표현식
result = re.sub('<[^>]+>', '', html)
print(result)
```

- 위 코드에서는 먼저 사용자로부터 입력 받은 HTML 문장을 `input()` 함수를 사용하여 입력받는다.
- '`<[^>]+>`' 패턴은 `<`와 `>` 사이에 있는 모든 문자열을 찾을 수 있다. 마지막으로, `re.sub()` 함수를 사용하여 HTML 문장에서 HTML 태그를 제거한다. `sub()` 함수는 HTML 문장에서 정규식 패턴과 일치하는 문자열을 찾아 제거한 후, 새로운 문자열을 반환한다. 이를 `result` 변수에 저장하여 출력한다.

192

파이썬 라이브러리 : 넘파이



1

1 파이썬 리스트와 넘파이

- 데이터 과학을 위한 많은 라이브러리 중에서 가장 기본이 되는 라이브러리가 바로 **넘파이Numpy**이다.
- 2020년 9월 저명한 과학 저널 **네이처Nature**에 넘파이에 대한 리뷰 논문이 게재 되었다.
- 논문은 **텐서tensor**라 불리는 다차원 배열을 효율적으로 다루는 넘파이가 과학 분야에 파이썬 생태계를 제공했고, 여러 과학 분야의 수치 데이터 처리 기술들이 상호운용성을 가질 수 있도록 하는 역할을 했다고 밝히고 있다.
- 넘파이는 데이터를 다차원 배열에 저장할 때 같은 자료형으로만 저장할 수 있다. 이렇게 같은 자료형으로만 데이터를 저장하면 각 데이터 항목에 필요한 저장공간이 일정하다. 따라서 몇 번째 위치에 있는 항목이든 그 순서만 안다면 간단한 계산으로 바로 접근할 수 있는 것이다.



2

1 파이썬 리스트와 넘파이

- 파이썬의 리스트는 여러 자료형의 값들을 하나의 변수에 저장할 수 있는 자료구조로서 강력하고 활용도가 높다.
- 하지만 데이터를 처리할 때는 리스트와 리스트 간의 다양한 연산이 필요한데, 이러한 점에서 기능이 다소 부족하고, 연산 속도도 빠르지 않다. 따라서 데이터 과학자들은 넘파이 배열을 선호한다.

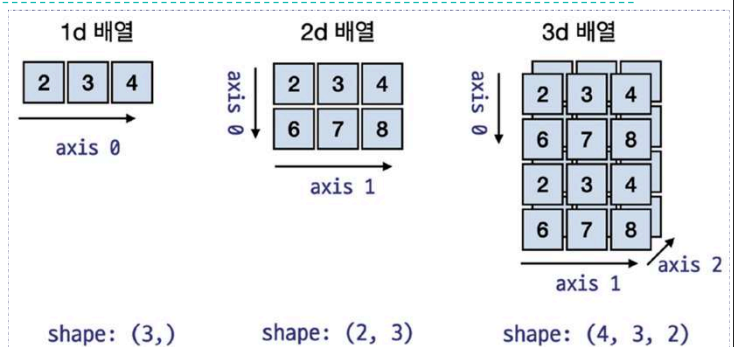
```
import numpy as np      # numpy의 별칭으로 np를 지정함
a = np.array([2, 3, 4]) # 다차원 배열 객체
```

- 넘파이를 사용할 때는 반드시 numpy 모듈을 import 해야만 한다. 일반적으로 넘파이 모듈에 대해서는 다음과 같이 as 키워드 다음에 나오는 np라는 별칭을 지정하여 사용한다.
- 넘파이의 가장 기본이 되는 다차원 배열을 array() 함수와 리스트를 이용해서 위와 같이 생성해 보도록 하자.

3

1 파이썬 리스트와 넘파이

- 이 배열은 2, 3, 4의 정수 값으로 구성되어 있는데 이 개별적인 값을 **항목item** 또는 **원소element**라고 한다.
- 이 배열 내부의 각 요소는 **인덱스index**라고 불리는 정수들로 참조된다.
- 또한 넘파이에서 차원은 **축axis**이라고도 한다.



- 위의 그림은 넘파이의 1차원, 2차원, 3차원 배열의 축과 그 **형상shape**을 나타내고 있다.
- 그림의 가장 오른쪽에 있는 배열은 형상이 (4, 3, 2)와 같은 형태로 표시되는데 이 형상의 의미는 3개의 튜플 형식으로 표현되는 **3차원 배열 내부의 각 축이 가지는 최대 원소의 개수**를 나타낸다.

4

2 다차원 배열의 속성들

- 넘파이의 핵심이 되는 **다차원 배열 ndarray**은 다음과 같은 속성을 가지고 있다.
- 아래와 같이 간단한 값을 가지는 다차원 배열 객체 `a`를 만들고 이 객체의 속성을 출력해 보자. 우리는 이러한 속성을 이용하여 프로그램의 오류를 찾거나 배열의 상세한 정보를 손쉽게 조회할 수 있다.

```

▶ a = np.array([2, 3, 4])      # 넘파이 ndarray 객체의 생성

# a 객체의 형상(shape), 차원, 요소의 자료형, 요소의 크기(byte), 요소의 수
a.shape, a.ndim, a.dtype, a.itemsize, a.size

((3,), 1, dtype('int64'), 8, 3)

```

5

2 다차원 배열의 속성들

- 이 배열 `a`는 다음과 같은 속성을 가지는데, 이 속성의 자세한 설명은 다음과 같다.

속 성	설 명
<code>ndim</code>	배열 축 혹은 차원의 개수를 나타냄.
<code>shape</code>	배열의 형상을 기술하며 (m, n) 형식의 튜플 형으로 나타냄.
<code>size</code>	배열 원소의 개수를 말하며 (m, n) 형상 배열의 size는 $m \cdot n$ 임.
<code>dtype</code>	배열 원소의 자료형을 기술함.
<code>itemsize</code>	배열 원소의 크기를 바이트 단위로 기술함. 예를 들어 <code>int32</code> 자료형의 크기는 $32/8 = 4$ 바이트가 됨.
<code>data</code>	배열의 실제 원소를 포함하고 있는 버퍼임.
<code>stride</code>	배열의 차원별로 다음 요소로 점프하는 데에 필요한 거리를 바이트로 표시한 값.

6

2 다차원 배열의 속성들

- 여러 가지 속성 중에서 다차원 배열의 행과 열을 비롯한 차원의 형상을 출력하는 **shape** 속성은 넘파이 이용자들이 가장 자주 사용하는 속성이다.
- a 배열은 (2, 3, 4)의 원소를 가지는 일차원 배열이기 때문에 그 shape 값이 (3,)과 같이 나타나며, 다음과 같은 2행 3열의 형태를 가진 2차원 배열 b의 속성은 (2, 3)으로 나타난다.

```
 b = np.array([[1, 2, 3], [4, 5, 6]]) # 넘파이 ndarray 객체의 b 생성
b.shape
(2, 3)
```

- 다차원 배열은 파이썬 리스트와 유사하게 + 나 * 연산자와 같은 연산자를 적용할 수 있다. 하지만 이 연산자들이 하는 일은 리스트와는 다르다.

7

2 다차원 배열의 속성들

- 예를 들어서, 다음과 같은 값을 가진 a, b라는 ndarray 객체가 있을 때 이 두 객체에 사칙연산을 적용해 보도록 하자.

```
 a = np.array([10, 20, 30]) # 넘파이 ndarray 객체 a 생성
b = np.array([1, 2, 3]) # 넘파이 ndarray 객체 b 생성
a + b
array([11, 22, 33])

 a - b
array([ 9, 18, 27])
```

8

2 다차원 배열의 속성들

	<code>a * b</code>
	<code>array([10, 40, 90])</code>
	<code>a / b</code>
	<code>array([10., 10., 10.])</code>

- 이 결과에서 알 수 있는 것은 사칙연산을 수행할 때 개별 원소별로 덧셈, 뺄셈, 곱셈 나눗셈이 이루어진다는 것이다.
- 따라서 a의 10 원소와 b의 1 원소가 사칙연산을 수행하여 다차원 배열의 첫 원소 11(10 + 1의 결과), 9(10 - 1의 결과), 10(10 * 1의 결과), 10.(10 / 1의 결과)이 됨을 알 수 있다.

9

3 다차원 배열과 브로드캐스팅

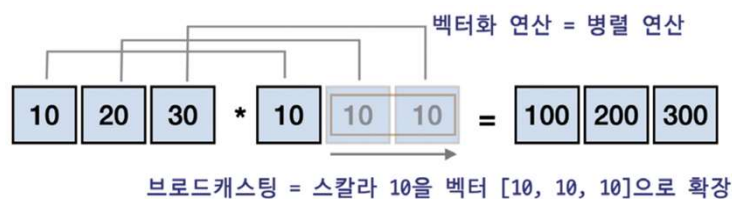
- 다음과 같이 a 배열을 생성하여 스칼라값 10을 곱하면 a의 모든 원소에 대하여 10이 곱해져서 결과가 100, 200, 300이 되는 것을 확인할 수 있다.
- 덧셈에 대해서도 동일하게 원소별 덧셈을 하는 것도 확인할 수 있다.

	<code>a = np.array([10, 20, 30])</code> <code>a * 10</code>
	<code>array([100, 200, 300])</code>
	<code>a + 10</code>
	<code>array([20, 30, 40])</code>

10

3 다차원 배열과 브로드캐스팅

- 넘파이는 $a * 10$ 을 수행할 때 `np.array([10, 20, 30]) * np.array([10, 10, 10])`와 같이 a 의 차원에 맞게, 그림과 같이 스칼라 10을 벡터로 확장시켜주는 작업을 하는데 이를 **브로드캐스팅**(broadcasting)이라고 한다.
- 브로드캐스팅과 동시에 파이썬은 하나의 명령을 여러 데이터에 적용하여 병렬적으로 연산하는데 이것을 **벡터화 연산**(vectorized operation)이라고도 한다.



11

3 다차원 배열과 브로드캐스팅

- 다음과 같이 2차원 배열 b 를 만들고 이 배열에 1차원 배열 c 를 더하고, 곱할 경우 어떤 결과가 나올지 알아보자.

```

▶ b = np.array([[10, 20, 30],
                [40, 50, 60]])
  c = np.array([2, 3, 4])
  b + c

array([[12, 23, 34],
       [42, 53, 64]])

▶ b * c

array([[ 20,  60, 120],
       [ 80, 150, 240]])

```

12

3 다차원 배열과 브로드캐스팅

- 이 코드의 수행 결과는 그림과 같이 [2, 3, 4] 라는 값을 가진 배열 c가 같은 값을 가지는 행을 하나 더 생성하여 [[2, 3, 4], [2, 3, 4]] 배열로 변환된 후 [[10, 20, 30], [40, 50, 60]] 과의 덧셈을 병렬적으로 수행한다.
- 두 배열 덧셈의 결과는 [[12, 23, 34], [42, 53, 64]]가 된다.

10	20	30
40	50	60

+

2	3	4
2	3	4

=

12	23	34
42	53	64

브로드캐스팅

13

3 다차원 배열과 브로드캐스팅

- 넘파이는 다차원 배열을 쉽게 생성하는 함수도 지원하고 있다. zeros((n, m)) 함수는 n×m 배열을 생성하는데 그 초기값은 0으로 해준다.
- 만약에 ones((n, m)) 함수를 호출하면 초기값을 1로 가지는 n×m 배열을 생성할 수 있다.
- 초기값을 임의의 수 x로 지정하고 싶을 경우 full((n, m), x)이라고 하는 함수를 호출하면 된다.
- 행과 열의 크기가 동일한 정방행렬 중에서 대각선 성분이 1이고 나머지 성분은 0인 행렬을 **단위행렬 identity matrix**이라고 하는데, eye(n) 함수를 이용하면 n×n 크기의 단위행렬을 생성할 수 있다.

14

3 다차원 배열과 브로드캐스팅

 `np.zeros((2, 3))` # 2행 3열의 행렬 생성 시 모든 값을 0으로

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

 `np.ones((2, 3))` # 2행 3열의 행렬 생성 시 모든 값을 1로

```
array([[1., 1., 1.],  
       [1., 1., 1.]])
```

 `np.full((2, 3), 100)` # 2행 3열의 행렬 생성 시 모든 값을 100으로

```
array([[100, 100, 100],  
       [100, 100, 100]])
```

 `np.eye(3)` # 3x3 크기의 단위행렬 생성

```
array([[1., 0., 0.],  
       [0., 1., 0.],  
       [0., 0., 1.]])
```

15

4 연속적인 값을 가지는 다차원 배열의 생성

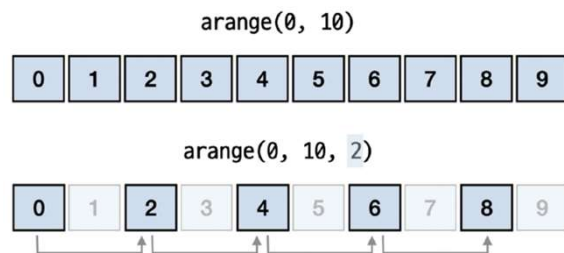
- 다음으로는 숫자의 열을 생성하는 함수 `arange(m, n)`에 대해서 알아보자.
- 이 함수는 리스트의 `range()` 함수와 유사하며, `m`부터 `n`미만까지 연속적인 정수나 실수를 만들어준다.
- 이 함수를 호출할 때 `arange(m, n, step)`과 같이 3개의 인자를 넣어주게 되면 `m`부터 `n`까지 `step`만큼 건너뛰며 연속적인 정수나 실수를 생성한다.

16

4 연속적인 값을 가지는 다차원 배열의 생성

- 먼저 `arange(0, 10)`을 실행하면 다음과 같은 다차원 배열 객체가 생성된다. 이 객체는 0부터 9까지의 값을 원소로 가진 배열을 생성한다.
- 이제 `arange(0, 10, 2)`과 같이 스텝 값 2를 추가해 주면 0부터 2씩 건너뛰면서 원소를 생성한다.

- 따라서 `arange(0, 10, 2)`는 `[0, 2, 4, 6, 8]`의 값을 가지는 배열을 생성한다. 넘파이의 스텝 값으로는 2뿐만 아니라 다른 값도 넣을 수 있는데 이 값은 정수 자료형이 아니어도 가능하다.



17

4 연속적인 값을 가지는 다차원 배열의 생성

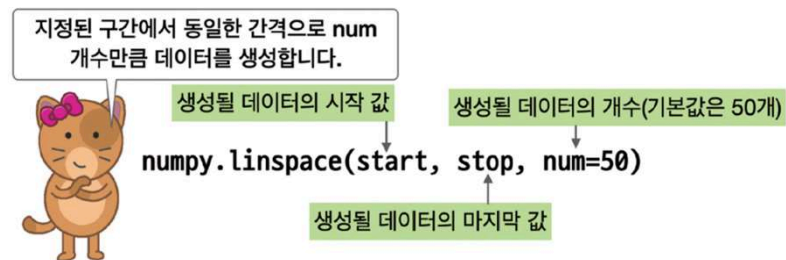
- 그러므로, 아래 마지막 코드와 같이 `arange(0.0, 1.0, 0.2)`와 같은 코드의 실행이 가능한 것이다.

	<code>np.arange(0, 10)</code>	# 0에서 9까지 연속적인 수열을 생성
	<code>array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])</code>	
	<code>np.arange(0, 10, 2)</code>	# 2씩 증가하는 수열을 생성
	<code>array([0, 2, 4, 6, 8])</code>	
	<code>np.arange(0, 10, 3)</code>	# 3씩 증가하는 수열을 생성
	<code>array([0, 3, 6, 9])</code>	
	<code>np.arange(0.0, 1.0, 0.2)</code>	# 0.2씩 증가하는 수열을 생성
	<code>array([0. , 0.2, 0.4, 0.6, 0.8])</code>	

18

4 연속적인 값을 가지는 다차원 배열의 생성

- 넘파이의 `linspace()` 함수는 지정된 구간에서 동일한 간격의 값을 지정된 횟수만큼 생성하는 기능이 있다.



19

4 연속적인 값을 가지는 다차원 배열의 생성

- 예를 들어 `np.linspace(0, 10, 5)` 함수는 0에서 10 사이의 구간에 대해(시작 값 0, 마지막 값 10을 포함) 5개의 값을 같은 간격으로 생성하므로 0.0, 2.5, 5.0, 7.5, 10.0의 값 5개를 생성한다.
- 이때 구간의 간격은 동일하며 지정된 개수로 자동으로 계산된다. 만일 값의 개수를 4개로 줄일 경우 0.0, 3.33, 6.66, 10.0의 값 4개를 생성한다.

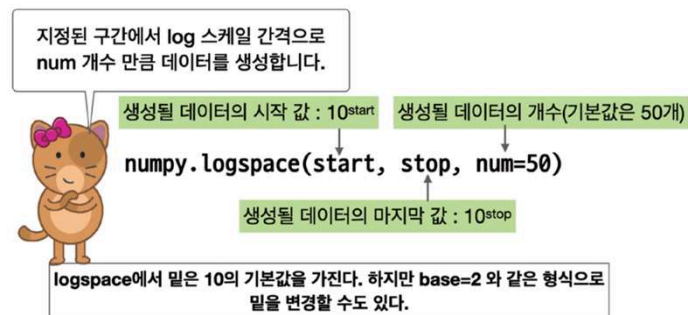
```
np.linspace(0, 10, 5)
array([ 0. , 2.5, 5. , 7.5, 10. ])

np.linspace(0, 10, 4)
array([ 0. , 3.33333333, 6.66666667, 10. ])
```

20

4 연속적인 값을 가지는 다차원 배열의 생성

- 또 다른 함수로 `logspace()` 가 있다. 이것은 로그 스케일로 수들을 생성한다.
- 형식은 `logspace(x, y, n)`이며, 생성되는 수의 시작은 10^x 부터 10^y 까지가 되며, n 개의 수가 생성된다. 수들 사이의 간격은 로그 스케일로 균등하게 결정된다.



21

4 연속적인 값을 가지는 다차원 배열의 생성

- 로그 스케일로 균등한 간격의 의미는 아래 코드와 같이 1, 10, 100, 1000, ...의 값에 대한 로그값이 0, 1, 2, 3, ...으로 균등하다는 의미이다.

```

▶ a = np.logspace(0, 5, 6).round(10) # 10^0에서 10^5까지 6개의 수를
a                                     # 로그 스케일 균일한 간격으로 생성한다.

array([1.e+00, 1.e+01, 1.e+02, 1.e+03, 1.e+04, 1.e+05])

▶ np.log10(a) # a 배열에 로그를 취하면 0, 1, 2, 3, 4, 5가 생성됨

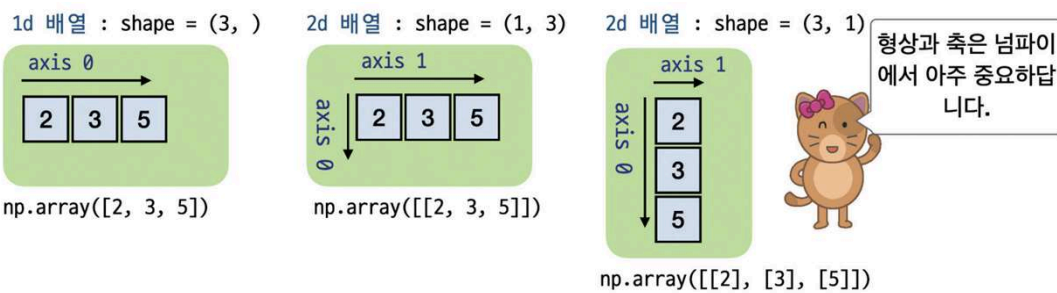
array([0., 1., 2., 3., 4., 5.])

```

22

5 다차원 배열의 축과 삽입

- 다차원 배열의 형상 값에서 (3,)와 (1, 3), 그리고 (3, 1)의 차이를 제대로 이해하는 것은 중요하다.
- 여기서 (3,)은 축이 하나인 1차원 배열이고, (1, 3), (3, 1)은 모두 2차원 배열이다.
- 2차원 배열은 [[로 시작하여]]로 표시된다는 점에 유의하자.



23

5 다차원 배열의 축과 삽입

- 다차원 배열에서 축을 이용하여 원소를 삽입하는 방법에 대하여 알아보자.
- 다음과 같은 다차원 배열 a가 있을 때, 이 배열의 두 번째 원소로 2를 삽입하고자 하면 다음과 같은 insert() 함수를 사용할 수 있다.

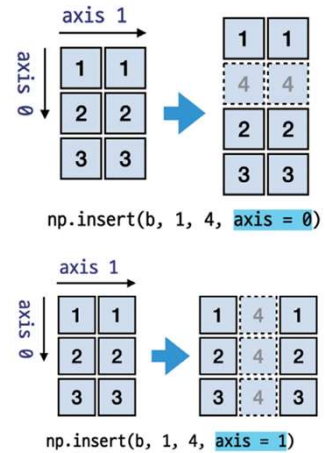
```
▶ a = np.array([1, 3, 4])
  np.insert(a, 1, 2)

array([1, 2, 3, 4])
```

24

5 다차원 배열의 축과 삽입

- 이제 다음과 같은 2차원 배열에 대해서 특정 값을 행과 열에 삽입하는 경우를 고려해 보자.
- axis를 명시하여 `[[1, 1], [2, 2], [3, 3]]` 형상의 2차원 배열에 `insert(a, 1, 4, axis = 0)`를 하게 되면 `[[1, 1], [4, 4], [2, 2], [3, 3]]`와 같은 배열이 된다.
- `insert()` 함수의 첫 매개변수는 배열 객체이며, 두 번째는 삽입할 위치, 세 번째는 삽입할 값, 네 번째는 삽입 방향인데 `axis = 0`으로 할 경우 0축(그림의 axis 0) 방향이 된다.



25

5 다차원 배열의 축과 삽입

- `insert(a, 1, 4, axis = 0)`는 원소 삽입 시에 4라는 스칼라값을 `[4, 4]`와 같은 벡터값으로 브로드캐스팅하므로, `insert(a, 1, (4, 4), axis = 0)`과 동일하다.
- `axis = 0`을 `axis = 1`로 변경할 경우 그림과 같이 축 1의 방향(2차원 배열에서 열 방향)의 두 번째 항목에 4를 삽입한다.
- 따라서, 원래의 `[1, 1]`을 `[1, 4, 1]`로, `[2, 2]`를 `[2, 4, 2]`로, `[3, 3]`을 `[3, 4, 3]`으로 모든 행의 값을 변경한다.

```

▶ b = np.array([[1, 1], [2, 2], [3, 3]])
  np.insert(b, 1, 4, axis = 0)

array([[1, 1], [4, 4], [2, 2], [3, 3]])

▶ np.insert(b, 1, 4, axis = 1)

array([[1, 4, 1],
       [2, 4, 2],
       [3, 4, 3]])
    
```

26

5 다차원 배열의 축과 삽입

- 또한, 넘파이에는 다음과 같은 `flip()` 함수가 있어서 지정된 축방향으로 원소들을 뒤집어 줄 수 있다.

```
▶ c = np.array([[1, 2, 3], [4, 5, 6]])  
  np.flip(c, axis=1)  
  
array([[3, 2, 1],  
       [6, 5, 4]])  
  
▶ np.flip(c, axis=0)  
  
array([[4, 5, 6],  
       [1, 2, 3]])
```

27

6 파이썬 리스트 vs 넘파이 다차원 배열

- 리스트와 넘파이의 가장 큰 차이는 계산 성능이다.
- 넘파이는 대용량의 배열과 행렬 연산을 수행하는 고차원적인 수학 연산자와 함수를 포함하고 있으며, 성능이 우수한 ndarray 객체를 제공한다.
- **배열array**은 동일한 자료형을 가진 데이터를 연속으로 저장한다.
- **파이썬의 리스트는 동일하지 않은 자료형을 가진 항목들을 담을 수 있다.** 이 때문에 파이썬의 리스트를 사용하면 원소에 접근하는 속도가 매우 느리다.
- 넘파이의 **ndarray 객체는 동일한 자료형의 항목들만 저장한다.** 이러한 차이로 넘파이가 원소에 접근하는 속도가 더 빠르다.

28

6 파이썬 리스트 vs 넘파이 다차원 배열

- 리스트는 데이터를 벡터의 개념으로 해석하지 않고 단순히 여러 데이터가 나열된 것으로만 생각한다.
- 넘파이는 배열을 선형대수의 벡터 개념으로 다룬다.
- 이 차이는 리스트와 넘파이 배열에 대해 덧셈 연산을 적용해서 확인해 보자. 동윤이는 두 군데 매장 A와 B를 소유하고 있다고 하고, 각 매장의 지난 3개월 매출이 다음과 같이 store_a와 store_b라는 리스트에 저장되었다고 하자.

```
▶ store_a = [20, 10, 30] # 매장 A의 매출 : 파이썬 리스트로 표현  
store_b = [70, 90, 70] # 매장 B의 매출 : 파이썬 리스트로 표현
```

29

6 파이썬 리스트 vs 넘파이 다차원 배열

- 두 리스트를 합하면 어떻게 될까? 아래와 같이 두 리스트는 월별로 합산되지 않고, 연결되어 버린다.

```
▶ list_sum = store_a + store_b # 파이썬 리스트의 더하기 연산  
list_sum  
[20, 10, 30, 70, 90, 70]
```

- 리스트를 이루는 원소들의 인덱스가 특별한 의미를 갖지 않고 단순히 하나의 리스트 내에서의 순서만을 나타내는 것으로 간주하기 때문이다.
- 이와는 반대로, 넘파이는 대응되는 인덱스들이 더해지고 뺄 수 있는 연관된 데이터라고 가정한다. 데이터를 **벡터**vector로 간주하는 것이다.

30

6 파이썬 리스트 vs 넘파이 다차원 배열

- 실제로 넘파이의 다차원 배열 덧셈을 수행해 보자. 넘파이의 ndarray는 파이썬의 리스트를 이용하여 쉽게 만들 수 있다. 넘파이가 가진 array() 함수에 파이썬 리스트를 넣으면 넘파이 배열을 돌려준다.



```
np_store_a = np.array(store_a) # store_a 리스트를 넘파이 배열로 변환
np_store_b = np.array(store_b) # store_b 리스트를 넘파이 배열로 변환
```

- 이 두 배열을 더해서 결과를 확인해 보자. 매달 발생한 두 매장의 매출을 월별로 더한 의미 있는 결과를 얻을 수 있을 것이다.



```
array_sum = np_store_a + np_store_b # 넘파이 배열의 덧셈 결과를 확인하자
array_sum
```

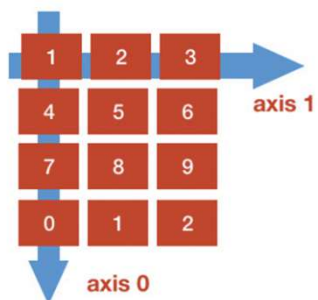
```
array([ 90, 100, 100])
```

31

7 넘파이 배열의 인덱싱과 슬라이싱

- 다차원 배열 내의 원소는 **인덱스index**라고 불리는 정수를 사용하여 참조할 수 있다.
- 리스트에서 사용했던 인덱싱과 슬라이싱은 넘파이 배열에도 거의 동일하게 적용된다.
- 1차원 배열의 인덱싱과 슬라이싱은 리스트와 큰 차이가 없으므로 2차원 배열을 이용하여 기본적인 인덱싱과 슬라이싱을 연습해 보자.
- 그림과 같은 2차원 배열을 만들어 보자. 이것은 아래와 같이 리스트의 리스트(이중 리스트)를 만들어 생성할 수 있다.

2차원 배열 (ndim = 2)

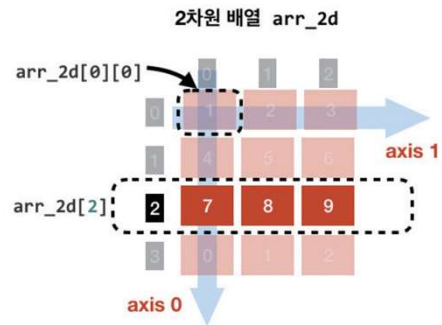


```
arr_2d = np.array( [[1, 2, 3], [4, 5, 6],
                    [7, 8, 9], [0, 1, 2]] )
```

32

7 넘파이 배열의 인덱싱과 슬라이싱

- 인덱싱을 먼저 수행해 보자. 인덱싱은 두 축에 대해서 모두 가능하다.
- 첫 번째 축으로 0, 두 번째 축으로 0을 지정하면 가장 첫 원소인 1이 나타날 것이다.
- 두 번째 축에 대해 인덱싱을 하지 않으면, 첫 번째 축에서 지정된 위치의 모든 데이터가 나타난다.



```
print(arr_2d[0][0])
print(arr_2d[2])
```

```
1
[7, 8, 9]
```

33

8 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 다차원 배열 ndarray의 메소드에 대하여 살펴보도록 하자.
- [10, 20, 30]의 세 원소를 가진 1차원 배열의 max(), min(), mean() 메소드는 최대값 30, 최소값 10, 평균값 20.0을 다음과 같이 출력한다.
- 만일 원소들의 자료형을 변환하고자 한다면 astype() 메소드를 사용한다.

```
a = np.array([10, 20, 30])
a.max(), a.min() # a 배열원소 중 최대값, 최소값을 출력함
```

```
(30, 10)
```

```
a.mean() # a의 평균을 출력함
```

```
20.0
```

```
a.astype(np.float64) # 원소의 자료형을 float64로 변환
```

```
array([10., 20., 30.])
```

34

8 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 다차원 배열이 2차원 이상의 배열이라면 이를 1차원 배열로 변환할 필요성도 있는데 이때 사용하는 메소드가 바로 `flatten()` 메소드이다. 이 과정을 평탄화라고 한다.

```
b = np.array([[1, 1], [2, 2], [3, 3]])
b.flatten() # 넘파이 다차원 배열의 평탄화 메소드

array([1, 1, 2, 2, 3, 3])
```

- 2차원 배열의 행과 열의 순서를 서로 교환하는 것을 **전치** `transpose`라고 하는데 다차원 배열의 전치는 `.T` 속성값을 이용하면 된다.

```
b = np.array([[1, 1], [2, 2], [3, 3]])
b.T

array([[1, 2, 3],
       [1, 2, 3]])
```

35

8 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 파이썬의 리스트는 정렬을 위한 `sort()` 함수를 가지고 있는데 넘파이 다차원 배열 역시 정렬을 위한 `sort()` 함수를 가지고 있다.

```
c = np.array([35, 24, 55, 69, 19, 99])
c.sort() # 배열의 정렬(오름차순)
c[::-1] # 내림차순 정렬

array([99, 69, 55, 35, 24, 19])
```

- 2차원 배열이 있을 때 이 배열에 `sort()` 메소드를 사용하면 원소들은 `axis 1` 방향으로 오름차순 정렬이 된다.
- 키워드 인자로 `axis=0`을 넣어주면 `axis 0`의 방향으로 오름차순 정렬이 된다.

36

8 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

```
d = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])
d.sort() # 배열의 정렬, 디폴트 axis는 1
d
```

```
array([[24, 35, 55],
       [ 9, 19, 69],
       [ 1,  4, 11]])
```

```
d = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])
d.sort(axis=0) # axis=0 방향 정렬
d
```

```
array([[ 1,  4, 11],
       [ 9, 19, 55],
       [24, 35, 69]])
```

37

9 다차원 배열을 위한 append() 함수와 행렬 곱셈

- append() 함수는 인자로 입력된 두 배열을 합하는 일을 하는데 axis를 통해서 축을 명시하지 않으면 디폴트 형상인 1차원 배열로 변환하게 된다.
- 이때 axis = 0과 같이 축을 명시해 준다면 다음과 같이 특정한 축으로 합하게 되어 배열의 차원이 유지된다.

```
a = np.array([1, 2, 3])
b = np.array([[4, 5, 6], [7, 8, 9]])
np.append(a, b)
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.append(a, b, axis = 0) # [a]를 통해 2차원 배열로 만들어야 함
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

38

9 다차원 배열을 위한 append() 함수와 행렬 곱셈

- 선형대수에서 사용되는 행렬 곱셈은 행렬에 있는 행의 성분과 열의 성분을 각각 원소끼리 곱한 것을 더한 결과이다.
- 그러나, 아래와 같은 a, b에 대하여 $a * b$ 를 구하면 a, b의 각 원소끼리의 곱을 단순 계산한다.
- 넘파이에서 제공하는 행렬 곱 함수는 `matmul()`이다.
- 이 `matmul()` 함수를 조금 더 편리하게 사용할 수 있는 것이 `@` 연산자이다.

```

▶ a = np.array([[1, 2], [3, 4]]) # 2차원 배열 a
  b = np.array([[10, 20], [30, 40]]) # 2차원 배열 b
  a * b          # 2차원 배열의 곱하기 연산 - 원소 간의 곱셈

array([[ 10, 40],
       [ 90, 160]])

```

39

9 다차원 배열을 위한 append() 함수와 행렬 곱셈

```

▶ np.matmul(a, b) # 2차원 배열 a, b의 행렬곱, a @ b와 같다

```

```

array([[ 70, 100],
       [150, 220]])

```

```

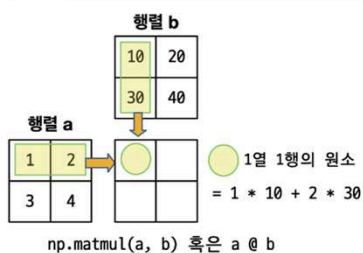
▶ a @ b # 2차원 배열 a, b의 행렬곱, 위의 결과와 동일함

```

```

array([[ 70, 100],
       [150, 220]])

```



$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 1*10 & 2*20 \\ 3*30 & 4*40 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} @ \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} = \begin{bmatrix} 10*10+2*30 & 1*20+2*40 \\ 3*10+4*30 & 3*20+4*40 \end{bmatrix}$$

40

배열의 형태를 바꾸는 reshape() [보충자료]

- reshape() 메소드는 상당히 많이 사용되는 메소드이다. 데이터의 개수는 유지한 채로 배열의 차원과 형태를 변경한다. 이 메소드의 인자인 shape을 튜플의 형태로 넘겨주는 것이 원칙이지만 reshape(x, y)라고 하면 reshape((x, y))와 동일하게 처리된다.

변경하여 얻고 싶은 형태를 넘겨 줌
1차원: (n,)
2차원: (n, m)
3차원: (n, m, l)

new_array = old_array.reshape(shape)



이전 배열 old_array의 형태가 (n,m)이고, 새롭게 얻고 싶은 형태 shape이 (l,k)라고 하면 $n \times m = l \times k$ 를 만족해야만 형태를 바꿀 수 있습니다.

```
>>> y = np.arange(12)
>>> y
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

```
>>> y.reshape(3, 4)
array([[ 0, 1, 2, 3],
       [ 4, 5, 6, 7],
       [ 8, 9, 10, 11]])
```

배열의 형태가 3행 4열의 모양이 되도록 만듭니다

41

배열의 형태를 바꾸는 reshape() 과 flatten() [보충자료]

```
>>> y.reshape(6, -1)
array([[ 0, 1],
       [ 2, 3],
       [ 4, 5],
       [ 6, 7],
       [ 8, 9],
       [10, 11]])
```

인수로 -1을 전달하면 데이터의 개수에 맞춰서 자동으로 배열의 형태가 결정

```
>>> y.reshape(7, 2)
...
y.reshape(7, 2)
ValueError: cannot reshape array of size 12 into shape (7,2)
```

reshape()에 의해 생성될 배열의 형태가 호환되지 않을 경우 발생하는 오류

```
>>> y.flatten() # 2차원 배열을 1차원 배열로 만들어 준다
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11])
```

flatten()은 평탄화 함수로 2차원 이상의 고차원 배열을 1차원 배열로 만들어 준다.

42

9 다차원 배열을 위한 append() 함수와 행렬 곱셈



도전문제 3.5 : 다차원 행렬의 생성과 활용(난이도 : 중)

1. 1에서 9까지의 모든 정수 값을 크기 순서대로 가지는 3×3 크기의 행렬 a를 arange()와 reshape()을 이용하여 생성하고, 모든 성분의 값이 3인 3×3 크기의 행렬 b를 full() 함수를 사용하여 생성한 다음, 다음과 같이 출력하여야.

```
a = [[1 2 3]
      [4 5 6]
      [7 8 9]]
```

```
b = [[3 3 3]
      [3 3 3]
      [3 3 3]]
```

2. 다음과 같은 행렬 연산을 수행한 후 그 결과를 출력하여야.

```
a + b, a - b, a * b, a / b, a @ b, a ** 2
```

43

10 과학자들이 사랑하는 수: 난수

- 데이터 과학을 위해서 사용하는 데이터는 1) **실세계의 관측을 통해서 획득한 데이터**, 2) **실세계 데이터와 유사한 특성을 가지는 가상의 데이터**를 사용하는 것이 일반적이다.
- 데이터 과학의 원래 목적을 생각해 본다면 당연히 실세계의 관측을 통해서 데이터를 획득하는 것이 바람직할 것으로 생각되지만, 이 경우 **많은 비용이 필요하며 장시간의 관측**이 필요한 경우가 많다.
- 따라서 실세계와 유사한 특성을 가지는 가상의 데이터를 만들어서 분석하고 시뮬레이션을 한 후에 이 모델을 실세계 데이터 분석에 적용하는 경우가 많다.
- 이를 위해서는 난수에 대한 개념을 이해하고 난수를 생성하는 알고리즘을 먼저 알아야 한다.



규칙성이 없는 수가 난수인 데요. 프로그램을 통해서 만든 규칙을 통해 난수를 생성할 수 있을까요?

44

10 과학자들이 사랑하는 수: 난수

- 난수란 **인간의 의도와 무관하게 생성된 수**를 의미하므로, 인간이 만든 기계에 특정한 알고리즘을 통해서 난수를 생성한다는 것 자체가 모순이다.
- 하지만 난수와 매우 유사한 특징을 가지는 수를 생성하는 것은 가능한데 이러한 난수는 **의사 난수pseudo random** 라고 한다.
- 의사(疑似)란 '의심스러운', '비슷한'을 의미하는데 진정한 의미의 난수가 아니지만, **난수와 유사한 기능을 한다**는 의미로 이 단어를 사용한다.
- 의사 난수 생성기의 특징은 이 생성기가 만들어낸 임의의 난수 값 (X_n) 을 다음 난수 값 (X_{n+1}) 을 만드는 재료로 사용한다는 점이다.
- 다음은 의사 난수 생성기의 예이다.

$$X_{n+1} = (aX_n + b) \bmod m$$

45

10 과학자들이 사랑하는 수: 난수

- 여기서 x 는 의사 랜덤 값의 **열sequence**이 되며, x_n 값은 x_{n+1} 값을 만드는 데 사용될 수 있다.
- a 값과 b , m 은 임의의 값으로 선택하면 되는데 여기서는 $a = 1103515245$, $b = 12345$, $m = 231$ 로 두고 의사 난수 함수를 만들어 보자.

```
def pseudo_rand(x):  
    a = 1103515245  
    b = 12345  
    m = 2 ** 31  
    new_x = (a * x + b) % m  
    return new_x
```

46

10 과학자들이 사랑하는 수: 난수

```
seed = 100          # 초기값을 임의로 설정하자
x = pseudo_rand(seed) # seed를 입력으로 하여 새로운 x를 만들자
print(x)
x = pseudo_rand(x)   # x_n을 입력으로 하여 새로운 x_(n+1)을 만들자
print(x)
```

```
829870797
1533044610
```

- 난수를 생성하는 코드에는 다음 난수를 생성하기 위한 최초 값이 필요한데 이를 **씨드 값**(seed number)이라고 한다.
- 보통 시드값은 컴퓨터로부터 시간 정보를 얻어서 이 값을 사용하는 경우가 많다.
- 이 책에서 사용하게 될 난수 값은 모두 **의사 난수로 생성한 값**이지만 이를 구분하지 않고 **난수 값으로 지칭**하도록 한다.

47

10 과학자들이 사랑하는 수: 난수

- 난수를 생성하는 함수나 모듈은 대부분의 프로그래밍 언어에서 라이브러리 형태로 제공한다.
- 넘파이로 가상의 성인 10명의 키를 생성하는 경우를 고려해 보자.
- 이 데이터가 150에서 190센티미터 사이의 임의의 값을 가진다고 가정하자.
- 이 값이 정수값일 경우 넘파이의 random서브 모듈에 있는 randint() 함수를 사용할 수 있다.
- randint() 함수는 인자로 최소값, 최대값, 원소의 개수를 줄 수 있다.
- 이때, 최대값이 191일 경우 191은 원소에 포함되지 않는다.

```
np.random.randint(150, 191, size=10)

array([186, 157, 167, 153, 159, 173, 176, 187, 163, 179])
```

48

11 다양한 난수 만들기 함수를 살펴보자

- 넘파이는 풍부한 난수 생성 기능을 제공하므로 이것을 활용하면 데이터 과학을 위한 임의의 데이터를 쉽게 얻을 수 있다. 넘파이의 난수는 random 서브 모듈에서 제공하는데 randn() 함수는 표준 정규 분포를 따르는 난수를 생성하는 데 사용된다.
- 만일 이 데이터들이 평균값이 165, 표준편차가 10인 정규 분포를 따른다고 할 경우 다음과 같은 방법을 사용할 수 있다.
- randn(5)는 5개의 난수를 생성하는데 이 난수는 평균이 0이고 분산이 1인 **표준 정규 분포** *standard normal distribution*를 따른다. 따라서 이 값에 10을 곱하고 165를 더하게 되며 우리가 원하는 데이터를 얻을 수 있다.

```
rnd = np.random.randn(5) * 10 + 165 # 평균값이 165, 표준편차가 10인 값 생성
rnd

array([176.07979332, 177.91821264, 165.45715947, 164.34633461,
       167.9482772 ])
```

49

11 다양한 난수 만들기 함수를 살펴보자

- 이 결과에 round(2)와 같은 방법으로 소수점 아래 숫자의 범위를 제한하거나 astype(int) 메소드를 사용하여 정수형 자료로 변환할 수 있다.

```
rnd.round(2) # 소수점 아래 둘째 자리까지 값으로 변환

array([176.08, 177.92, 165.46, 164.35, 167.95])

rnd.astype(int) # 정수형 자료형으로 변환

array([176, 177, 165, 164, 167])
```

- 만일 seed를 다음과 같이 42와 같은 특정한 값으로 지정하면 매번 동일한 난수의 열을 얻을 수 있다.

```
np.random.seed(42) # 씨드값이 특정되면 매번 동일한 난수의 열이 생성됨
```

50

11 다양한 난수 만들기 함수를 살펴보자

- 정규 분포는 매우 중요한 난수로 `normal()` 함수를 사용해서 얻을 수도 있다.
- 이전에 살펴본 평균값이 165, 표준편차가 10인 정규 분포 함수는 다음과 같이 생성 가능하다.

```
nums = np.random.normal(loc=165, scale=10, size=(3, 4)).round(2)
nums

array([[160.32, 156.77, 164.35, 157.87],
       [174.06, 172.66, 173.26, 151.76],
       [147.48, 175.02, 170.45, 183.95]])
```

51

11 다양한 난수 만들기 함수를 살펴보자

- 다음은 `random` 서브 모듈에 포함된 중요한 난수 관련 함수이다.

함수 이름	설명
<code>rand()</code>	균등분포의 난수 표본을 추출한다.
<code>randn()</code>	표준 정규 분포의 난수 표본을 추출한다.
<code>randint()</code>	최소값과 최대값을 인자로 받아, 주어진 범위 내의 정수 난수를 생성한다. 이때 최대값은 포함하지 않는다.
<code>normal()</code>	평균과 표준편차를 인자로 받아, 정규 분포의 난수 표본을 추출한다.
<code>uniform()</code>	균등분포에서 표본을 추출한다.
<code>shuffle()</code>	주어진 리스트나 다차원 배열의 순서를 임의로 섞는다.
<code>permutation()</code>	주어진 범위의 수를 생성하여 섞거나, 배열을 임의의 순서로 섞는다.

52

11 다양한 난수 만들기 함수를 살펴보자

- 다음은 이 함수들을 사용하여 임의의 수를 생성하거나 섞는 예시이다.

```
 a = np.arange(10)      # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 값을 생성
np.random.shuffle(a)
a                        # a에는 순서가 뒤섞인 값이 들어있다

array([2, 8, 9, 6, 4, 0, 1, 5, 7, 3])

 np.random.permutation([2, 4, 6, 8, 10])

array([ 8,  4, 10,  6,  2])
```

53

11 다양한 난수 만들기 함수를 살펴보자



도전문제 3.6(난이도 : 중)

`arange()`를 사용하여 1에서 50까지의 원소를 가지는 다차원 배열을 만들자. 이 배열의 원소 50개를 랜덤하게 섞은 후 80%의 데이터는 `train_data`에 넣고 나머지 20%의 데이터는 `test_data`라는 배열에 넣어서 이 두 배열을 반환하는 함수 `train_test_split()`을 만들자. 반환된 배열 값을 각각 출력하여라.

54

12 넘파이 스타일의 슬라이싱과 논리 인덱싱

- 넘파이 스타일의 인덱싱을 이용하여 슬라이싱을 하면 각각의 슬라이싱 범위는 리스트의 슬라이싱처럼 순차적으로 적용되는 것이 아니라, 각각의 축에 별도로 적용된다.
- 따라서 다음과 같은 슬라이싱이 가능하다.

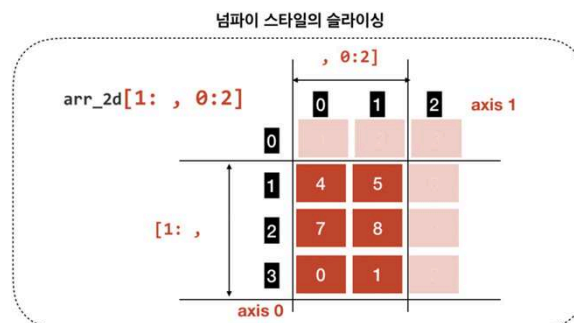
```
arr_2d = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9],
                   [0, 1, 2]])

arr_2d[1: , 0:2]

array([[4, 5],
       [7, 8],
       [0, 1]])
```

55

12 넘파이 스타일의 슬라이싱과 논리 인덱싱

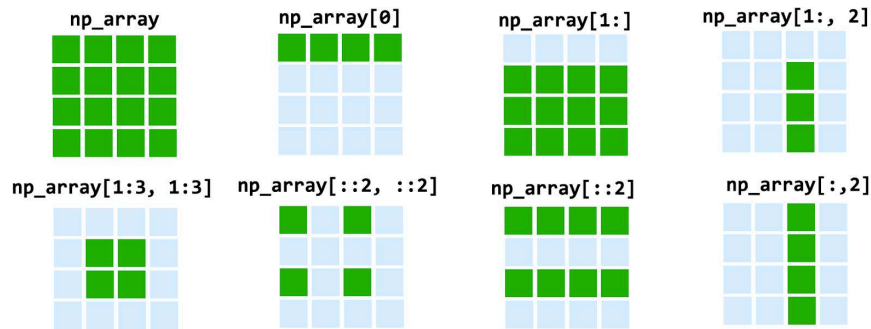


- 이것은 행과 열로 이루어진 데이터에서 특정한 행들과 열만 추려서 뽑아내는 데에 매우 유용한 방식이며, 선형대수의 행렬 관련 기능을 구현하는 데에 매우 중요한 기능이기도 하다.
- 넘파이 스타일의 슬라이싱은 큰 행렬에서 작은 행렬을 끄집어내는 것으로 이해하면 된다.

56

12 넘파이 스타일의 슬라이싱과 논리 인덱싱

- 4x4 크기의 np_array라는 다차원배열의 인덱싱 및 슬라이싱 결과는 다음과 같다



57

12 넘파이 스타일의 슬라이싱과 논리 인덱싱

- 넘파이는 데이터를 추려내는 데에 슬라이싱 뿐만 아니라 **논리 인덱싱** (logical indexing)이란 효율적인 방법도 제공한다. 이것은 특정한 어떤 조건을 주어서 배열에서 원하는 값을 추려내는 것이다.
- 넘파이 배열에 조건을 주면 불리언 배열이 만들어진다. 2차원 배열을 이용하여 확인해 보자.
- 아래의 예는 16개의 원소를 가진 2차원 넘파이 배열을 만들었다. 그리고, 동일한 크기의 배열인데, 각 원소가 "2의 배수"인 경우에 참이 되는 불리언 배열을 다음과 같이 만들 수 있다. 방법은 넘파이 배열을 조건식에 넣기만 하면 된다.

```
np_array = np.array([[1,2,3,4], [5,6,7,8], [9,10,11,12], [13,14,15,16]])
np_array % 2 == 0

array([[False,  True, False,  True],
       [False,  True, False,  True],
       [False,  True, False,  True],
       [False,  True, False,  True]])
```

58

12 넘파이 스타일의 슬라이싱과 논리 인덱싱

- 이렇게 얻어진 배열을 이용하여 인덱스로 사용하면 해당 조건에 부합하는 원소만 추출할 수 있다.
- 이렇게 하면 np_array 배열 내의 모든 원소 중에서 참인 위치의 값, 즉 2의 배수인 것만 추출된다.

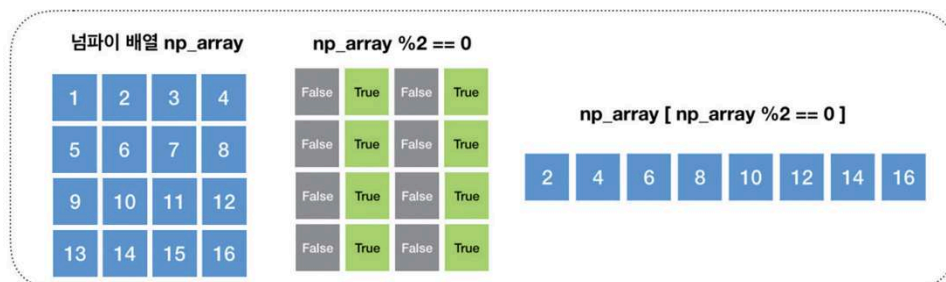
```
np_array[ np_array % 2 == 0 ]
```

```
array([ 2,  4,  6,  8, 10, 12, 14, 16])
```

59

12 넘파이 스타일의 슬라이싱과 논리 인덱싱

- 이 동작을 시각적으로 표시하면 다음과 같은 그림으로 설명할 수 있다.



- 넘파이의 다차원 배열은 데이터 과학자들이 다루는 벡터 형태의 데이터와 행렬을 다루기에 매우 적합한 기능들을 가지고 있는 것을 확인할 수 있었다.
- 넘파이는 다량의 수치 데이터를 고속으로 처리할 수 있는 데이터 병렬 처리 특성이 있다.

60

13 넘파이는 왜 성공했나

- 넘파이는 다량의 수치 데이터를 다루고, 일부를 추출하는 데에 효율적인 기능을 제공하고 있지만, 이것만으로 넘파이의 성공 이유를 모두 설명할 수는 없다.
- 넘파이는 사실 강력한 **배열 프로그래밍** `array programming` 기능 덕분에 오늘의 인기를 누리게 되었다.
- 배열 프로그래밍은 데이터에 연산을 실시할 때, 데이터를 구성하는 값 전체에 대해 한 번에 연산이 적용되도록 하는 방식을 의미한다.
- 이러한 해법을 제공하지 못하는 언어를 **스칼라** `scalar` 언어라고 하며, 전통적인 C, C++ 등은 스칼라 언어에 해당한다.

61

13 넘파이는 왜 성공했나

- 배열 프로그래밍은 데이터를 다루는 연산의 표현을 간결하게 만든다. 예를 들어 두 개의 배열 `arr_a`와 `arr_b`가 있고, 두 배열 원소의 개수는 `arr_len`이라고 가정해 보자. 스칼라 언어를 사용한다면 이 두 배열을 더하는 코드는 다음과 같은 의사 코드로 표현될 것이다.

```
For i from 0 to arr_len-1
  arr_sum[i] = arr_a[i] + arr_b[i]
end For
```

- 그러나 배열 프로그래밍을 지원하는 언어는 다음과 같은 표현을 지원한다.

```
arr_sum = arr_a + arr_b
```

62

13 넘파이는 왜 성공했나

- 위와 같은 표현은 연산의 대상이 되는 **피연산자**(operand)가 다수의 원소를 가진 배열, 즉 수학적으로 여러 수로 이루어진 벡터 데이터라는 것을 가정하기 때문에 가능하다.
- 여기서 더하기 연산자는 배열과 배열, 수학적 표현으로는 벡터와 벡터의 더하기를 의미하는 것이다.
- 이러한 이유로 배열 프로그래밍에서 사용하는 연산을 **벡터화 연산**(vectorized operation)이라고 한다.

63

13 넘파이는 왜 성공했나

- 물론 표현의 간결성만이 배열 프로그래밍의 장점이 아니다. 배열 프로그래밍이 사용하는 벡터화 연산은 강력한 병렬 처리가 뒷받침될 때 완성된다.
- 즉, **For** 반복문을 사용한 스칼라 언어가 각각의 원소들을 순차적으로 처리해야 한다면, 벡터화 연산은 두 배열의 대응하는 원소를 서로 더해서 새로운 배열에 저장하는 일을 차례를 기다리지 않고 동시에 수행할 수 있다.
- 이렇게 데이터를 잘게 쪼개어 각각에 대해 동일한 연산을 독립적으로 수행하는 방식을 **데이터 병렬성**(data parallelism)이라고 한다.

64

13 넘파이는 왜 성공했나

- 프로그래밍에서 문제를 해결할 때 사용하는 병렬성은 크게 두 가지 유형으로 구분된다.
- 하나는 전통적 병렬 프로그래밍의 주제인 **작업 병렬성**task parallelism이다. 이것은 문제를 해결하는 방법을 이루는 작은 작업들 사이의 의존성을 분석한 뒤 동시에 처리할 수 있는 일은 여러 개의 계산 코어가 동시에 진행하는 방식이다.
- 또 다른 방식은 처리해야 하는 데이터를 잘게 쪼개어 각각의 조각에 대해 동일한 작업을 여러 개의 코어가 동시에 진행하는 방식이다. 이것을 데이터 병렬성이라고 부른다.
- 이 방식은 엄청나게 많은 픽셀을 그려야 하지만, 각각의 픽셀을 그리는 데에 필요한 작업은 동일한 절차를 따르면 되는 컴퓨터 그래픽스와 같은 분야에 적합하다.
- 이러한 이유로 **그래픽 처리 장치**GPU는 고성능 데이터 병렬 처리기로 발전하여 현재 슈퍼컴퓨팅 자원으로 적극 활용되고 있는 것이다.

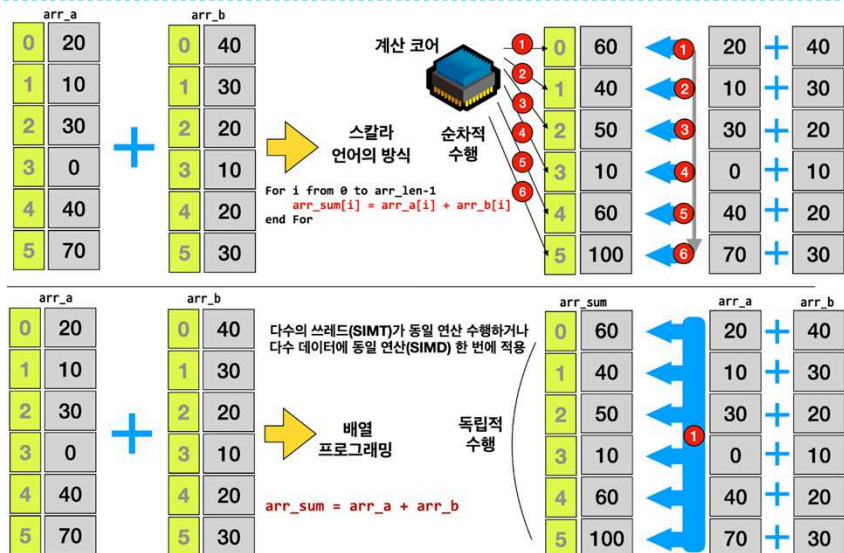
65

13 넘파이는 왜 성공했나

- GPU는 다수의 **쓰레드**thread를 만들어 동일한 연산을 수행하게 하는데, 이러한 구조를 **단일 명령어 다중 쓰레드**single instruction multiple thread:SIMT 방식이라고 한다.
- CPU는 GPU와 달리 쓰레드를 생성하지 않고, 하나의 연산이 다수의 데이터에 동시에 적용되도록 하는 **단일 명령어 다중 데이터**single instruction multiple data:SIMD 방식을 사용한다.
- 전통적인 언어로도 이러한 데이터 병렬성을 활용할 수 있지만, 벡터화 연산을 그대로 표현하는 효율적인 코딩은 어렵다.
- 넘파이는 강력하고 압축적인 프로그래밍 구문을 제공하며, 프로그래머가 효율적으로 고성능의 데이터 병렬 처리를 할 수 있게 해준다.
- 넘파이의 성공 이유로 꼽아야 할 가장 중요한 요소가 바로 이것이다.

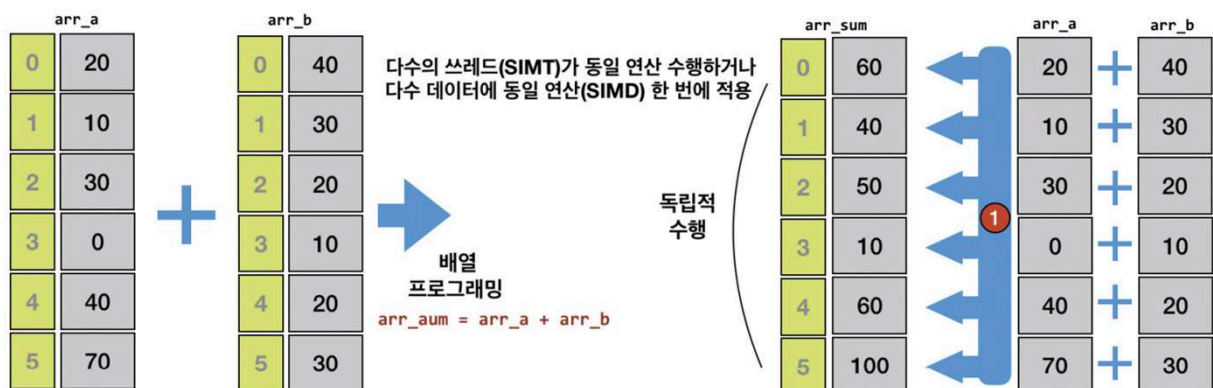
66

13 넘파이는 왜 성공했나



67

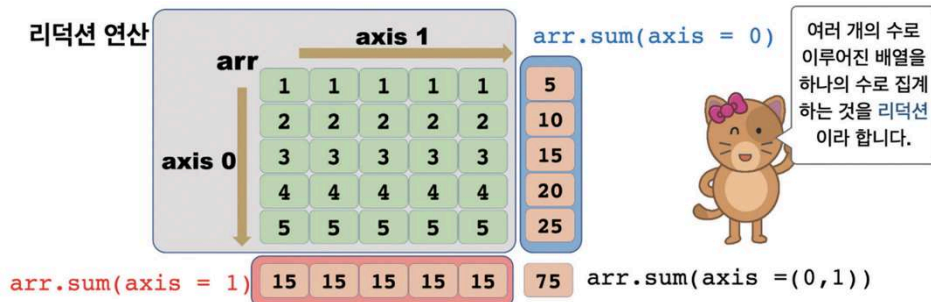
13 넘파이는 왜 성공했나



68

15 리덕션 : 배열을 더 강력하게 만드는 기능

- 넘파이의 다차원 배열은 **리덕션(reduction)**이라는 이름의 멋진 기능을 제공한다.
- 리덕션이라는 것은 여러 개의 수로 이루어진 배열을 하나의 수로 **집계(aggregation)**하는 것이다.
- 넘파이는 이 집계를 위한 함수로 `sum()`, `mean()`, `min()`, `max()` 같은 함수를 제공한다.



69

15 리덕션 : 배열을 더 강력하게 만드는 기능

- 실제 코드를 통해 이 연산을 수행해 보자. 위의 그림과 같은 결과가 나온 것을 확인할 수 있다.

```
arr = np.array([[1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                [1,2,3,4,5],
                ])
print(np.sum(arr, axis = 0))
print(np.sum(arr, axis = 1))
np.sum(arr, axis=(0, 1)) # axis = (0, 1)은 생략 가능
```

```
[ 5 10 15 20 25]
[15 15 15 15 15]
75
```

70

16 선형방정식을 풀어보자

- 넘파이에는 `numpy.linalg`이라는 서브 모듈이 있으며 이 모듈은 선형대수를 위한 다양한 함수를 제공한다.
- 아래와 같이 x, y 두 개의 미지수를 가지는 선형 연립 방정식이 있다고 가정하자.

$$x + 2y = 6$$

$$x - 3y = 1$$

- 이 방정식의 해를 넘파이의 `linalg` 서브 모듈에서 다음과 같은 함수를 호출하여 구할 수 있다.

```
a = np.array([[1, 2], [1, -3]])  
b = np.array([6, 1])  
s = np.linalg.solve(a, b)  
print(s)
```

```
[4. 1.]
```

71

16 선형방정식을 풀어보자

- 연립 방정식이 해를 가지는 경우도 있지만 그렇지 않은 경우도 있다. 이를 판별하기 위해서는 행렬식을 이용할 수 있다.
- **행렬식**(determinant)은 정방 행렬에 수를 대응시키는 함수인데, 연립 방정식이 해를 가지는지 아닌지를 판별하는 데 사용할 수 있다.
- 다음은 2×2 , 3×3 크기의 행렬에 대한 행렬식을 구하는 방법이다.

- 2×2 행렬의 행렬식 $\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc$

- 3×3 행렬의 행렬식 $\det \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bgf + cdh - ceg - bdi - afh$

72

16 선형방정식을 풀어보자

- 2×2 행렬과 3×3 행렬의 행렬식은 위의 수식을 이용하여 구할 수 있는데, 넘파이를 이용하면 아래와 같이 `det()`라는 한 줄의 함수 호출을 통해 간단하게 구할 수 있다.

```
 a = np.array([[2, 1], [4, 5]])  
det_a = np.linalg.det(a)           # a 행렬의 행렬식  
b = np.array([[1, 2], [3, -6]])  
det_b = np.linalg.det(b)           # b 행렬의 행렬식  
print(det_a, det_b)  
  
6.0 -12.0
```

73

17 배열의 결합 concatenate, vstack, hstack

- 여러 개의 다차원 배열을 결합하여 하나의 다차원 배열을 만들 때 사용할 수 있는 넘파이 함수에 대해서 알아보자. 우선 다음과 같은 다차원 배열 a, b, c와 리스트 d를 만들어 보자.

```
 a = np.arange(10, 18).reshape(2,4)  
b = np.arange(12).reshape(3,4)  
c = np.arange(8).reshape(2,4)  
d = list(range(4))  
a # (2, 4) 형상의 다차원 배열로 10에서 17까지의 원소를 가짐  
  
array([[10, 11, 12, 13],  
       [14, 15, 16, 17]])  
  
 b # (3, 4) 형상의 다차원 배열로 0에서 11까지의 원소를 가짐  
  
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

74

17 배열의 결합 concatenate, vstack, hstack

▶ c # (2, 4) 형상의 다차원 배열로 0에서 7까지의 원소를 가짐

```
array([[0, 1, 2, 3],  
       [4, 5, 6, 7]])
```

▶ d # d는 1차원 리스트로 0에서 3까지의 원소를 가짐

```
[0, 1, 2, 3]
```

- 리스트를 결합하기 위해서는 + 연산자를 사용할 수 있으나 다차원 배열의 +는 두 배열의 원소의 합이 되므로 이 연산자 대신 다음과 같이 넘파이의 concatenate() 함수를 사용해서 두 리스트를 결합할 수 있다.

▶ np.concatenate((a, b)) # a 배열의 마지막 원소 다음에 b 배열을 넣는다

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17],  
       [ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

75

17 배열의 결합 concatenate, vstack, hstack

- 이 concatenate() 함수는 a 배열의 마지막 원소 다음에 b 배열의 원소를 차례로 넣는 방식으로 두 다차원 배열을 결합하고 이를 반환한다.
- 넘파이의 vstack(), hstack()은 이와 비슷하게 수직 방향(vertical)과 수평 방향(horizontal)으로 다차원 배열을 결합한다.
- 위의 함수는 다음과 동일한 함수로 구현할 수 있다.

▶ np.vstack((a, b)) # a 배열과 b 배열을 수직 방향으로 결합

```
array([[10, 11, 12, 13],  
       [14, 15, 16, 17],  
       [ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

76

17 배열의 결합 concatenate, vstack, hstack

- 넘파이의 vstack()은 배열과 리스트를 결합할 수 있으며, 다차원 배열 3개를 한꺼번에 결합하는 것도 가능하다.

 np.vstack((a, d)) # a 배열과 d 리스트의 결합

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [ 0,  1,  2,  3]])
```

 np.vstack((a, b, d)) # 다차원 배열 3개를 결합

```
array([[10, 11, 12, 13],
       [14, 15, 16, 17],
       [ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [ 0,  1,  2,  3]])
```

77

17 배열의 결합 concatenate, vstack, hstack

- 반면 hstack()은 다음과 같이 axis 0의 차원값이 동일한 때에만 두 벡터를 수평축으로 결합하는 기능이 있다.

 np.hstack((a, c)) # 다차원 배열 a, c를 수평으로 결합

```
array([[10, 11, 12, 13, 0, 1, 2, 3],
       [14, 15, 16, 17, 4, 5, 6, 7]])
```

 # a 배열은 형상이 (2, 4), b 배열은 (3, 4)로 수평 결합이 안 됨
np.hstack((a, b)) # a 배열과 b 배열의 hstack 결합-오류

```
ValueError: all the input array dimensions for the concatenation axis must
match exactly, but along dimension 0, the array at index 0 has size 2 and
the array at index 1 has size 3
```

78

18 배열을 결합하는 `r_`, `c_` 클래스와 `column_stack()` 함수

- 여러 개의 다차원 배열을 결합하는 또 다른 방법을 살펴보자.
- 이를 위하여 다음과 같은 4개의 다차원 배열을 생성하자.

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[10, 20, 30], [40, 50, 60]])
```

- 이제 a와 b를 연결하는데 `concatenate()`나 `hstack()` 대신 `r_` 클래스를 이용하여 연결하는 간단한 방법도 있다.
- `r_`은 큰 괄호 []안에 연결할 원소를 넣거나 다차원 배열을 넣어서 배열을 연결하는 간단하면서도 강력한 클래스이다.
- 이때 연결하는 방향은 첫 번째 축인 `axis=0` 방향이 된다.

79

18 배열을 결합하는 `r_`, `c_` 클래스와 `column_stack()` 함수

```
np.r_[a, b] # a, b 배열을 결합함 np.hstack([a, b])와 같다
```

```
array([ 1,  2,  3, 10, 20, 30])
```

```
np.r_[a, 4, 5, 6, b] # 중간에 개별 원소 삽입도 가능하다
```

```
array([ 1,  2,  3,  4,  5,  6, 10, 20, 30])
```

```
np.r_[[0]*3, 5, 6]
```

```
array([0, 0, 0, 5, 6])
```

```
np.r_[c, d] # 2차원 배열을 axis=0 방향으로 결합한다
```

```
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [10, 20, 30],
       [40, 50, 60]])
```

80

18 배열을 결합하는 r_, c_ 클래스와 column_stack() 함수

- 만일 a와 b를 2차원 배열로 만들어서 r_ 클래스로 결합한다면 다음과 같은 결과를 얻을 수 있을 것이다.

```
# a, b를 2차원 배열로 만들어서 axis=0 방향으로 결합
np.r_[a.reshape(-1,1), b.reshape(-1,1)]
```

```
array([[ 1],
       [ 2],
       [ 3],
       [10],
       [20],
       [30]])
```

- 다음은 a, b의 각 원소끼리 묶어서 [1, 10], [2, 20], [3, 30]을 원소로 하는 다차원 배열을 만드는 방법인데, a, b를 2차원 배열로 만들고 나서 axis = 1 축으로 결합하는 방법도 있으나 c_ 클래스를 사용하여 묶어주는 방법과 column_stack() 함수를 사용하여 묶어주는 방법도 있다.

81

18 배열을 결합하는 r_, c_ 클래스와 column_stack() 함수

```
np.concatenate((a.reshape(-1, 1), b.reshape(-1, 1)), axis=1)
```

```
array([[ 1, 10],
       [ 2, 20],
       [ 3, 30]])
```

```
np.c_[a, b]
```

```
array([[ 1, 10],
       [ 2, 20],
       [ 3, 30]])
```

```
np.column_stack([a, b])
```

```
array([[ 1, 10],
       [ 2, 20],
       [ 3, 30]])
```

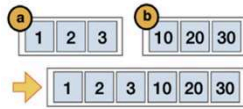
```
# c와 d의 전치행렬을 결합시킨다
np.concatenate((c.T, d.T), axis=1)
```

```
array([[ 1,  4, 10, 40],
       [ 2,  5, 20, 50],
       [ 3,  6, 30, 60]])
```

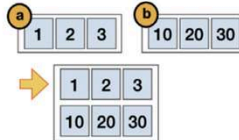
82

18 배열을 결합하는 r_, c_ 클래스와 column_stack() 함수

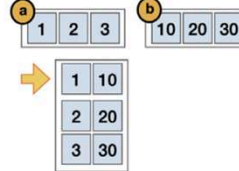
np.r_[a, b]
np.hstack([a, b])
np.concatenate((a, b))



np.r_[[a], [b]]
np.vstack([a, b])



np.c_[a, b]
np.column_stack([a, b])



- 넘파이에는 지나치게 많다고 생각이 들 정도의 다양하고도 풍부하면서도 강력한 배열 함수들과 클래스들이 있는데 이 때문에 과학자들의 많은 사랑을 받고 있다.
- 이 밖에도 많은 함수가 제공되고 있으나 이들은 numpy.org의 넘파이 문서를 참고하도록 하자.

데이터 시각화 도구 맷플롯립



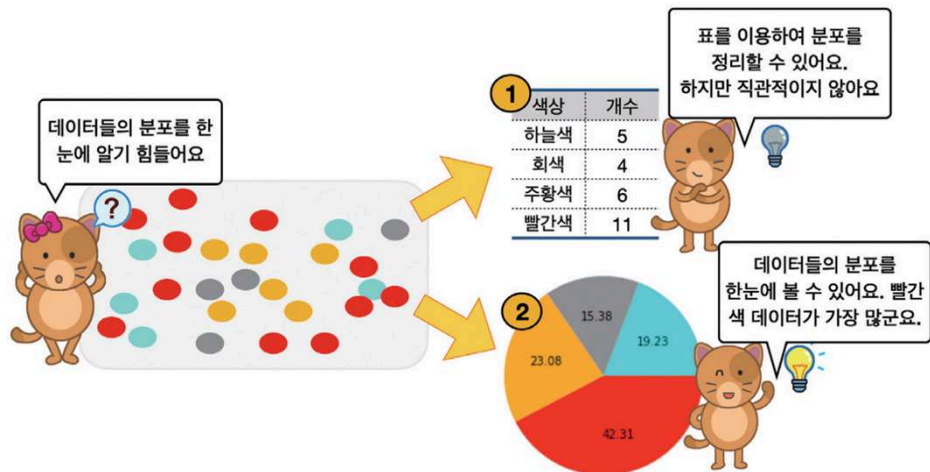
1

1 데이터 과학과 효과적인 시각화의 필요성

- 이번 장에서는 컴퓨터 화면에 시각적 이미지를 이용하여 데이터를 효과적으로 보여주는 방법인 **데이터 시각화**(data visualization)에 대해 알아보자.
- 일반적으로 사람은 필요한 정보의 80% 가량을 시각 시스템을 통해서 받아들인다고 알려져 있다.
- 따라서, 효과적인 시각화는 사용자가 데이터를 분석하고 추론하는 데 도움이 된다. 사람들은 수치 데이터보다 시각적으로 보이는 그림을 더 직관적으로 이해할 수 있기 때문이다.

2

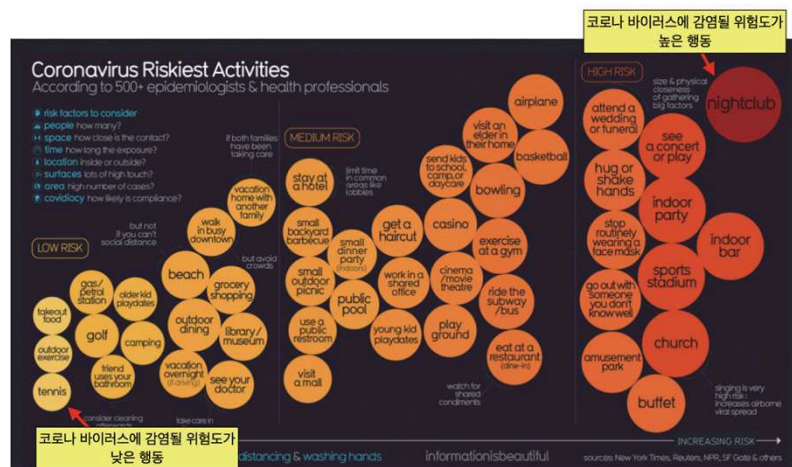
1 데이터 과학과 효과적인 시각화의 필요성



3

1 데이터 과학과 효과적인 시각화의 필요성

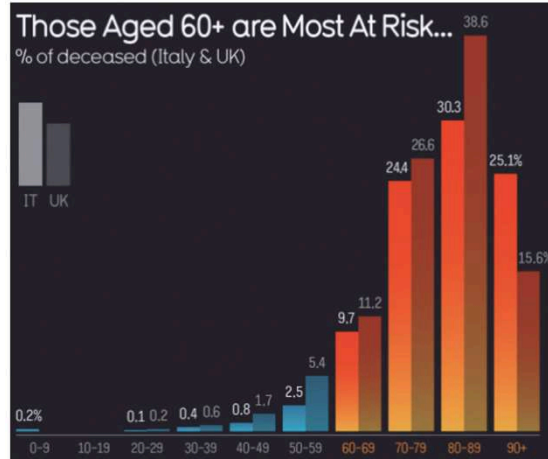
- informationisbeautiful.net이라는 웹사이트에서 제공하고 있는 멋진 시각화 사례의 일부이다.
- 이 웹사이트에서는 어떠한 행동이 코로나19 바이러스 감염에 취약한 행동인가를 다양한 색상과 크기를 가진 원으로 잘 표현하고 있다.



4

1 데이터 과학과 효과적인 시각화의 필요성

- COVID-19 감염질환은 나이가 많은 감염자의 치명률이 높지만 나이가 어린 감염자는 치명률이 낮은 편이다.
- 이처럼 데이터의 수집과 전처리 못지 않게 **시각화를 통해서 그 의미를 잘 전달하는 것** 역시 데이터 과학자가 해야 할 중요한 일이다.



5

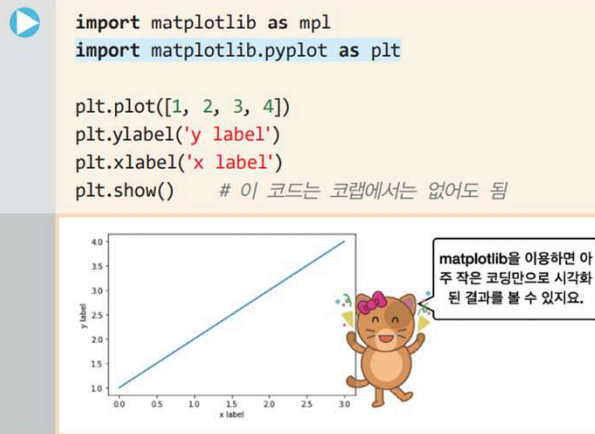
2 데이터 과학을 위한 시각화 도구 matplotlib

- 데이터를 시각화하기 위한 많은 라이브러리가 파이썬에 있지만 우리는 **matplotlib** 라이브러리를 사용할 것이다. **matplotlib**은 파이썬에서 가장 널리 사용되는 시각화 도구로 간단한 막대 그래프, 선 그래프, 산점도를 빠르게 그리는 용도로 적합하다.
- 그뿐만 아니라 고차원적인 데이터를 정밀하게 시각화하는 고급 기능도 가지고 있다. 우리는 **matplotlib**의 하위 모듈 중에서 **pyplot**이라는 서브(하위) 모듈을 주로 사용할 것이다. 이 서브 모듈에는 **시각화를 위한 핵심적인 함수들과 클래스들이 정의되어 있다.**

6

2 데이터 과학을 위한 시각화 도구 matplotlib

- 다음과 같은 간단한 예제 코드를 입력해서 레이블을 가진 이차원 평면에서 선 차트를 만들어보자.



7

2 데이터 과학을 위한 시각화 도구 matplotlib

- **matplotlib**의 별칭은 **mpl**로, 이 모듈의 서브 모듈인 **pyplot**은 보통 **plt**라는 별칭으로 사용한다.
- **plt**의 **plot()** 함수는 리스트 자료값 **[1, 2, 3, 4]**를 **y** 값으로 삼아 화면에 그려주는 일을 한다 이때, **x** 값의 범위를 생략할 수 있는데 이 값은 디폴트로 **[0, 1, ... n]**으로 설정된다. **n**은 **y** 값의 개수에 의해 결정되므로, 여기서는 **[0, 1, 2, 3]**이 **x** 값의 범위가 된다. 따라서, 위의 코드에서 **plot([1, 2, 3, 4])** 부분은 다음 코드와 동일하다.

```
plt.plot([0, 1, 2, 3], [1, 2, 3, 4])
```

8

2 데이터 과학을 위한 시각화 도구 matplotlib

- `plt.xlabel()`과 `plt.ylabel()`은 각각 x 축과 y 축에 레이블을 적어주는 코드이며 각 축의 속성을 기록하면 된다.
- 그리고, 가장 하단에 있는 `plt.show()`는 화면에 이 그래프를 그려주는 일을 하는데 구글의 코랩과 같은 주피터 노트북 기반의 환경에서는 이 코드를 생략할 수 있다.
- 하지만 그 외의 경우에는 반드시 이 코드가 있어야만 화면에 출력이 이루어진다. 이 책에서는 이 부분은 생략할 것이다.

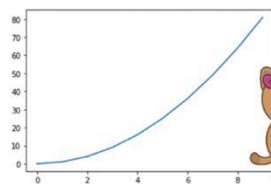
9

2 데이터 과학을 위한 시각화 도구 matplotlib

- `plot()` 함수의 입력값으로 리스트를 사용할 수도 있으나, 다음과 같이 넘파이의 다차원 배열을 사용할 수도 있다. 이 경우 브로드캐스팅 기능을 통해서 쉽게 2차 함수를 그려볼 수 있다. 아래의 코드는 $y = x^2$ 함수를 시각화한 것으로 x 축의 구간은 0에서 9 사이의 값이다.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(10) # 0에서 9 사이의 정수값을 생성함
plt.plot(x**2)    # x^2 함수를 그린다
```

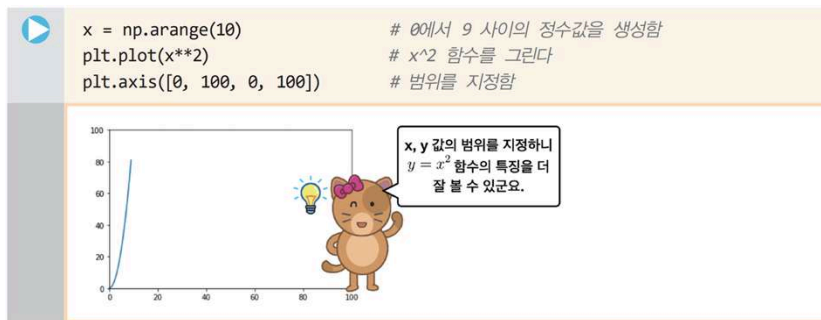


넘파이의 브로드캐스팅을 사용해서, $y = x^2$ 함수를 그려볼 수 있군요.

10

2 데이터 과학을 위한 시각화 도구 matplotlib

- 이 코드에서 주의해야 할 내용은 **x** 축의 축척이 0에서 9 사이의 범위를 가지는 데 비해, **y** 축은 0에서 81까지의 범위를 가진다는 점이다. 이 구간의 크기를 비슷하게 설정하려면 다음과 같이 **plt.axis()** 함수를 이용하여 **x** 축의 범위와 **y** 축의 범위를 모두 0에서 100으로 설정할 수 있다. 이 코드를 추가하여 실행하면 이전과는 달리 **x** 값이 증가할 경우 **y** 값이 매우 가파르게 증가하는 형태의 함수 모양을 제대로 볼 수 있다.



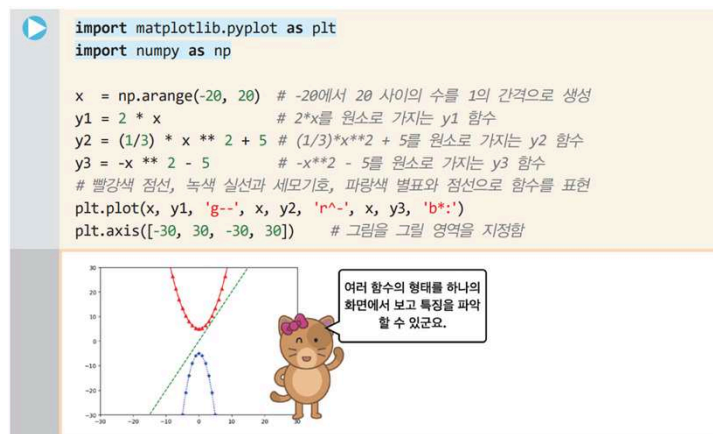
11

3 plot() 함수의 선 그리기 기능들을 알아보자

- 이번에는 **plot()** 함수를 수정하여 한 번의 호출만으로 $2x$, $(1/3)x^2 + 5$, $-x^2 - 5$ 의 세 함수를 그려보도록 하자.

- 이때 한 화면에 세 개의 함수가 모두 나타나야 하기 때문에 코드는 아래와 같이 다소 복잡하게 보일 수 있다.

- 이때, 다음과 같이 **plot()** 함수를 한 번만 호출하여 세 개의 함수를 그려보도록 하자.



12

3 plot() 함수의 선 그리기 기능들을 알아보자

- 첫 두 줄은 그림을 그리기 위한 **pyplot** 서브 모듈을 가져오는 작업이며, 마찬가지로 넘파이를 **np**라는 별칭으로 사용하는 작업이다. 또한 다음 코드는 넘파이에 서 지정된 구간에 대하여 수의 시퀀스를 생성하여 다차원배열 **x**에 할당하는 코드이다.

```
x = np.arange(-20, 20) # -20에서 20 사이의 수를 1의 간격으로 생성
```

- 그 아래의 코드 역시 **y1, y2, y3** 함수를 정의하는 내용이다.

```
y1 = 2 * x          # 2*x를 원소로 가지는 y1 함수
y2 = (1/3) * x ** 2 + 5 # (1/3)*x**2 + 5를 원소로 가지는 y2 함수
y3 = -x ** 2 - 5      # -x**2 - 5를 원소로 가지는 y3 함수
```

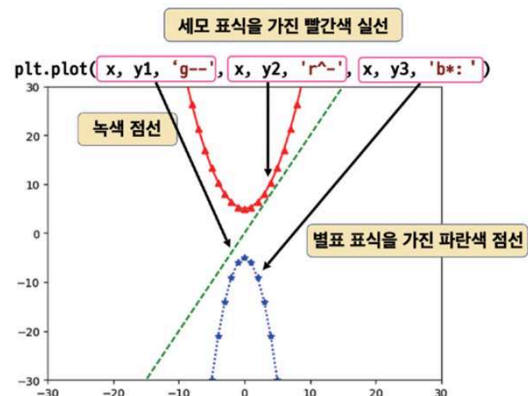
13

3 plot() 함수의 선 그리기 기능들을 알아보자

- 다음의 코드는 **y1, y2, y3** 함수를 각각 다른 스타일로 그리는 코드이다.

```
plt.plot(x, y1, 'g--', x, y2, 'r^--', x, y3, 'b*:')
```

- 이 코드에서 **plot** 함수의 첫 인자와 두 번째 인자는 **x, y1**이 사용되었다.
- **x, y1**은 **x** 값과 **y** 값을 각각 나타내는데, 세 번째 인자인 **'g--'**인자를 통해서 이 함수를 어떻게 그릴지를 지정한다.
- 여기서는 녹색(green:g) 점선(--) 형태의 선으로 이 함수를 그릴 것을 기술하고 있다.



14

3 plot() 함수의 선 그리기 기능들을 알아보자

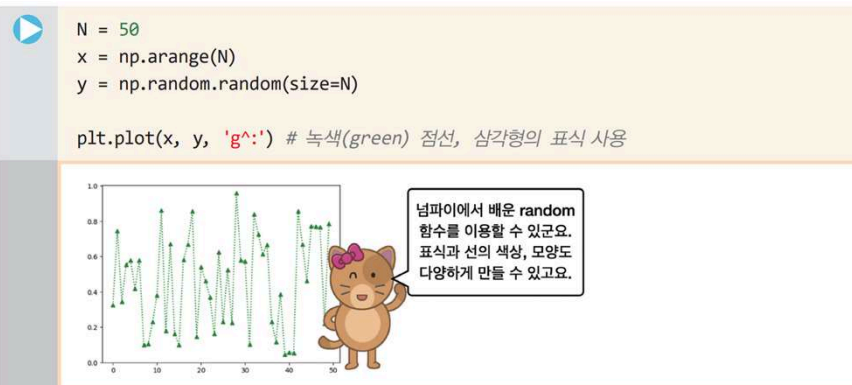
- 다음으로 나타나는 **x, y2** 역시 **y2** 함수를 그리라는 명령인데 **'r^.'** 인자를 통해서 이 함수를 어떻게 그릴지 지정한다.
- 여기서는 빨간색(**red:r**)의 실선(-)을 그리는데 해당 좌표에 세모(삼각형) 모양의 표식(^)을 나타낼 것을 기술한다.
- 다음으로 **y3** 함수는 **'b*.'**를 통해서 파란색(**blue:b**), 점선(.), 그리고 별표 표식(*)을 화면에 표시할 것을 나타내고 있다.
- 왼쪽의 표는 matplotlib.org 홈페이지에서 제공하는 다양한 **표식marker**의 일부이다.

marker	symbol	description
"."	•	point
"f"	.	pixel
"o"	●	circle
"v"	▼	triangle_down
"^"	▲	triangle_up
"<"	◀	triangle_left
">"	▶	triangle_right
"1"	⌵	tri_down
"2"	⌶	tri_up
"3"	⌷	tri_left
"4"	⌸	tri_right
"8"	◼	octagon
"s"	■	square
"p"	⬠	pentagon
"P"	⊕	plus (filled)
"*"	★	star

15

4 복잡한 선을 그리고 이미지로 저장하자

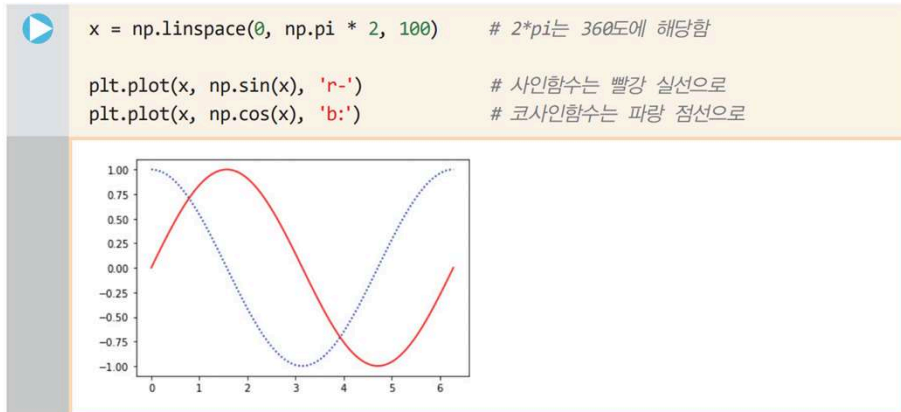
- **plot()** 함수는 매우 강력한 기능이 있는데 우선 다음과 같이 **0**에서 **50** 사이의 **x** 구간에 대하여, **0**에서 **1** 사이의 임의의 **y** 값을 넘파이의 **random()** 함수를 사용해서 생성하고 그려보자.



16

4 복잡한 선을 그리고 이미지로 저장하자

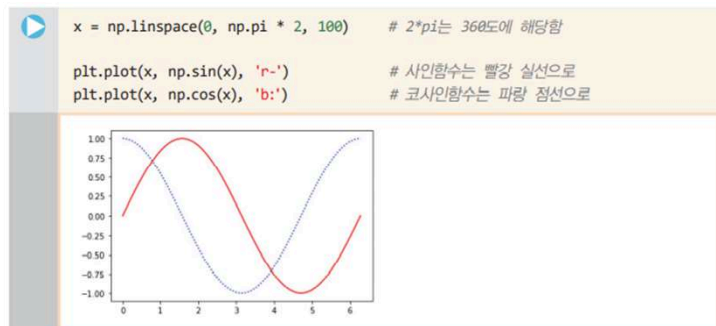
- 난수 자료값을 그린 선분 대신, 넘파이의 `sin()`, `cos()`과 같은 주기함수를 사용하면 다음과 같이 나타낼 수 있다.



17

4 복잡한 선을 그리고 이미지로 저장하자

- 난수 자료값을 그린 선분 대신, 넘파이의 `sin()`, `cos()`과 같은 주기함수를 사용하면 다음과 같이 나타낼 수 있다.
- 이와 같이 만들어진 그림은 **figure** 객체의 **savefig()** 메소드를 사용하여 저장할 수 있다.
- 이를 위해서는 반드시 다음과 같이 **plt.figure()**를 통해서 **figure** 객체를 생성하여야 한다.
- 여기에서는 **sin_cos_fig.png**와 같이 **png** 파일 형식의 파일로 저장할 것을 지시하고 있다.



18

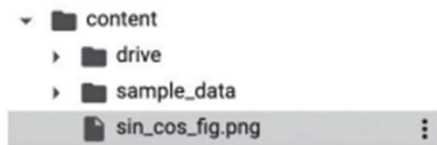
4 복잡한 선을 그리고 이미지로 저장하자

- **savefig()** 메소드로 저장 가능한 파일 형식은 **png, jpg, svg, tif, pdf**를 비롯한 10여 가지 이상의 다양한 파일 형식이 있다.

- 이 파일 형식은 **fig.canvas.get_supported_filetypes()** 명령으로 확인해 볼 수 있다.

```
x = np.linspace(0, np.pi * 2, 100)
fig = plt.figure()
plt.plot(x, np.sin(x), 'r~')
plt.plot(x, np.cos(x), 'b:')
fig.savefig('sin_cos_fig.png')
```

- **savefig()**로 저장된 파일은 구글의 코랩을 사용할 경우 다음과 같이 시스템의 **root** 디렉토리 아래 **content** 디렉토리에 저장되며 오른쪽 클릭을 통해서 여러분의 컴퓨터로 다운로드하는 것도 가능하다.



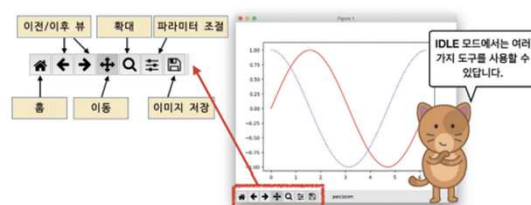
19

4 복잡한 선을 그리고 이미지로 저장하자

- 또한 다음과 같은 명령을 통해서 현재 노트북 화면에 이 그림을 불러올 수도 있다. 이 명령은 **IPython.display**라는 서브 모듈에 있는 **Image** 객체를 사용하여 파일의 내용을 화면에 출력하는 기능이다.

```
from IPython.display import Image
Image('sin_cos_fig.png')
```

- 만일 구글 코랩에서 실행하지 않고 코드를 **IDLE**에서 실행하게 되면 아래 그림과 같은 윈도우가 나타나는데 이 윈도우의 하단에 나타나는 아이콘을 이용하여 홈으로 이동하거나 이전, 이후 뷰로 이동, 단순 이동, 확대, 파라미터 조절, 이미지 저장 등의 기능을 이용할 수 있다.



20

5 제목과 레이블, 스타일에 대해 알아보자

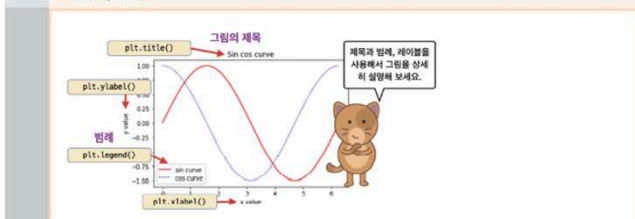
- 앞 절의 그림에 제목과 범례 등을 넣어서 그림을 좀 더 이해하기 쉽게 만드는 방법을 알아보자.

- 그림의 제목은 `plt.title()` 함수로 만들 수 있으며, `plt.xlabel()`, `plt.ylabel()`로 x 축, y 축에 레이블링하는 것도 가능하다.

- 또한, `legend()` 함수를 통해서 범례를 추가하는 것도 가능하다.

```
x = np.linspace(0, np.pi * 2, 100)

plt.title('Sin cos curve')
plt.plot(x, np.sin(x), 'r-', label='sin curve')
plt.plot(x, np.cos(x), 'b-', label='cos curve')
plt.xlabel('x value')
plt.ylabel('y value')
plt.legend()
```



21

5 제목과 레이블, 스타일에 대해 알아보자

- `legend()`를 사용하여 범례를 추가하기 위해서는 `plot()` 메소드의 `label` 인자에 이 메소드의 설명글을 추가하는 작업이 필요하다.
- 만일 `label` 인자를 주지 않으면 `legend()` 메소드는 아무것도 출력하지 않을 것이다. 이 범례의 위치를 조절할 수도 있는데 `legend()`의 `loc` 인자를 다음과 같이 설정하면 범례의 위치는 그림의 위쪽 오른쪽이 된다.

```
plt.legend(loc='upper right') # 위쪽 오른쪽에 범례를 표시
```

- matplotlib에서 사용할 수 있는 범례의 위치는 다음과 같다.

```
'best', 'upper right', 'upper left', 'lower left', 'lower right', 'right', 'center left', 'center right', 'lower center', 'upper center', 'center'
```

22

5 제목과 레이블, 스타일에 대해 알아보자

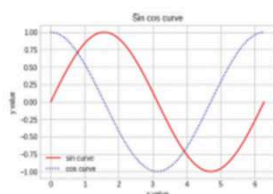
- **matplotlib**의 그림을 그릴 때는 하나의 스타일만이 아닌 다양한 스타일을 사용할 수 있다.
- 이를 테마 혹은 **스타일시트 stylesheet**라고 하는데, 다음과 같은 코드를 추가하면 격자모양의 무늬가 나타나는 것을 볼 수 있다.

```
plt.style.use('seaborn-whitegrid')
```

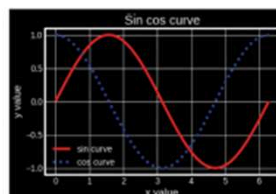
23

5 제목과 레이블, 스타일에 대해 알아보자

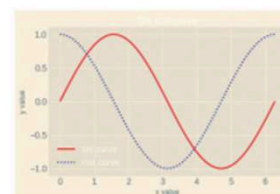
- **plt**의 **style** 객체는 **use()** 메소드를 가지는데 이 메소드는 미리 만들어져있는 그림판의 스타일을 지정할 수 있다.
- 예를 들어 **'seaborn-whitegrid'** 스타일을 지정할 경우 그림의 왼쪽과 같이 흰색 배경에 격자 모양의 패턴이 나타난다. 이처럼 미리 정의된 몇 가지 스타일과 그 스타일에 따른 출력 결과가 아래 그림에 있다.



'seaborn-whitegrid'



'dark_background'



'Solarize_Light2'

24

5 제목과 레이블, 스타일에 대해 알아보자

- 다음 코드는 현재 여러분이 사용하고 있는 **matplotlib**에서 사용 가능한 스타일들의 목록을
- 보여줄 것이며, 디폴트 스타일로 설정을 변경하고 싶다면 **'default'**를 입력한다.

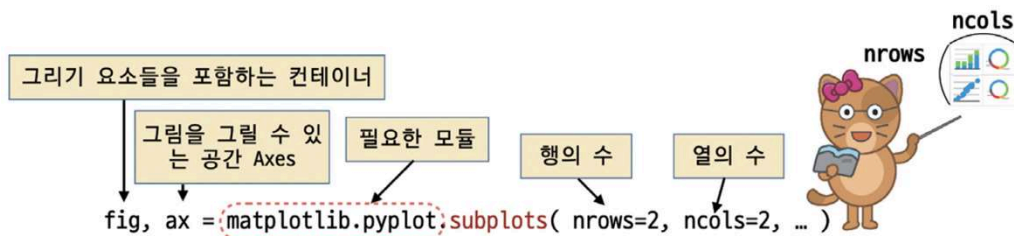
```
print(plt.style.available)

['Solarize_Light2', '_classic_test_patch', 'bmh', 'classic', 'dark_
background', 'fast', 'fivethirtyeight', 'ggplot', 'grayscale', 'seaborn',
'seaborn-bright', 'seaborn-colorblind', 'seaborn-dark', 'seaborn-dark-
palette', 'seaborn-darkgrid', 'seaborn-deep', 'seaborn-muted', 'seaborn-
notebook', 'seaborn-paper', 'seaborn-pastel', 'seaborn-poster', 'seaborn-
talk', 'seaborn-ticks', 'seaborn-white', 'seaborn-whitegrid', 'tableau-
colorblind10']
```

25

6 Figure, axes에 대하여 살펴보자

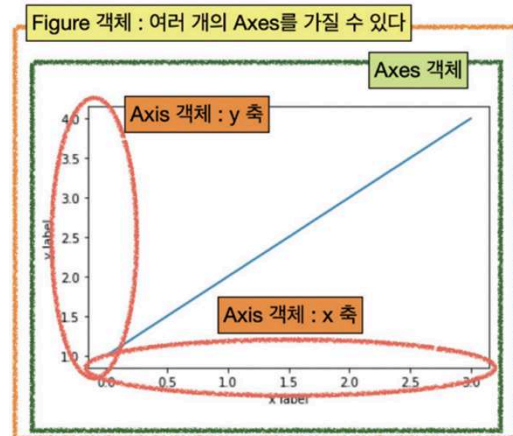
- matplotlib의 pyplot 서브 모듈의 plot() 함수를 사용해서 그림을 그리는 것은 간단한 그림을 빠르게 그리는데 편리하기는 하지만 다양한 종류의 그래프를 한 화면에 여러 개 그리기에는 부족하다.
- 이 기능을 사용하기 위해서는 pyplot 모듈의 subplots() 함수를 사용해야 한다.
- 이 함수는 다음과 같이 사용하며, 주로 필요한 행의 수와 열의 수를 입력한다.



26

6 Figure, axes에 대하여 살펴보자

- `subplots()` 함수는 Figure 객체와 Axes 객체를 반환하는데 Figure 객체는 축과 그래픽, 텍스트 등의 **그리기 객체들을 포함하는 컨테이너 객체**이며, Axes 객체는 그림을 그릴 수 있는 테두리 상자 모양의 공간이다.
- `ax`는 다차원 배열 객체이므로 `ax[0, 0]`과 같은 인덱스를 사용하여 특정한 Axes 객체에 그림을 그릴 수 있다.
- 반면 Axis는 x, y축 객체이니 표기가 유사한 **Axes와 Axis를 혼동하지 않도록 주의**하도록 하자.



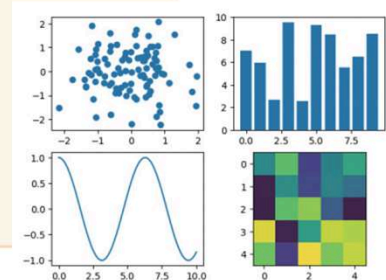
27

6 Figure, axes에 대하여 살펴보자

```

▶ fig, ax = plt.subplots(2, 2) # 2행 2열의 그림을 그리는 공간을 만든다

X = np.random.randn(100) # 정규 분포를 가지는 데이터
Y = np.random.randn(100) # 정규 분포를 가지는 데이터
ax[0, 0].scatter(X, Y) # 산점도 그림
X = np.arange(10) # 0에서 9 사이의 연속값
Y = np.random.uniform(1, 10, 10) # 균일분포값 생성
ax[0, 1].bar(X, Y) # 막대 차트 그림
X = np.linspace(0, 10, 100)
Y = np.cos(X)
ax[1, 0].plot(X, Y) # 실선으로 함수를 그림
Z = np.random.uniform(0, 1, (5, 5))
ax[1, 1].imshow(Z) # 분포를 2D 이미지로 그림
    
```



28

6 Figure, axes에 대하여 살펴보자

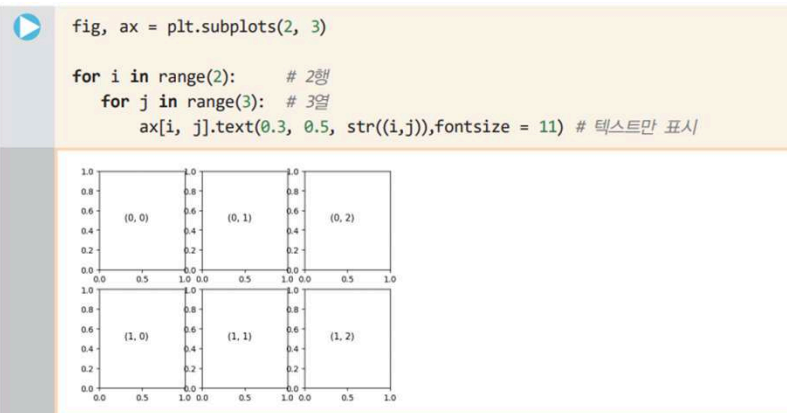
- 이 코드의 결과는 그림과 같이 산점도 그림, 막대 차트 그림, 실선 함수 그림, 이차원 이미지와 같이 각기 다른 스타일의 그림이 나타난다.
- 그림을 그리는 함수와 그 기능은 아래 표와 같다.

<code>scatter()</code>	산점도 그림을 그리는 기능으로 2차원 좌표에 흩뿌려진 점들의 좌표로 자료값을 나타냄.
<code>bar()</code>	막대 그래프를 그리는 기능, 막대의 모양은 수직 막대.
<code>barh()</code>	수평 막대 그래프를 그리는 기능.
<code>pie()</code>	파이 형태의 그림을 그리는 기능.
<code>imshow()</code>	이미지를 화면에 그리는 기능.
<code>plot()</code>	실선이나 점선으로 데이터를 나타냄.

29

7 subplot()의 고급기능

- 다음은 2행 3열의 격자 공간을 만들고 이 공간에 그리기 객체를 배치하는 방법이다.
- 우선 `subplots()` 메소드의 인자를 2, 3으로 두고 모두 6개의 그리드를 생성하였다.



30

7 subplot()의 고급기능

- 아래의 코드에서 인자 2, 3은 행과 열의 수를 나타내며 wspace는 수평 여백을 표시하는데 이 값은 axis의 크기에 대한 상대적인 비율값이다.
- 여기서는 0.4로 설정하여 비교적 여유있는 너비 여백을 지정하였다. hspace 역시 수직 여백을 표시하는 비율값이다.
- 이 값을 **0으로 줄 때 격자 그림 사이의 간격은 사라진다는 점**에 유의하자.

```

▶ fig, ax = plt.subplots(2, 3)

grid = plt.GridSpec(2, 3, wspace=0.4, hspace=0.3)
X = np.random.randn(100) # 정규 분포를 가지는 데이터
Y = np.random.randn(100) # 정규 분포를 가지는 데이터
plt.subplot(grid[0, 0]).scatter(X, Y) # 산점도 그림
    
```

31

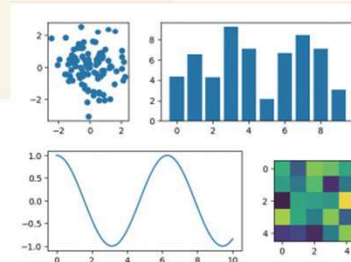
7 subplot()의 고급기능

```

X = np.arange(10) # 0에서 9 사이의 연속값
Y = np.random.uniform(1, 10, 10) # 균일분포값 생성
plt.subplot(grid[0, 1:]).bar(X, Y) # 막대 차트 그림

X = np.linspace(0, 10, 100)
Y = np.cos(X)
plt.subplot(grid[1, :2]).plot(X, Y) # 실선으로 함수를 그림

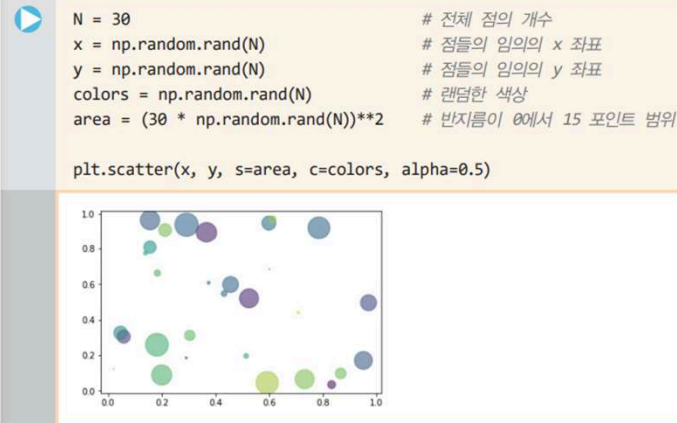
Z = np.random.uniform(0, 1, (5, 5))
plt.subplot(grid[1, 2]).imshow(Z) # 분포를 2D 이미지로 그림
    
```



32

8 자료값의 분포를 나타내는 산점도와 막대 그래프

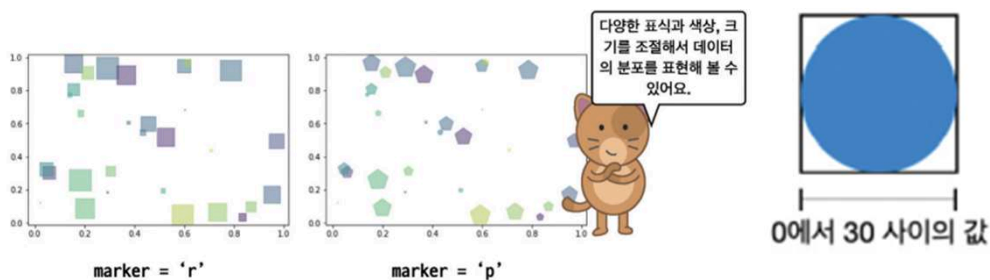
- 산점도는 두 변수의 상관관계를 평면에 데이터의 점으로 표현하는 그래프로 `scatter()` 함수를 사용하면 쉽게 산점도를 그릴 수 있다.



33

8 자료값의 분포를 나타내는 산점도와 막대 그래프

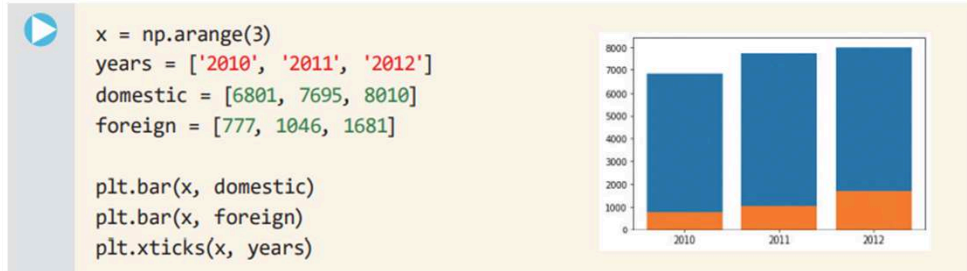
- `scatter()` 함수의 인자 중에서 `s` 키워드 인자는 점의 면적, `c`는 점의 색상, `alpha`는 투명도를 나타낸다.
- 점의 면적 `s`는 포인트 단위로 출력하는데 여기에서는 $(30 * \text{랜덤값})$ 의 거듭제곱을 사용한다.
- 넘파이의 랜덤값의 범위가 0에서 1 사이의 값이므로 이 범위는 0에서 30가 된다.
- `marker` 키워드 인자를 넣지 않으면 디폴트 표식은 원이 되는데, 이 표식을 그림과 같이 사각형(`r:rectangle`)과 오각형(`p:pentagon`) 등의 다양한 모양으로 변경할 수도 있다.



34

8 자료값의 분포를 나타내는 산점도와 막대 그래프

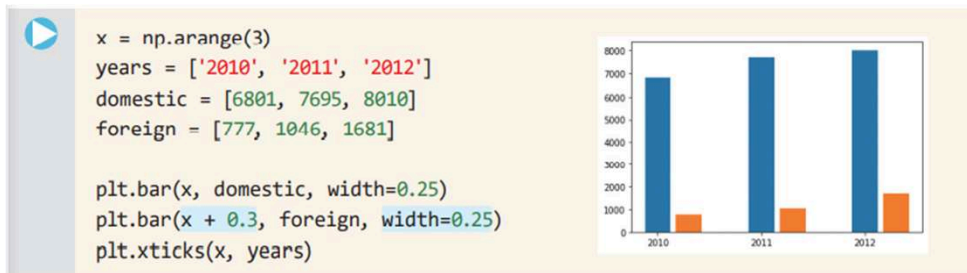
- 막대 그래프는 기본적으로 세로 직사각형 모양으로 데이터값을 나타낼 수 있다.
- plt.bar() 함수 사용
- plt.xticks(x, years) 를 이용하여 x축으로 연도(years) 눈금값 표시가 가능함(그림과 같이 2010, 2011, 2012 눈금이 나타남)



35

8 자료값의 분포를 나타내는 산점도와 막대 그래프

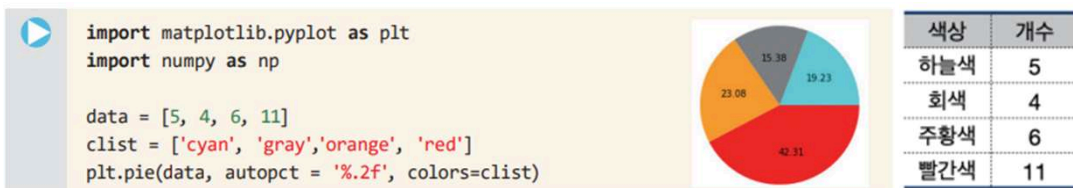
- 다소 보기에 불편한 이전의 자료값은 다음과 같이 **width** 키워드 인자를 통해 막대의 너비를 0.25로 줄 수 있다.
- 이 값의 의미는 원래 두께의 25%를 의미하며, x 값에 0.3을 더하여 x 축의 간격을 조절하는 것도 가능하다.



36

9 파이 차트와 히트맵 표현

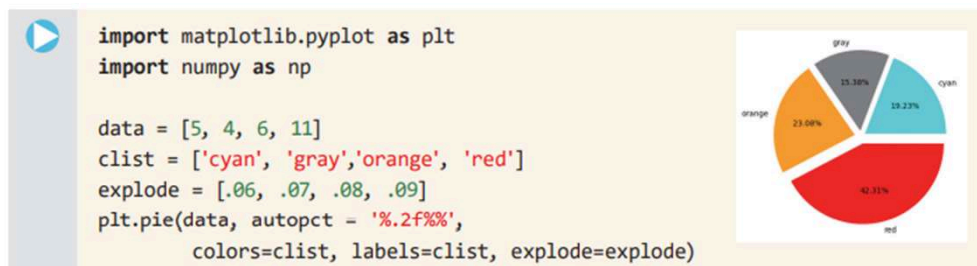
- 이 장의 첫 그림에 나타난 다음과 같은 분포를 가지는 자료들이 있다고 가정하자.
- 이와 같은 범주형 데이터(categorical data)들이 가지는 전체적인 비율을 한눈에 보고자 할 때는 막대 그래프나 산점도 그래프보다 파이 형태의 그래프가 매우 적절할 것이다.
- 이를 위하여 pie() 함수를 사용하도록 하자



37

9 파이 차트와 히트맵 표현

- pie() 함수의 인자인 autopct는 부채꼴 안에 표시될 숫자의 형식을 지정하며 %.2f 값은 소수점 아래 둘째 자리까지를 표시하라는 명령이다. 아래에서 explode 인자는 각 항목이 파이의 원점에서 튀어나오는 정도를 나타낸다.
- 만일 %까지 표시하고자 할 경우 %.2f%% 와 같이 %를 두개 연속해서 입력해야만 한다. colors 키워드 인자는 각 색상의 색상을 지정하는 역할을 하는데 이를 생략하면 디폴트 색상을 부여한다.



38

9 파이 차트와 히트맵 표현

- **히트맵**heat map이란 열을 의미하는 **heat**와 지도를 의미하는 **map**을 결합한 단어로 이미지 위에 **열 분포 형태로 정보를 표현하는 방법**을 의미한다.



- 열 분포란 이 사례와 같이 실제 지표의 온도를 의미하기도 하지만 데이터의 값이 클 경우와 작은 경우에 대하여 서로 다른 색상을 사용하여 데이터의 분포를 쉽게 이해할 수 있는 유용한 기능이다.

39

9 파이 차트와 히트맵 표현

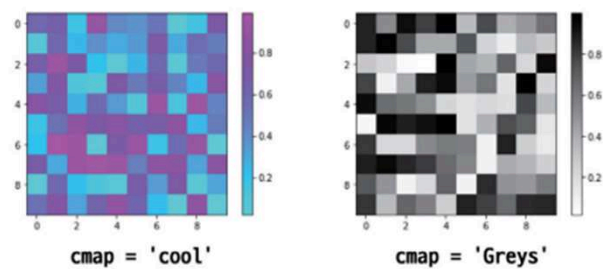
- 다음의 코드는 10×10 크기의 격자 공간에 0에서 1 사이의 임의의 수를 할당한 후 이 값들을 히트맵을 이용하여 그려본 것이다.
- 그림의 오른쪽에 있는 색상 막대를 살펴보면 이 색상의 범위가 0에서 1 사이임을 알 수 있다.



40

9 파이 차트와 히트맵 표현

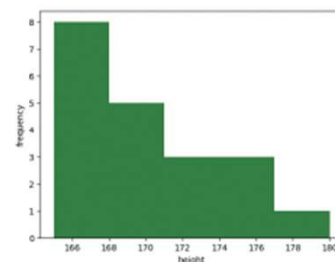
- `imshow()` 함수 내의 키워드 인자로 **color map**의 약어인 **cmap**을 넣을 수 있는데, 이 키워드 인자에 다음과 같이 'cool', 'Greys' 등과 같은 색상맵 타입을 설정하여 다양한 색상을 가진 히트맵을 얻을 수 있다.



41

10 히스토그램

- **히스토그램** `histogram`은 주어진 자료를 몇 개의 구간으로 나누고 각 구간의 **도수** `frequency`를 조사하여 나타낸 막대 그래프이다. 이 그림의 히스토그램의 가로 축은 구간, 세로 축은 도수를 나타낸다.



- 히스토그램의 예제로 동민이네 반 30명 학생의 키를 조사하여 그 분포를 지정된 구간에 나타내고자 한다. 다음의 `heights` 넘파이 배열 자료는 다음과 같이 학생들의 키를 모은 자료이다.

```
heights = np.array([175, 165, 164, 164, 171, 165, 160, 169, 164, 159, 163, 167,
163, 172, 159, 160, 156, 162, 166, 162, 158, 167, 160, 161, 156, 172, 168, 165, 165,
177])
```

42

10 히스토그램

- 이 자료들을 사용하여 히스토그램을 만들어주려면 데이터의 값을 동일한 구간으로 나눠야 한다. 데이터가 들어가는 통을 **빈bin**이라고 한다.



- 다음 코드는 **heights** 자료값을 6개의 빈에 넣어서 히스토그램으로 시각화하는 코드이다. **plt.hist()** 함수에 **heights** 인자와 **bins** 키워드 인자를 넣는 것만으로 손쉽게 히스토그램을 그려볼 수 있다.

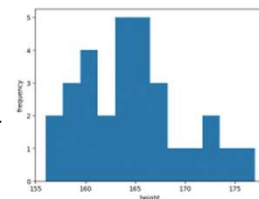
```
heights = np.array([175, 165, 164, 164, 171, 165, 160, 169, 164, 159, 163,
167, 163, 172, 159, 160, 156, 162, 166, 162, 158, 167, 160, 161, 156, 172,
168, 165, 165, 177])
plt.hist(heights, bins=6) # 히스토그램 bins는 구간의 수
plt.xlabel("height")
plt.ylabel("frequency")
```



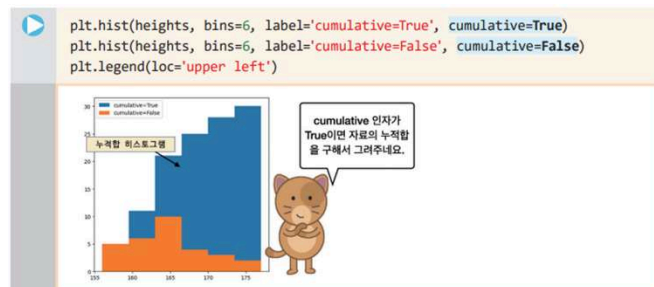
43

10 히스토그램

- 이 히스토그램의 bins 값을 크게 하면 할수록 그림과 같이 많은 구간에 자료값이 들어가는 것을 볼 수 있는데, 오른쪽 그림과 같이 bins가 12로 너무 크면 오히려 자료값들의 분포를 살펴보는 데 방해가 될 수 있다.



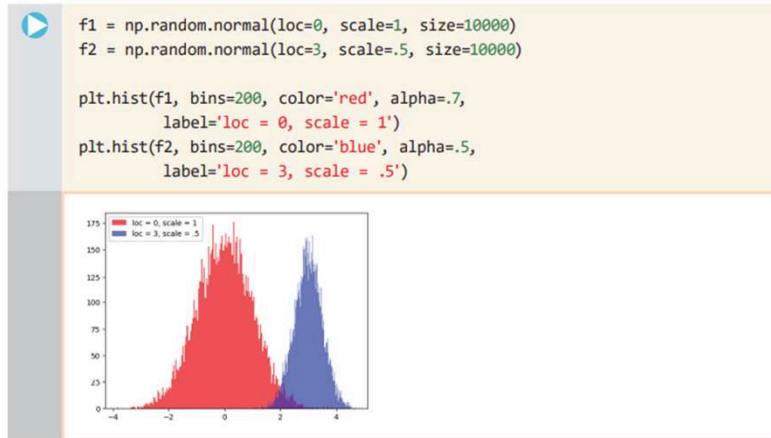
- 이 코드는 아래 그림의 결과와 같이 청색 히스토그램에 누적된 자료값의 합을 출력하고 있는데, 데이터의 개수가 30이므로 가장 오른쪽의 히스토그램 높이가 30인 것도 확인해 볼 수 있다.



44

11 히스토그램을 이용한 정규 분포 함수와 확률 밀도 함수 그리기

- 정규 분포(normal distribution)란 연속 확률 분포의 하나이며 평균이 0이고 분산이 1인 정규 분포를 표준 정규 분포라 한다.



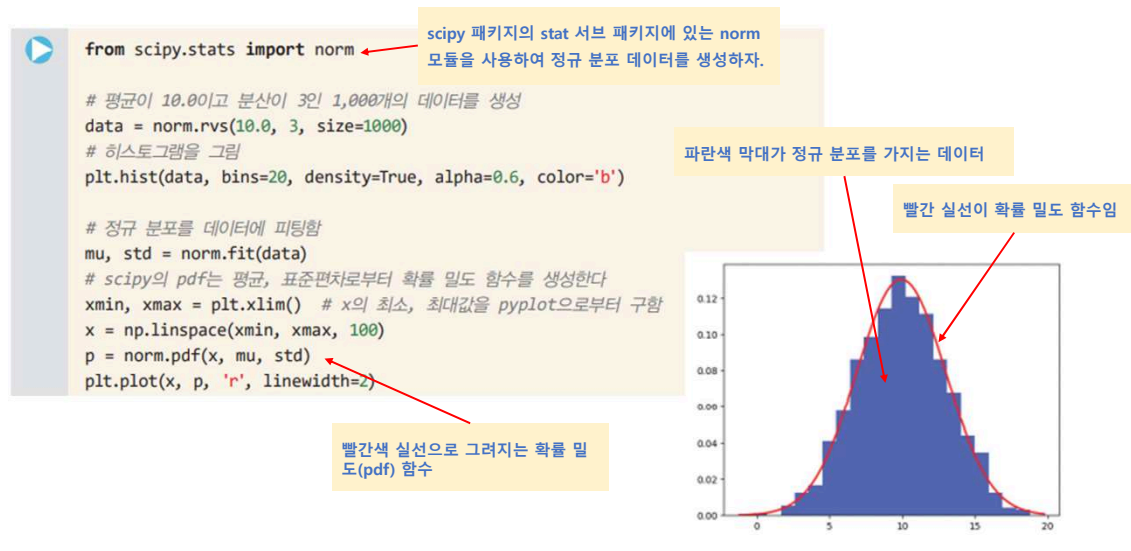
45

11 히스토그램을 이용한 정규 분포 함수와 확률 밀도 함수 그리기

- 다음 코드를 살펴보면 1,000개의 데이터를 생성하여 히스토그램으로 그리는 과정은 앞의 내용과 동일하다.
- 이때 hist() 함수의 density 인자가 True로 설정되어 있음을 눈여겨 보도록 하자. 이 값이 True로 설정될 경우 히스토그램을 그리면서 동시에 **확률 밀도(probability density)** 값을 반환한다.
- scipy 패키지의 stat 서브 패키지에 있는 norm 모듈을 사용해 보자. 이 모듈의 norm.rvs() 함수는 랜덤 변수 생성 함수이다.
- 따라서 이 데이터를 바탕으로 norm.fit() 함수를 사용할 경우 **평균(mu)**과 **표준편차(std)**를 얻을 수 있다.
- 이 평균과 표준편차를 바탕으로 **확률 밀도 함수(probability density function)**를 빨간색으로 그리게 되면 아래 그림과 같이 히스토그램과 확률 밀도 함수를 한 화면에 그려볼 수 있다.

46

11 히스토그램을 이용한 정규 분포 함수와 확률 밀도 함수 그리기



47

12 상자 수염 그리기

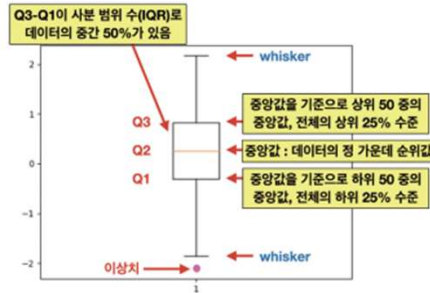
- **상자 수염 그림** box-and-whisker plot은 수치 데이터를 표현하는 그래프의 한 종류이다. 이 그림은 흔히 **상자 차트** box plot라고도 한다.
- 이 그래프는 **최대, 최소, 중간값과 사분위 수** 등의 통계량을 하나의 상자 그림으로 **가시화**할 수 있는 방법이다.

▶ `# 넘파이의 난수 모듈에 있는 정규 분포에 따르는 난수 100개를 생성`
`rand_data = np.random.randn(100)`
`plt.boxplot(rand_data)`

48

12 상자 수염 그리기

- 이 그림에서 나타난 상자 모양 직사각형은 주황색으로 나타난 **중앙값** median Q2를 중심으로 상위 25%(Q3로 나타남)와 하위 25%(Q1으로 나타남)의 값이 모여 있는 구간을 표시한다.
- 이 그림에서 상자의 상단인 Q3에서 상자의 하단인 Q1의 값을 뺀 Q3-Q1을 **사분 범위** 혹은 **IQR** inter-quartile range이라고 한다.
- 그리고 상자 아래위로 직선이 그어져 있고 이를 위와 아래에서 막는 선이 있는데 이것을 **위스커** whisker 혹은 수염이라고 한다.



49

12 상자 수염 그리기

- 상단 수염과 하단 수염의 범위는 다음과 같다.

- 상단 수염 : $Q3 + 1.5 * IQR$ 값보다 작은 데이터 중 가장 큰 값
 - 하단 수염 : $Q1 - 1.5 * IQR$ 값보다 큰 데이터 중 가장 작은 값

- 이 데이터 분포의 아래, 위 1.5배 이내의 자료는 수염으로 표시하고 수염의 끝에 막대를 그어 **이상치** outlier 자료 값의 경계를 표시하는 데 유용하게 사용할 수 있다.
- 즉 아래쪽 위스커에서 위쪽 위스커까지의 범위가 대부분의 데이터가 분포하는 범위라고 볼 수도 있다.

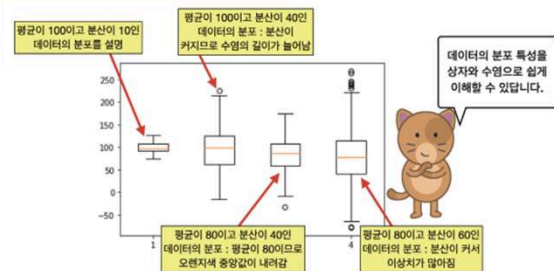
50

12 상자 수염 그리기

- 이제 다음과 같이 네 종류의 데이터가 준비되어 있다고 할 때, 이들을 각각 상자 차트로 그려 보도록 하자.

```
np.random.seed(85)
data1 = np.random.normal(100, 10, 200) # 평균이 100이고 분산이 10
data2 = np.random.normal(100, 40, 200) # 평균이 100이고 분산이 40
data3 = np.random.normal(80, 40, 200) # 평균이 80이고 분산이 40
data4 = np.random.normal(80, 60, 200) # 평균이 80이고 분산이 60

plt.boxplot([data1, data2, data3, data4])
```

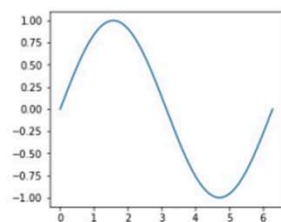


51

13 그래프의 크기와 그리드 그리기

- matplotlib**에는 화면에 나타날 그래프의 크기를 조절하는 기능도 있는데 **plt.figure()** 함수의 키워드 인자로 **figsize**를 줄 수 있다.

```
X = np.linspace(0, 2*np.pi, 200)
Y = np.sin(X)
plt.figure(figsize=(4.2, 3.6))
plt.plot(X, Y)
```

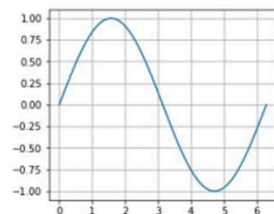


52

13 그래프의 크기와 그리드 그리기

- **matplotlib**의 **pyplot** 모듈에 있는 **grid()** 함수는 다음과 같이 그래프의 내부에 격자를 그려서 그래프의 모양을 좀 더 정확하게 이해하는 데 도움을 준다.

```
X = np.linspace(0, 2*np.pi, 200)
Y = np.sin(X)
plt.figure(figsize=(4.2, 3.6))
plt.plot(X, Y)
plt.grid()
```

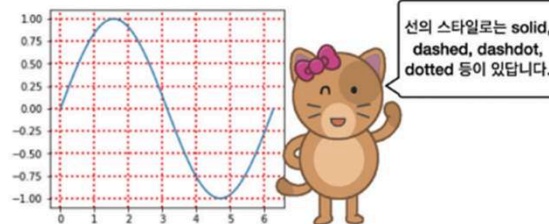


53

13 그래프의 크기와 그리드 그리기

- 이 코드에서 **grid()** 함수의 내부에 키워드 인자로 **color**, **linestyle**, **linewidth** 등을 사용하여 격자의 색상, 선의 형태, 선의 굵기를 변경할 수도 있다.

```
# plt.grid() 대신 다음과 같은 키워드 인자를 지정하여 호출
plt.grid(color='r', linestyle='dotted', linewidth=2)
```

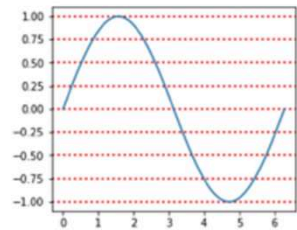


선의 스타일은 solid, dashed, dashdot, dotted 등이 있습니다.

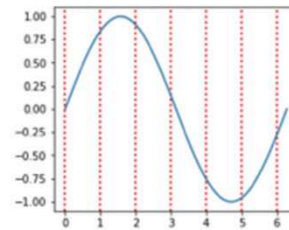
54

13 그래프의 크기와 그리드 그리기

- 이 밖에도 `axis='y'`나 `axis='x'` 인자를 사용할 경우 수평선, 수직선 격자 선분을 그릴 수도 있다.



`axis='y'` 인자를 사용한 경우



`axis='x'` 인자를 사용한 경우

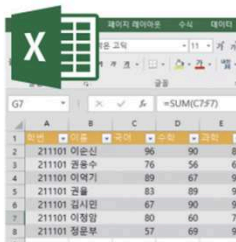
판다스



1

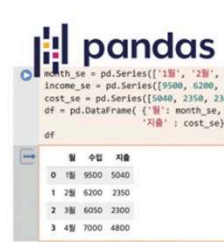
1 엑셀보다 빠르고 강력한 판다스

- 아래의 프로그램은 현재 업무용 소프트웨어로 가장 인기가 있는 마이크로소프트사의 **엑셀Excel**이라는 프로그램이다.
- 데이터 과학자는 전통적으로 이와 같은 테이블 형태의 데이터를 선호한다. 2차원 **행렬matrix** 형태의 데이터는 개발자가 편리하게 각 요소나 행, 열에 접근할 수 있기 때문이다.
- 파이썬의 **판다스pandas** 패키지를 사용하면 이러한 문제를 해결할 수 있다.



엑셀은 행과 열로 이루어진 데이터 처리에 탁월한 성능을 보여 줍니다.

vs



판다스는 엑셀같은 스프레드시트 프로그램보다 연산을 빠르고 효율적으로 할 수 있어요.

2

1 엑셀보다 빠르고 강력한 판다스

- 판다스는 넘파이를 기반으로 하기 때문에 처리속도가 빠르고, 행과 열로 잘 구조화된 데이터프레임을 제공하며, 이 데이터프레임을 조작할 수 있는 다양한 함수를 지원한다.

판다스의 주요 특징

- 빠르고 효율적이며 다양한 표현력을 갖춘 자료 구조.
실세계 데이터 분석을 위해 만들어진 파이썬 패키지로 강력하면서도 다양한 표현력을 가지고 있다.
 - 다양한 형태의 데이터에 적합
이중 자료형의 열을 가진 테이블 데이터
시계열 데이터
레이블을 가진 다양한 행렬 데이터
다양한 관측 통계 데이터
 - 핵심 구조
시리즈Series : 1차원 구조를 가진 하나의 열
데이터프레임DataFrame : 복수의 열을 가진 2차원 데이터
 - 판다스가 잘하는 일
결측 데이터 처리와 필터링
데이터 추가와 삭제
열 데이터의 정렬과 다양한 데이터 조작
파이썬 리스트, 딕셔너리, 넘파이 배열을 데이터프레임으로 손쉽게 변환
데이터프레임의 각 필드들 사이의 상관 관계를 구하는 일
각 데이터프레임 열에서 결측값의 개수와 정상값의 수를 계산
matplotlib과 잘 결합되어 손쉽게 데이터를 시각화
한 데이터프레임을 특정한 기준에 따라 여러 개의 데이터프레임으로 분할
두 데이터프레임을 결합하고 다시 인덱싱하는 능력
- 판다스는 여기에 열거한 기능 이외에도 데이터 처리와 분석을 위한 다양하면서도 강력한 기능을 제공하고 있다.

3

2 데이터 교환을 위한 csv 파일형식

- 앞 장에서 살펴본 넘파이의 2차원 행렬 데이터는 모두 같은 자료값을 가지는 단순한 수치 정보 중심으로만 구성되어 있다는 한계가 있다.
- 이 한계는 판다스pandas 패키지를 사용하여 극복할 수 있다.
- 넘파이의 다차원 배열과 같이 판다스는 시리즈와 데이터프레임이라는 두 가지의 기본적인고도 다재다능한 데이터 구조를 제공한다.

4

2 데이터 교환을 위한 csv 파일형식

- 판다스의 **시리즈series**는 같은 자료형의 데이터를 저장하는 인덱싱된 1차원 배열인데, 시리즈 데이터를 만드는 것은 Series 클래스를 이용한다.

```
import numpy as np
import pandas as pd

se = pd.Series([1, 2, np.nan, 4]) # np.nan은 결측값
se
```

0	1.0
1	2.0
2	NaN
3	4.0

dtype: float64

5

2 데이터 교환을 위한 csv 파일형식

- 데이터 과학을 위한 데이터를 다루게 될 경우, 잘 정제된 좋은 데이터보다는 정제되지 않은 데이터나 결측 데이터를 많이 다루게 된다.
- 이것을 **결손값missing value** 혹은 결측값이라 하는데 판다스는 이것을 탐지하고 수정하는 함수를 제공한다.
- 데이터에 **결손값**이 있는지를 **불리언boolean** 값으로 반환하는 함수는 isna()이다.

```
se.isna() # np.nan이 포함된 세 번째 항목만 True를 반환
```

0	False
1	False
2	True
3	False

dtype: bool

6

2 데이터 교환을 위한 csv 파일형식

- 시리즈 자료구조 se의 첫 번째 원소와 두 번째 원소를 추출하기 위해서는 다음과 같은 인덱싱 기법을 사용한다. 이 방법은 리스트 인덱싱과 동일한 문법을 사용한다.

```
se[0], se[1]
(1.0, 2.0)
```

7

2 데이터 교환을 위한 csv 파일형식

- 앞서 살펴본 시리즈 se는 0, 1, 2, 3과 같은 인덱스를 가진 자료값이었는데 이 인덱스는 다음과 같은 방법을 사용하여 'a', 'b', 'c', 'd'와 같은 알파벳 문자로 변경할 수도 있다.

```
data = [1, 2, np.nan, 4]
indexed_se = pd.Series(data, index = ['a', 'b', 'c', 'd'])
print(indexed_se)

a    1.0
b    2.0
c    NaN
d    4.0
dtype: float64

indexed_se['a'], indexed_se['b'] # 인덱스가 'a', 'b'... 임
(1.0, 2.0)
```

```
data = [1, 2, np.nan, 4]
pd.Series(data, index = ['a', 'b', 'c', 'd'])
```

	data
a	1.0
b	2.0
c	NaN
d	4.0

인덱스를 가진 시리즈의 정의 방법과 그 구조

8

3 판다스의 기본 구조인 시리즈와 데이터프레임

- 판다스에서는 각 월의 수익을 아래와 같이 딕셔너리 구조로 정의한 후, 이 딕셔너리를 이용하여 income_se 시리즈를 생성하는 코드를 만들 수 있다.

```
income = {'1월' : 9500, '2월': 6200, '3월': 6050, '4월': 7000}
income_se = pd.Series(income) # 월을 인덱스로 하는 매출 시리즈
print('동윤이네 상점의 수익')
print(income_se)
```

동윤이네 상점의 수익	
1월	9500
2월	6200
3월	6050
4월	7000

dtype: int64

9

3 판다스의 기본 구조인 시리즈와 데이터프레임

- 앞서 살펴본 동윤이네 상점의 수익 정보와 함께 **지출 정보**와 같은 새로운 정보가 더 필요한 경우를 생각해보자.
- 만일 이 상점에 매달 일정한 물건 구입 비용이 발생한 경우, 수익과 지출을 모두 표기하기 위해서는 **1차원의 시리즈 자료구조로는 표현이 어렵다**.
- 따라서 다음과 같이 expenses_se 시리즈 자료를 만들어서 새롭게 데이터를 만들어야 한다. 이때 사용되는 2차원 자료구조가 바로 **데이터프레임 dataframe**이다.

10

3 판다스의 기본 구조인 시리즈와 데이터프레임

- 여기서 df를 이용하여 출력할 경우 주피터 노트북에서는 아래와 같이 표를 만들어 주며, print(df)를 사용할 경우 텍스트 형식의 출력을 하게 된다.

```

▶ month_se = pd.Series(['1월', '2월', '3월', '4월'])
  income_se = pd.Series([9500, 6200, 6050, 7000])
  expenses_se = pd.Series([5040, 2350, 2300, 4800])
  df = pd.DataFrame( {'월': month_se, '수익': income_se,
                      '지출' : expenses_se})

df    # print(df)를 이용해서 출력한 후 출력 형식을 비교해 보자

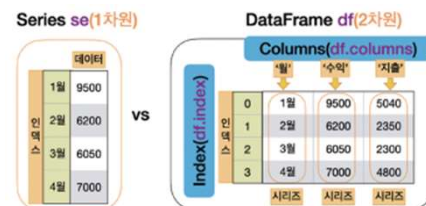
```

	월	수익	지출
0	1월	9500	5040
1	2월	6200	2350
2	3월	6050	2300
3	4월	7000	4800

11

3 판다스의 기본 구조인 시리즈와 데이터프레임

- 다음은 1차원 데이터인 시리즈와 2차원 데이터인 데이터프레임을 비교하는 것으로 왼쪽의 시리즈는 2차원처럼 보이지만 1월, 2월, 3월, 4월이 인덱스이므로 실제로는 **1차원 데이터**이다.



- 다음으로 income_se 시리즈의 최대값과 평균값을 출력해 보자.

```

▶ # 판다스 Series를 이용하여 최대 수익 월을 출력하기
  m_idx = np.argmax(income_se) # 넘파이의 argmax() 사용
  print('최대 수익이 발생한 월:', month_se[m_idx])

최대 수익이 발생한 월: 1월

▶ print('월 최대 수익:', income_se.max(), \
      ', 월 평균 수익:', income_se.mean())

월 최대 수익: 9500 , 월 평균 수익: 7187.5

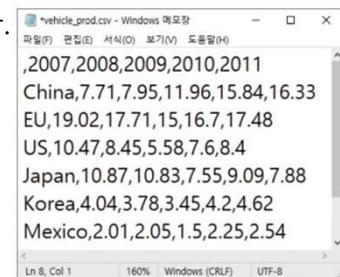
```

12

4 csv 데이터를 읽고 확인하기

- CSV는 쉼표로 구분한 변수 comma separated variables의 약자이다.
- 탭 tab 문자를 구분자로 사용하는 파일을 TSV라고 한다.

- 데이터의 중간에 쉼표(,)가 포함되어 있다. 이러한 경우에는 구분자로 사용되는 쉼표와 구분하기 위하여 반드시 데이터를 따옴표로 감싸야 한다.



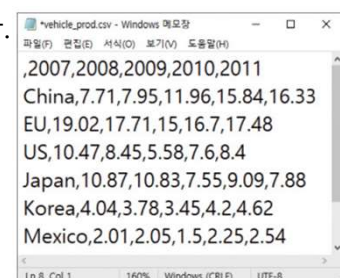
- 앞서 살펴본 넘파이는 2차원 행렬 형태의 데이터를 지원하지만, 데이터의 속성을 표시하는 행이나 열의 레이블을 가지고 있지 않다. 따라서, 데이터의 속성을 한눈에 이해하기에는 불편하다.

13

4 csv 데이터를 읽고 확인하기

- CSV는 쉼표로 구분한 변수 comma separated variables의 약자이다.
- 탭 tab 문자를 구분자로 사용하는 파일을 TSV라고 한다.

- 데이터의 중간에 쉼표(,)가 포함되어 있다. 이러한 경우에는 구분자로 사용되는 쉼표와 구분하기 위하여 반드시 데이터를 따옴표로 감싸야 한다.



- 앞서 살펴본 넘파이는 2차원 행렬 형태의 데이터를 지원하지만, 데이터의 속성을 표시하는 행이나 열의 레이블을 가지고 있지 않다. 따라서, 데이터의 속성을 한눈에 이해하기에는 불편하다.

14

4 csv 데이터를 읽고 확인하기

- 다음의 read_csv() 함수는 웹 상에 있는 파일도 바로 읽을 수 있다. 이 책의 코드는 다음과 같은 github.com 사이트의 저장소에서 다운받을 수 있으며 이 주소를 입력값으로 두고 사용할 수 있다.

```
path = 'https://github.com/dongupak/DataML/raw/main/csv/'
file = path+'vehicle_prod.csv'
df = pd.read_csv(file) # 원격지에 접속하여 csv를 읽어옴
```

- 자신의 PC의 'D:\data' 경로에 vehicle_prod.csv 파일이 있을 경우에는 다음과 같은 경로 정보를 이용하여 파일을 읽어올 수도 있다.

```
df = pd.read_csv('D:\data\vehicle_prod.csv') # 현재 컴퓨터에 있을 경우
```

15

4 csv 데이터를 읽고 확인하기

- 이 파일에는 다음과 같이 국가에 대한 정보와 연도별 생산 대수가 저장되어 있는데, 첫 번째 열에는 인덱스, 두 번째는 국가명, 세 번째에서 다섯 번째 열은 2007년부터 2011년도 의 생산 대수가 백만 단위로 저장되어 있다.

```
print(df)
```

Unnamed: 0		2007	2008	2009	2010	2011
0	China	7.71	7.95	11.96	15.84	16.33
1	EU	19.02	17.71	15.00	16.70	17.48
2	US	10.47	8.45	5.58	7.60	8.40
3	Japan	10.87	10.83	7.55	9.09	7.88
4	Korea	4.04	3.78	3.45	4.20	4.62
5	Mexico	2.01	2.05	1.50	2.25	2.54

- 각 열들은 서로 다른 속성(property)을 나타낸다.

16

5 데이터프레임의 구조

- 데이터프레임에서는 다음과 같이 **인덱스index**와 **컬럼스columns** 객체를 정의하여 사용한다.
- 판다스의 read_csv() 함수에 index_col이라는 키워드 매개변수에 인자 0을 넘겨주면 **첫 번째 열이 인덱스로** 사용된다.

csv 파일의 첫 행으로 만들어진 column

	Unnamed: 0	2007	2008	2009	2010	2011
0	China	7.71	7.95	11.96	15.84	16.33
1	EU	19.02	17.71	15	16.7	17.48
2	US	10.47	8.45	5.58	7.6	8.4
3	Japan	10.87	10.83	7.55	9.09	7.88
4	Korea	4.04	3.78	3.45	4.2	4.62
5	Mexico	2.01	2.05	1.5	2.25	2.54

자동으로 생성된 index

```
df = pd.read_csv(file, index_col = 0)
print(df)
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

17

5 데이터프레임의 구조

- 이 데이터프레임의 컬럼값과 인덱스값을 출력하기 위해서는 다음과 같이 columns, index 속성을 사용하면 된다.

```
df.columns # 데이터프레임의 컬럼값들을 살펴보자
```

```
Index(['2007', '2008', '2009', '2010', '2011'], dtype='object')
```

```
df.index # 데이터프레임의 인덱스값들을 살펴보자
```

```
Index(['China', 'EU', 'US', 'Japan', 'Korea', 'Mexico'], dtype='object')
```

18

5 데이터프레임의 구조

- 데이터프레임에서 특정한 컬럼의 내용을 살펴보고자 할 경우 `df['2007']`과 같은 특정한 컬럼값을 인덱스로 사용하면 된다.

	<code>df['2007']</code> # 데이터프레임의 2007년도 컬럼값들을 살펴보자												
	<table><tr><td>China</td><td>7.71</td></tr><tr><td>EU</td><td>19.02</td></tr><tr><td>US</td><td>10.47</td></tr><tr><td>Japan</td><td>10.87</td></tr><tr><td>Korea</td><td>4.04</td></tr><tr><td>Mexico</td><td>2.01</td></tr></table> Name: 2007, dtype: float64	China	7.71	EU	19.02	US	10.47	Japan	10.87	Korea	4.04	Mexico	2.01
China	7.71												
EU	19.02												
US	10.47												
Japan	10.87												
Korea	4.04												
Mexico	2.01												
	<code>df.columns.tolist()</code> # 데이터프레임의 컬럼들을 리스트형으로 반환함												
	<code>['2007', '2008', '2009', '2010', '2011']</code>												
	<code>df['2007'].tolist()</code> # 2007년도 컬럼들의 값들을 리스트형으로 반환함												
	<code>[7.71, 19.02, 10.47, 10.87, 4.04, 2.01]</code>												

19

6 새로운 열을 생성하자

- 이 표에서 모든 레코드의 값을 합하는 `df['2007'] + df['2008'] + df['2009'] + df['2010'] + df['2011']`와 같은 표현은 `df.sum(axis = 1)`과 같은 표현과 동일하다. 여기서 `axis = 1`은 행 방향을 가리키며, `axis = 0`은 열 방향을 가리킨다.



```
df['total'] = df.sum(axis = 1)
print(df)
```

	2007	2008	2009	2010	2011	total
China	7.71	7.95	11.96	15.84	16.33	59.79
EU	19.02	17.71	15.00	16.70	17.48	85.91
US	10.47	8.45	5.58	7.60	8.40	40.50
Japan	10.87	10.83	7.55	9.09	7.88	46.22
Korea	4.04	3.78	3.45	4.20	4.62	20.09
Mexico	2.01	2.05	1.50	2.25	2.54	10.35

20

6 새로운 열을 생성하자

- 이 예제처럼 다음과 같은 방법으로 특정한 열을 인덱싱 한 후, `sum(axis=1)`, `mean(axis=1)`과 같이 메소드의 이름과 연산이 수행되는 축을 명시하여 5년간 자동차 생산 대수의 합과 평균을 구할 수도 있다.

```
df['total'] = df[['2007', '2008', '2009', '2010', '2011']].sum(axis=1)
df['mean'] = df[['2007', '2008', '2009', '2010', '2011']].mean(axis=1)
print(df)
```

	2007	2008	2009	2010	2011	total	mean
China	7.71	7.95	11.96	15.84	16.33	59.79	11.958
EU	19.02	17.71	15.00	16.70	17.48	85.91	17.182
US	10.47	8.45	5.58	7.60	8.40	40.50	8.100
Japan	10.87	10.83	7.55	9.09	7.88	46.22	9.244
Korea	4.04	3.78	3.45	4.20	4.62	20.09	4.018
Mexico	2.01	2.05	1.50	2.25	2.54	10.35	2.070

21

6 새로운 열을 생성하자

```
df.drop('2007', inplace=True, axis=1)
print(df) # 이전에 구한 total, mean 값이 나타남
```

	2008	2009	2010	2011	total	mean
China	7.95	11.96	15.84	16.33	59.79	11.958
EU	17.71	15.00	16.70	17.48	85.91	17.182
US	8.45	5.58	7.60	8.40	40.50	8.100
Japan	10.83	7.55	9.09	7.88	46.22	9.244
Korea	3.78	3.45	4.20	4.62	20.09	4.018
Mexico	2.05	1.50	2.25	2.54	10.35	2.070

```
# 2008년부터 2011년도까지의 총생산 대수와 평균을 다시 계산함
df['total'] = df[['2008', '2009', '2010', '2011']].sum(axis=1)
df['mean'] = df[['2008', '2009', '2010', '2011']].mean(axis=1)
print(df) # 새로 계산한 total, mean 값이 나타남
```

	2008	2009	2010	2011	total	mean
China	7.95	11.96	15.84	16.33	52.08	13.0200
EU	17.71	15.00	16.70	17.48	66.89	16.7225
US	8.45	5.58	7.60	8.40	30.03	7.5075
Japan	10.83	7.55	9.09	7.88	35.35	8.8375
Korea	3.78	3.45	4.20	4.62	16.05	4.0125
Mexico	2.05	1.50	2.25	2.54	8.34	2.0850

22

7 inplace로 데이터프레임 갱신하기

- 이 옵션이 True이면 명령어를 실행한 후 메소드가 적용된 기존의 데이터프레임을 갱신하게 되며, False이면 기존 데이터프레임은 그대로 두고 갱신된 데이터프레임을 반환하게 된다.

```
d_df = pd.DataFrame(data = [[10, 20, 30, 40], [50, 60, 70, 80]],
                      columns = ['A', 'B', 'C', 'D'])
new_df = d_df.drop('B', axis=1, inplace=False)
print(new_df)      # 'B'열이 삭제된 새로운 데이터프레임
```

	A	C	D
0	10	30	40
1	50	70	80

```
print(d_df)      # d_df 데이터프레임은 변화가 없음
```

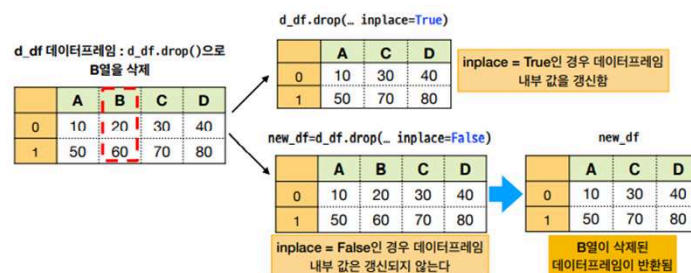
	A	B	C	D
0	10	20	30	40
1	50	60	70	80

```
d_df.drop('B', axis=1, inplace=True) # inplace 키워드 인자가 True
print(d_df)      # d_df 데이터프레임 내부의 값이 변경됨
```

	A	C	D
0	10	30	40
1	50	70	80

23

7 inplace로 데이터프레임 갱신하기



- inplace가 True일 경우 d_df 데이터프레임 자체의 값이 변경되므로 'B' 열이 완전히 없어지는 것을 볼 수 있다.

24

7 inplace로 데이터프레임 갱신하기

- inplace=True로 하여 df 데이터프레임을 갱신하는 방식을 사용해 보자.
- 이를 위해서 axis = 0으로 축을 명시해야 한다는 것에 유의하도록 하자.

```
path = 'https://github.com/dongupak/DataML/raw/main/csv/'
file = path+'vehicle_prod.csv'
df = pd.read_csv(file, index_col=0) # 원격지에 접속하여 csv를 읽어옴
print(df)
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

```
df.drop('Mexico', axis=0, inplace=True) # Mexico행을 삭제하고 df를 갱신
print(df)
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62

25

8 데이터프레임 시각화

- 두 데이터프레임에서 2007년도 레이블을 가진 열만을 추출하기 위해서는 원하는 컬럼의 레이블 ['2007']을 사용하면 된다.

```
df = pd.read_csv(file, index_col = 0)
df_no_index = pd.read_csv(file)
print(df['2007'])
print(df_no_index['2007'])
```

	2007
China	7.71
EU	19.02
US	10.47
Japan	10.87
Korea	4.04
Mexico	2.01

```
Name: 2007, dtype: float64
```

	2007
0	7.71
1	19.02
2	10.47
3	10.87
4	4.04
5	2.01

```
Name: 2007, dtype: float64
```

26

8 데이터프레임 시각화

- 주의해서 볼 점은 index_col = 0과 같이 첫 열을 인덱스로 지정한 경우 국가 코드가 인덱스로 사용되기 때문에 국가 코드가 표시되고 있다는 점이며, 별도의 index_col을 지정하지 않을 경우 0부터 5까지의 인덱스를 자동으로 생성하여 인덱싱을 한다는 점이다.

```
print(df[['2007', '2008', '2009']])
```

	2007	2008	2009
China	7.71	7.95	11.96
EU	19.02	17.71	15.00
US	10.47	8.45	5.58
Japan	10.87	10.83	7.55
Korea	4.04	3.78	3.45
Mexico	2.01	2.05	1.50

27

8 데이터프레임 시각화

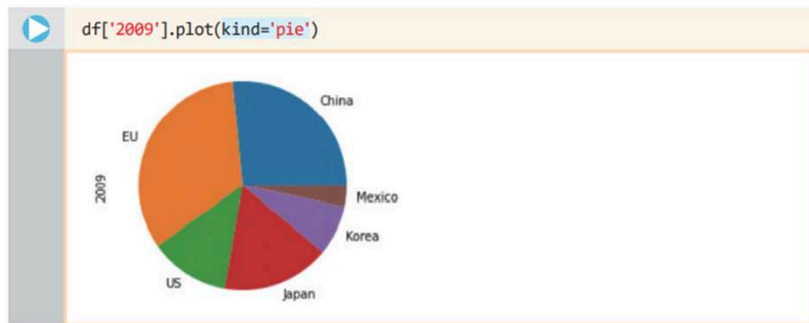
- 우리는 다음과 같은 방법으로 선택된 열을 쉽게 그래프로 그릴 수 있다. 이를 위하여 데이터프레임의 df['2009']을 통해서 2009년도 열을 추출하고 plot() 메소드만 추가하면 된다.



28

8 데이터프레임 시각화

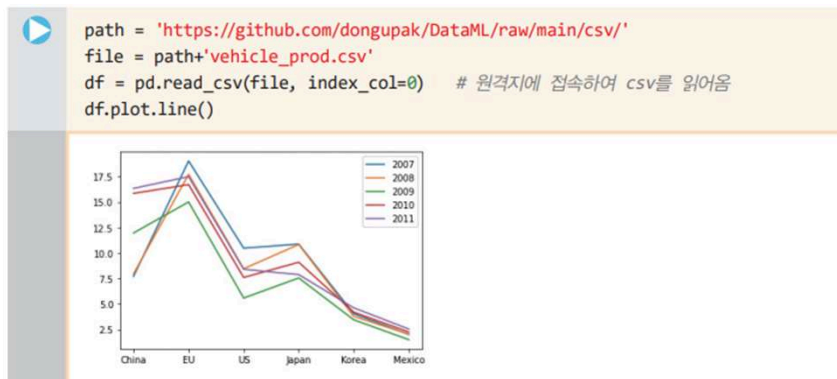
- 코드에서 plot 메소드의 키워드 매개변수 kind는 차트의 종류를 의미한다.
- 이제 다음과 같이 kind='pie'로 지정하고, color 키워드 인자 부분을 삭제하면 아래 그림처럼 파이 차트가 그려진다.



29

9 편리하고 강력한 시각화

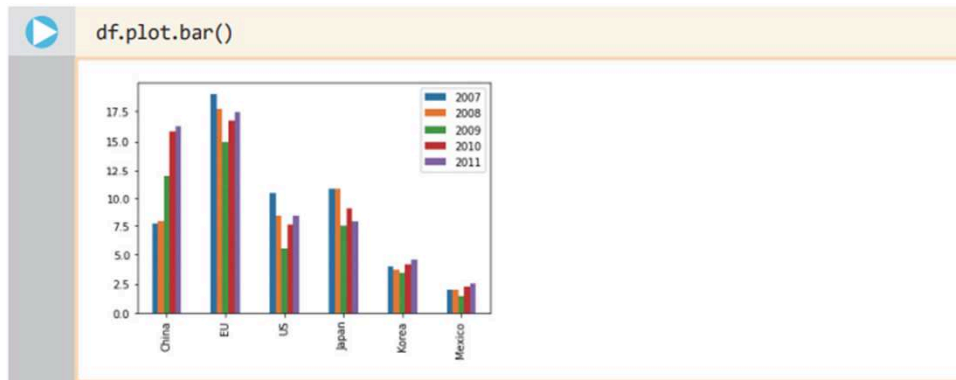
- 데이터프레임은 맷플롯립과 결합되어 있어서 자료값을 손쉽게 시각화하여 보여줄 수 있다.



30

9 편리하고 강력한 시각화

- 선 그래프가 아닌 막대 그래프로 보고자 할 경우 `df.plot.bar()`를 호출하면 된다.



31

9 편리하고 강력한 시각화

- 이 선 그래프를 통해서 중국의 자동차 생산량이 매년 가파르게 증가하고 있으며, 일본의 경우 완만하게 감소하는 것을 이해할 수 있을 것이다.



32

10 편리한 데이터 다루기 – 슬라이싱과 인덱싱

- 우선 처음 5행만 얻으려면 head()를 사용할 수 있다. 그리고, 마지막 5행만을 얻으려면 tail()을 사용한다. 만일 head()와 tail()에 정수를 인자로 넘겨주면 그 정수만큼의 행을 보여준다.

```
df = pd.read_csv(file, index_col=0) # 원격지에 접속하여 csv를 읽어옴
df.head(3)                         # 첫 3행의 정보를 가져온다
```

	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15.00	16.70	17.48
US	10.47	8.45	5.58	7.60	8.40

33

10 편리한 데이터 다루기 – 슬라이싱과 인덱싱

- 여기서는 행의 개수가 3개뿐이어서 특별한 장점이 없어 보이지만, 행의 개수가 아주 많은 경우에는 head()와 tail()이 도움이 된다. 더욱 일반적인 방법은 다음과 같이 슬라이싱 표기법을 사용하는 것이다.

```
df[2:6] # 지정된 범위의 레코드를 가져온다
```

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88
Korea	4.04	3.78	3.45	4.20	4.62
Mexico	2.01	2.05	1.50	2.25	2.54

34

10 편리한 데이터 다루기 – 슬라이싱과 인덱싱

- 만약 'Korea' 행을 선택하려면 `df.loc['Korea']`와 같이 큰 괄호를 사용하여 참조하려는 인덱스 이름을 적어주면 된다.



A Jupyter Notebook cell showing the command `df.loc['Korea']` and its output. The output is a Series with years as the index and values as the data. The values are 4.04 for 2007, 3.78 for 2008, 3.45 for 2009, 4.20 for 2010, and 4.62 for 2011. The dtype is float64.

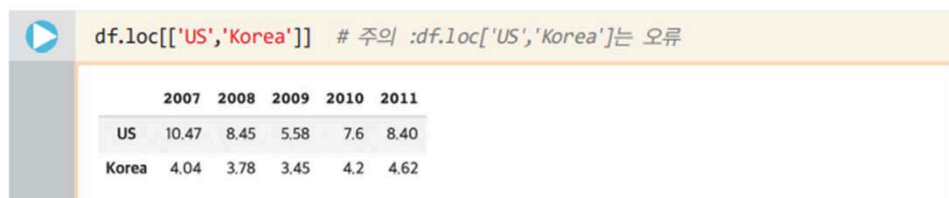
	2007	2008	2009	2010	2011
Korea	4.04	3.78	3.45	4.20	4.62

Name: Korea, dtype: float64

35

10 편리한 데이터 다루기 – 슬라이싱과 인덱싱

- 그리고, 'US','Korea' 인덱스를 사용할 경우에는 다음과 같이 `[['..']]` 내부에 인덱스의 이름을 적어주어야 하며, `df.loc['US','Korea']`와 접근할 경우 키 오류가 발생하니 주의하도록 하자.



A Jupyter Notebook cell showing the command `df.loc[['US','Korea']]` and its output. The output is a DataFrame with years as the index and values as the data. The values are 10.47 for US in 2007, 8.45 for US in 2008, 5.58 for US in 2009, 7.6 for US in 2010, 8.40 for US in 2011, 4.04 for Korea in 2007, 3.78 for Korea in 2008, 3.45 for Korea in 2009, 4.2 for Korea in 2010, and 4.62 for Korea in 2011. The dtype is float64.

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.6	8.40
Korea	4.04	3.78	3.45	4.2	4.62

주의 :df.loc['US','Korea']는 오류

36

10 편리한 데이터 다루기 – 슬라이싱과 인덱싱

- 다음으로는 열 선택과 행 선택을 결합하는 방법을 알아보자.

```
df['2011'][[0, 4]]
```

China	16.33
Korea	4.62

Name: 2011, dtype: float64

- 보다 상세하게 데이터프레임에서 특정한 요소 하나만을 선택하려면 간단하게 loc() 함수에 행 과 열의 레이블을 써주면 된다.

```
df.loc['Korea', '2011']
```

4.62

37

11 loc, iloc 인덱서

1. head(), tail() 메소드 : 첫, 마지막 항목 중에서 지정된 개수를 가져올 수 있다.
2. [m:n]을 사용 : 지정된 구간 m, n의 항목을 가져올 수 있다.
3. loc[] : 인덱스에서 특정 레이블이 있는 행을 가져온다.
4. iloc[] : 인덱스에서 정수형 인덱스를 사용하여 특정 위치에 있는 행을 가져온다.

38

11 loc, iloc 인덱서

- 여기서는 전체 데이터 중에서 가장 앞에 있는 3개를 가져와서, 2009년도 열의 값을 추출하였다.

Unnamed: 0	2007	2008	2009	2010	2011
China	7.71	7.95	11.96	15.84	16.33
EU	19.02	17.71	15	16.7	17.48
US	10.47	8.45	5.58	7.6	8.4
Japan	10.87				7.88
Korea	4.04	3.78	3.45	4.2	4.62
Mexico	2.01	2.05	1.5	2.25	2.54

- 다음과 같이 `df.iloc[4]`을 사용할 경우 Korea를 인덱스로 하는 항목을 다음과 같은 열 형식으로 출력해 볼 수 있다.

```
df.iloc[4] # df.loc['Korea'] 와 동일함, df.iloc['Korea']는 오류
```

2007	4.04
2008	3.78
2009	3.45
2010	4.20
2011	4.62

Name: Korea, dtype: float64

39

11 loc, iloc 인덱서

- 데이터프레임의 행 데이터를 가져오기 위해서는 입력된 문서에 대하여 색인을 자동으로 작성해 주는 프로그램이 필요한데 이를 **인덱서** `indexer`라고 한다

```
df = pd.read_csv(file, index_col=0) # 원격지에 접속하여 csv를 읽어옴
df.head(3)['2009'] # 첫 3행의 정보를 가져온다, 2009년도 값만을 가져온다
```

China	11.96
EU	15.00
US	5.58

Name: 2009, dtype: float64

40

11 loc, iloc 인덱서

- `iloc[]`는 정수형 인덱스만을 사용할 수 있기 때문에 `iloc['Korea']`를 사용할 경우 오류가 발생한다.



```
df.iloc[[2, 4]] # 주의: df.iloc[2, 4]는 US행의 2011년 데이터임
```

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.6	8.40
Korea	4.04	3.78	3.45	4.2	4.62



```
df.iloc[2, 4] # US행의 2011년 데이터를 출력
```

8.4

41

11 loc, iloc 인덱서

▶ `df.iloc[2:4]` # US행, Japan행의 데이터를 출력

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88

▶ `df.iloc[[2, 3]]` # US행, Japan행의 데이터를 출력

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88

▶ `df[2:4]` # US행, Japan행의 데이터를 출력

	2007	2008	2009	2010	2011
US	10.47	8.45	5.58	7.60	8.40
Japan	10.87	10.83	7.55	9.09	7.88

42

12 판다스를 이용한 기상 데이터 분석

- 데이터프레임이 저장한 데이터를 간단히 분석하려면 describe() 함수를 사용할 수 있다.
- 우선 다음과 같은 코드로 데이터를 읽어 보자. 이 파일에는 한글이 포함되어 있으므로 encoding='CP949'를 통해 한글 인코딩 방식을 알려주도록 하자.

```
path = 'https://github.com/dongupak/DataML/raw/main/csv/'
weather_file = path + 'weather.csv'

weather = pd.read_csv(weather_file, index_col = 0, encoding='CP949')
print(weather.head(3))
print('weather 데이터의 shape :', weather.shape)
```

	평균기온	최대풍속	평균풍속
일시			
2010-08-01	28.7	8.3	3.4
2010-08-02	25.2	8.7	3.8
2010-08-03	22.1	6.3	2.9

weather 데이터의 shape : (3653, 3)

43

12 판다스를 이용한 기상 데이터 분석

- 이제 데이터가 담고 있는 내용을 간략히 분석하기 위해 describe() 함수를 호출해 보자.

```
print(weather.describe())
```

	평균기온	최대풍속	평균풍속
count	3653.000000	3649.000000	3647.000000
mean	12.942102	7.911099	3.936441
std	8.538507	3.029862	1.888473
min	-9.000000	2.000000	0.200000
25%	5.400000	5.700000	2.500000
50%	13.800000	7.600000	3.600000
75%	20.100000	9.700000	5.000000
max	31.300000	26.000000	14.900000

44

12 판다스를 이용한 기상 데이터 분석

- describe() 메소드를 통해서 각 속성값들의 **개수**count, **평균값**mean, **표준편차**std, **최소값**min, **최대값**max 등을 쉽게 알 수 있다.
- 만약 문자열을 담고 있는 열이 있다면 해당 열은 분석에서 제외된다.
- 이 기능은 mean(), std() 함수를 통해 특정한 분석값만 볼 수도 있다.

```
print('평균 분석 -----')
print(weather.mean())
print('표준편차 분석 -----')
print(weather.std())
```

평균 분석	-----
평균기온	12.942102
최대풍속	7.911099
평균풍속	3.936441
dtype:	float64
표준편차 분석	-----
평균기온	8.538507
최대풍속	3.029862
평균풍속	1.888473
dtype:	float64

45

13 데이터 정제와 결손값의 처리

- 데이터를 처리하기 전에 반드시 거쳐야 하는 절차가 데이터 정제이다.
- 앞서 살펴본 shape 속성을 통해 전체 테이블의 크기를 살펴 볼 수 있지만, 개별적인 열의 개수를 알고 싶다면 count() 메소드를 사용할 수 있다.

```
weather.count() # 개별적인 열의 개수를 알 수 있다
```

평균기온	3653
최대풍속	3649
평균풍속	3647
dtype:	int64

46

13 데이터 정제와 결손값의 처리

- 어떤 이유로 인해 날씨 데이터를 수집하는 중에 값이 입력되지 못하면 그 항목이 비어 있을 수 있는데 이를 **결손값** missing data이라고 한다.
- 결손값**을 탐지하는 함수를 이미 살펴보았다. 데이터에 결손값이 있는지를 **불리언** boolean 값으로 반환하는 메소드는 `isna()`이다.

```
missing_data = weather[ weather['평균풍속'].isna() ]
print(missing_data)
```

	평균기온	최대풍속	평균풍속
일시			
2012-02-11	-0.7	NaN	NaN
2012-02-12	0.4	NaN	NaN
2012-02-13	4.0	NaN	NaN
2015-03-22	10.1	11.6	NaN
2015-04-01	7.3	12.1	NaN
2019-04-18	15.7	11.7	NaN

47

13 데이터 정제와 결손값의 처리

- 결손값을 다루는 가장 간단한 방법은 결손값을 가진 행이나 열 전체를 삭제하는 것이다.

축이 0이면 결손 데이터를 포함한 행을 삭제하고
축이 1이면 결손 데이터를 포함한 열을 삭제한다.

`inplace`가 `True`이면 원본 데이터에서 결손 데이터를 삭제하고
`False`인 경우는 원본은 그대로 두고 고쳐진 데이터프레임 반환

`pandas.DataFrame.dropna(axis=0, how='any', inplace=False)`

`how`의 값이 `'any'`이면 결손 데이터가 하나라도 포함되면 제거 대상이 되고,
`'all'`이면 `axis` 인자에 따라서 행 혹은 열 전체가 결손 데이터여야 제거한다.



48

13 데이터 정제와 결손값의 처리

- 결손값을 다루는 또 하나의 방법은 결손값을 다른 값으로 교체하는 것이다. fillna() 메소드를 사용하면 특정한 값을 다른 값으로 교체한다.

```
# 결손값을 0으로 채움, inplace를 True로 설정해 원본 데이터를 수정
weather.fillna(0, inplace = True)
print(weather.loc['2012-02-11'])
```

평균기온 -0.7
최대풍속 0.0
평균풍속 0.0
Name: 2012-02-11, dtype: float64

49

13 데이터 정제와 결손값의 처리

- 결손값을 단순히 0으로 채우게 되면 데이터의 개수가 적을 경우 전체 데이터의 대표값인 평균값을 왜곡할 수 있다.
- 열의 평균치를 구한다. 그리고 이 값으로 결손값을 채우면 다음과 같이 측정이 누락된 2012년 2월 11일의 풍속을 전체 데이터의 평균으로 채울 수 있다.

```
# 이전 코드는 fillna(0,..)을 통해 0으로 채움, 다음은 평균으로 채움
# 이전 코드 실행 후 아래 코드를 실행하면 이전 코드의 영향을 받음
# 커널 재시작을 하고 실행할 것
weather.fillna( weather['평균풍속'].mean(), inplace = True)
print(weather.loc['2012-02-11'])
```

평균기온 -0.7
최대풍속 3.936441
평균풍속 3.936441
Name: 2012-02-11, dtype: float64

50

14 시계열 자료 분석을 위한 DatetimeIndex

- **시계열** *time series* 데이터란 **시간의 흐름에 따라 순차적으로 관측한 값들의 집합**으로, 이 책에서 다루는 weather 데이터프레임이 바로 시계열 데이터의 예이다.
- 판다스의 DatetimeIndex는 특정한 순간에 기록된 타임스탬프 형식의 **시계열** 자료를 다루기 위한 용도로 사용된다.
- DatetimeIndex의 사용법을 다음과 같은 예제 코드로 살펴보자. 아래와 같이 다양한 형식으로 표기된 연, 월, 일, 시간 데이터에 대하여 year, month, day, hour, minute 등과 같은 속성을 효과적으로 파싱하는 것을 볼 수 있을 것이다.

51

14 시계열 자료 분석을 위한 DatetimeIndex

```
# 다양한 형식의 연, 월, 일 표시 데이터
d_list = ["01/03/2018", "01-03-2018", "2018-01-05", "2018/01/06"]
pd.DatetimeIndex(d_list).year    # 년도 값을 출력함

Int64Index([2018, 2018, 2018, 2018], dtype='int64')

pd.DatetimeIndex(d_list).month  # 월 값을 출력함

Int64Index([1, 1, 1, 1], dtype='int64')

pd.DatetimeIndex(d_list).day    # 일 값을 출력함

Int64Index([3, 3, 5, 6], dtype='int64')

dt_list = ["01/03/2018 11:12:13", "01-03-2018 11:22:13"]
pd.DatetimeIndex(dt_list).hour  # 시 값을 출력함

Int64Index([11, 11], dtype='int64')

pd.DatetimeIndex(dt_list).minute # 분 값을 출력함

Int64Index([12, 22], dtype='int64')
```

52

14 시계열 자료 분석을 위한 DatetimeIndex

- 우선 이 데이터에서 '일시'라는 이름의 열을 사용하여 연도와 달, 날을 얻기 위해서는 다음과 같은 방법을 사용하면 된다.

▶

```

weather = pd.read_csv(weather_file, encoding='CP949')
weather['일시'] = pd.DatetimeIndex(weather['일시']).year
weather

```

	일시	평균기온	최대풍속	평균풍속
0	2010	28.7	8.3	3.4
1	2010	25.2	8.7	3.8
2	2010	22.1	6.3	2.9

<이하 생략>

53

14 시계열 자료 분석을 위한 DatetimeIndex

▶

```

... # 코드 생략 주석문
weather = pd.read_csv(weather_file, encoding='CP949')
weather['month'] = pd.DatetimeIndex(weather['일시']).month

monthly = [ None for x in range(12) ] # 달별로 구분된 12개의 None 값
monthly_wind = [ 0 for x in range(12) ] # 각 달의 평균 풍속을 담은 리스트
for i in range(12) :
    monthly[i] = weather[ weather['month'] == i + 1 ] # 달별로 분리
    monthly_wind[i] = monthly[i].mean()['평균기온'] # 개별 데이터 분석

months = np.arange(1, 13) # 1에서 12월의 연속된 수를 생성
plt.bar(months, monthly_wind, color='green')
plt.xlabel('Month')
plt.ylabel('Temperature')

```

54

15 그룹핑과 필터링

- 데이터 분석을 할 때에는 특정한 값에 기반하여 데이터를 그룹으로 묶는 일이 많다. 이를 위해서 groupby()라는 메소드를 사용할 수 있다.
- 이제 새로운 'month' 열에 대하여 다음과 같이 groupby()를 적용하여 보자.

```
... # 코드 생략
weather['month'] = pd.DatetimeIndex(weather['일시']).month
monthly_means = weather.groupby('month').mean()
print(monthly_means)
```

	평균기온	최대풍속	평균풍속
month			
1	1.598387	8.158065	3.757419
2	2.136396	8.225357	3.946786
3	6.250323	8.871935	4.390291
4	11.064667	9.305017	4.622483
5	16.564194	8.548710	4.219355

....<중간 생략>

55

15 그룹핑과 필터링

- 만일 다음과 같이 DatetimeIndex().year 속성으로 year 열을 만들 경우 연도별 평균기온, 최대풍속, 평균풍속의 평균값을 살펴볼 수 있을 것이다.

```
weather['year'] = pd.DatetimeIndex(weather['일시']).year
yearly_means = weather.groupby('year').mean()
print(yearly_means)
```

- 다음으로 필터링에 대하여 알아보자.

```
monthly_means['평균풍속'] >= 4.0
```

month	
1	False
2	False
3	True
4	True
5	True
6	False
7	False

....<중간 생략>

56

15 그룹핑과 필터링

- 이 코드를 보니 3, 4, 5월의 평균풍속만 4m/s 이상이며, 나머지는 그 미만임을 알 수 있다.
- 특정 조건을 만족하는 데이터만을 추출하는 방식은 이 조건을 활용하여 논리 인덱싱을 다음과 같이 수행한다.
- 울릉도에는 3, 4, 5월에 평균풍속이 4m/s 이상으로 바람이 많이 부는 것을 확인할 수 있다.

```
monthly_means[ monthly_means['평균풍속'] >= 4.0 ]
```

	평균기온	최대풍속	평균풍속
month			
3	6.250323	8.871935	4.390291
4	11.064667	9.305017	4.622483
5	16.564194	8.548710	4.219355

57

16 데이터 구조를 변경하는 pivot()

- 판다스에서 이와 같이 테이블의 구조를 변경하는 대표적인 메소드가 바로 pivot()이다.
- 이 df 데이터프레임을 상품의 재질을 기준으로 다시 인덱싱하고자 한다.

```
df = pd.DataFrame({'상품' : ['시계', '반지', '반지', '목걸이', '팔찌'],  
                  '재질' : ['금', '은', '백금', '금', '은'],  
                  '가격' : [500000, 20000, 350000, 300000, 60000]})
```

df

	상품	재질	가격
0	시계	금	500000
1	반지	은	20000
2	반지	백금	350000
3	목걸이	금	300000
4	팔찌	은	60000

58

16 데이터 구조를 변경하는 pivot()

- 이때 사용된 pivot() 메소드에서 index='상품'을 사용하여 상품을 인덱스로 지정하도록 하자.
- 또한, columns='재질' 인자를 사용하여 재질을 열로하는 새로운 데이터프레임을 만들 수 있다.
- 이 데이터프레임은 가격을 값으로 가지는데 이 값들 중에서 존재하지 않는 항목은 fillna() 메소드를 사용해서 0으로 채워주었다.

```
new_df = df.pivot(index='상품', columns='재질', values='가격')
new_df.fillna(value=0)
```

재질	금	백금	은
상품			
목걸이	300000.0	0.0	0.0
반지	0.0	350000.0	20000.0
시계	500000.0	0.0	0.0
팔찌	0.0	0.0	60000.0

59

16 데이터 구조를 변경하는 pivot()

- 이와 같이 구조를 변경하게 되면, 다음과 같이 재질 컬럼스의 값인 '금', '백금', '은' 각각이 새로운 컬럼 값이 되며 내부는 가격이라는 열의 값으로 채워지게 된다.

df.pivot(index='상품', columns='재질', values='가격')

	상품	재질	가격
0	시계	금	500000
1	반지	은	20000
2	반지	백금	350000
3	목걸이	금	300000
4	팔찌	은	600

재질	금	백금	은
상품			
목걸이	300000.0	0.0	0.0
반지	0.0	350000.0	20000.0
시계	500000.0	0.0	0.0
팔찌	0.0	0.0	60000.0

new_df.fillna(value = 0)
값이 없는 셀은 0으로 채움

60

16 데이터 구조를 변경하는 pivot()

- 위의 pivot() 메소드와 유사한 pivot_table() 이라는 메소드를 통해서 데이터를 재구조화하는 것도 가능하다.
- 하지만 인덱스가 2개 이상인 경우 pivot() 메소드로는 재구조화가 안되며 pivot_table()을 사용해야만 한다.
- 마찬가지로 column이 2개 이상인 경우 pivot() 메소드로는 재구조화가 안되며 pivot_table()을 사용해야만 한다.

데이터 구조를 변경하는 방법은 pivot()과 pivot_table()로 할 수 있습니다. pivot_table() 메소드를 사용하여 좀 더 고급스러운 재구조화를 할 수 있지요.



61

17 두 개의 데이터프레임을 하나로 합치는 concat()

- 일반적으로 데이터들은 작은 테이블로 나누어져 있는 경우가 많다. 이러한 데이터를 하나로 합치는 방법 가운데 하나인 concat() 함수를 살펴보자.
- 이 두 개의 데이터프레임을 결합하여 하나의 데이터프레임으로 만드는 방법에 대해 알아볼 것이다.

```
df_1 = pd.DataFrame( {'A' : ['a10', 'a11', 'a12'],
                      'B' : ['b10', 'b11', 'b12'],
                      'C' : ['c10', 'c11', 'c12']} ,
                      index = ['가', '나', '다'] )

df_2 = pd.DataFrame( {'B' : ['b23', 'b24', 'b25'],
                      'C' : ['c23', 'c24', 'c25'],
                      'D' : ['d23', 'd24', 'd25']} ,
                      index = ['다', '라', '마'] )
```

62

17 두 개의 데이터프레임을 하나로 합치는 concat()

- 이 두 데이터프레임의 결합을 위해 concat() 함수를 사용하면 된다.
- 이 함수의 첫 인자는 데이터프레임의 리스트이다.

df_1			
	A	B	C
가	a10	b10	c10
나	a11	b11	c11
다	a12	b12	c12

인덱스

df_2			
	B	C	D
다	b25	c25	d25
라	b24	c24	d24
마	b23	c23	d23

인덱스

- 그리고 두 개의 중요한 키워드 매개변수 axis와 join의 의미가 그림에 설명되어 있다.

합칠 데이터프레임의 리스트
 pandas.concat(df_list, axis=0, join='outer')

축이 0이면 테이블의 행을 늘려서 붙여 나감
 축이 1이면 테이블의 열을 늘리며 붙여 나감

join 매개변수는 테이블들을 붙일 때 레이블들을 어떻게 사용할지 결정한다.
 이것의 인자가 'outer'이면 레이블들의 합집합으로 생성하고 'inner'이면
 레이블들의 교집합으로 생성된다.



63

17 두 개의 데이터프레임을 하나로 합치는 concat()

- df_1과 df_2의 결합을 위하여 concat() 함수의 인자로 합칠 데이터프레임 [df_1, df_2]를 인자로 주었다.

```

df_3 = pd.concat( [df_1, df_2] )
print(df_3)

```

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b23	c23	d23
라	NaN	b24	c24	d24
마	NaN	b25	c25	d25

64

17 두 개의 데이터프레임을 하나로 합치는 concat()

- concat() 함수에서 별다른 인자를 주지 않을 경우 디폴트 축0을 사용하여 행을 늘려서 테이블을 붙여간다.
- 따라서 다음과 같은 특징을 가지는 새로운 테이블이 만들어졌다.

	A	B	C	D
가	a10	b10	c10	NaN
나	a11	b11	c11	NaN
다	a12	b12	c12	NaN
다	NaN	b25	c25	d25
라	NaN	b24	c24	d24
마	NaN	b23	c23	d23

- 두 테이블이 각각 3개의 행을 가지므로, 모두 6개의 행을 가지는 테이블이 생성.
- 인덱스 '다'는 두 데이터프레임에 모두 존재하므로 별도의 행으로 중복됨.
- 'outer'를 디폴트 결합 방식으로 사용하여, A, B, C와 B, C, D의 합집합인 A, B, C, D가 최종적인 열이 됨.

65

17 두 개의 데이터프레임을 하나로 합치는 concat()

- 만일 다음과 같이 join='inner'를 인자로 줄 경우 두 테이블의 교집합인 B, C 열이 최종 열이 되는 것도 확인할 수 있다.

```
df_4 = pd.concat([df_1, df_2], join='inner')
print(df_4)
```

```
   B    C
가 b10 c10
나 b11 c11
다 b12 c12
다 b23 c23
라 b24 c24
마 b25 c25
```

66

18 테이블 데이터의 결합: concat()과 merge()

- concat()은 판다스의 함수로 결합을 위하여 하나 이상의 데이터프레임을 인자로 받는다.
- 반면 merge()는 데이터프레임의 메소드이며, 두 데이터프레임을 각 데이터에 존재하는 고유값(key)을 기준으로 병합할때 사용한다.

`DataFrame.merge(right, how='inner', on=None)`

right : 현재의 데이터프레임과 결합할 데이터프레임

how : 결합의 방식 'left', 'right', 'inner', 'outer' 가능

on : 조인 연산을 수행하기 위해 사용할 레이블(두 데이터프레임 모두에 존재해야 함)

67

18 테이블 데이터의 결합: concat()과 merge()

- on='B'를 이용하여 'B' 레이블을 가진 열의 데이터를 키로 조인 연산을 하였다.
- how 매개변수에는 아래와 같이 네 종류의 인자를 넘길 수 있다.
- on='B'를 유지한 상태로 how를 변경하여 다양한 결과를 확인해 보라.

```
print('left outer \n' , df_1.merge(df_2, how='left', on='B' ) )
print('right outer \n' ,df_1.merge(df_2, how='right', on='B' ) )
print('full outer \n' ,df_1.merge(df_2, how='outer', on='B' ) )
print('inner \n' ,df_1.merge(df_2, how='inner', on='B' ) )
```

left outer					full outer						
	A	B	C_x	C_y	D		A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN	0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN	1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN	2	a12	b12	c12	NaN	NaN
						3	NaN	b23	NaN	c23	d23
						4	NaN	b24	NaN	c24	d24
						5	NaN	b25	NaN	c25	d25

right outer					
	A	B	C_x	C_y	D
0	NaN	b23	NaN	c23	d23
1	NaN	b24	NaN	c24	d24
2	NaN	b25	NaN	c25	d25

inner
Empty DataFrame

68

18 테이블 데이터의 결합: concat()과 merge()

- 두 데이터프레임에서는 'B'뿐만이 아니라 'C' 레이블도 두 테이블에 동시에 나타난다.
- 이럴 경우 왼쪽 테이블에서 가져온 것은 'C_x', 오른쪽에서 가져온 것은 'C_y'로 이름을 새로 붙여 테이블을 만든다.

```
df_3 = df_1.merge(df_2, how='outer', on='B')
print(df_3)
```

	A	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN
3	NaN	b23	NaN	c23	d23
4	NaN	b24	NaN	c24	d24
5	NaN	b25	NaN	c25	d25

69

18 테이블 데이터의 결합: concat()과 merge()

- merge() 함수의 동작 결과를 살펴보면, 원래 테이블에 있던 인덱스는 사라진 것을 확인할 수 있다.
- 많은 경우 인덱스가 키의 역할을 수행하는 경우도 있다. 이런 경우에 인덱스를 키로 사용하라고 할 수도 있다.

on = 'B'

	left	B	C_x	C_y	D
0	a10	b10	c10	NaN	NaN
1	a11	b11	c11	NaN	NaN
2	a12	b12	c12	NaN	NaN
3	NaN	b23	NaN	c23	d23
4	NaN	b24	NaN	c24	d24
5	NaN	b25	NaN	c25	d25

right

outer: b10, b11, b12, b23, b24, b25

inner: 교집합 없음
{b10, b11, b12}
{b23, b24, b25}

70

18 테이블 데이터의 결합: concat()과 merge()

- 양쪽 테이블의 인덱스를 모두 사용하여 병합의 키가 되도록 하는 방식은 아래와 같다.
- 'outer' 방식이므로 두 인덱스의 합집합이 새로운 인덱스가 된다.
- 'inner'라면 교집합으로 '다' 행만 남게 된다.

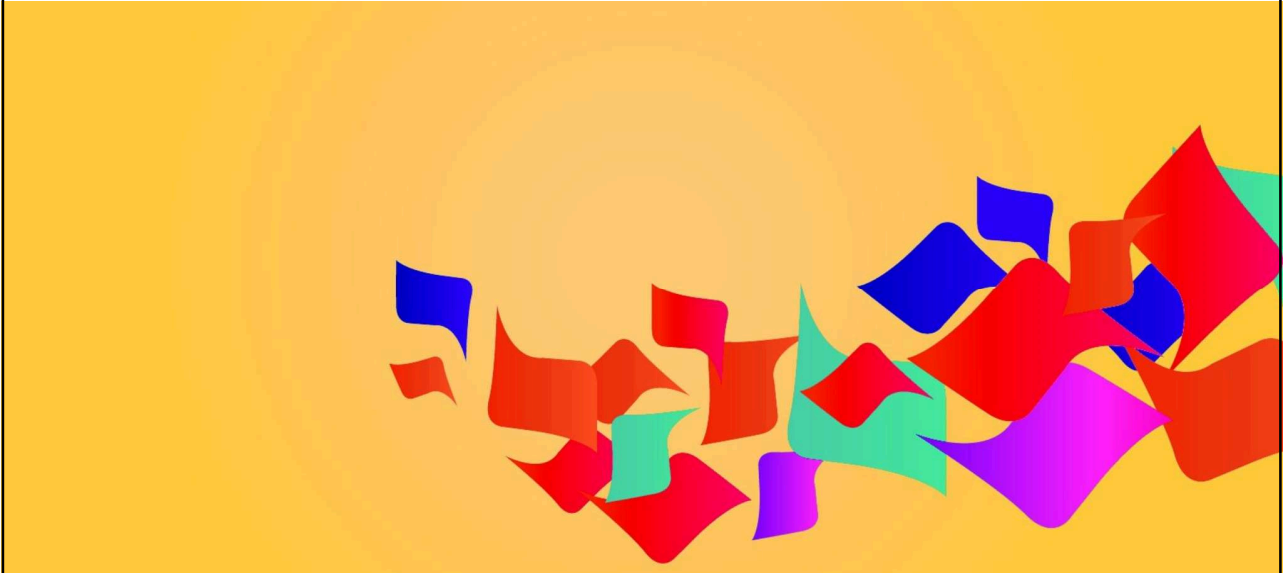
```
df_3 = df_1.merge(df_2, how = 'outer',
                  left_index = True, right_index = True )
print(df_3)
```

	A	B_x	C_x	B_y	C_y	D
가	a10	b10	c10	NaN	NaN	NaN
나	a11	b11	c11	NaN	NaN	NaN
다	a12	b12	c12	b23	c23	d23
라	NaN	NaN	NaN	b24	c24	d24
마	NaN	NaN	NaN	b25	c25	d25

자료출처 : 으뜸 데이터 분석과 머신러닝 / 박동규 · 강영민 지음 / 생능출판사 / 2021



기계학습과 sklearn



193

기계학습이란?

- **기계학습** machine learning은 인공 지능의 한 분야로, 컴퓨터에 학습 기능을 부여하기 위한 연구 분야
- “기계학습”이란 용어는 1959년 **아서 사무엘** Arthur Samuel에 의해 만들어짐
- 패턴 인식 및 계산 학습 이론에서 진화한 기계학습은 주어진 데이터를 보고 컴퓨터가 판단 방법을 학습하게 만드는 것
- 학습에 사용할 수 있는 데이터가 많아지면 판단을 내리는 알고리즘 성능이 향상 됨
- 학습 알고리즘은 항상 정해진 동작을 수행하는 명령어로 구성된 알고리즘과 달리 데이터를 이용하여 예측하고 판단할 수 있게 됨
- 기계학습이 주로 사용되는 분야는 풀이 방법을 적시하여 컴퓨터가 할 일을 나열하기 어려운 문제를 다루는 곳
- 예를 들어서 스팸 메일 필터링, 네트워크 침입자 자동검출, **광학 문자 인식** OCR: Optical Character Recognition, 컴퓨터 비전, 자율주행 등의 분야에서 활발하게 사용

194

194

기계학습이란?

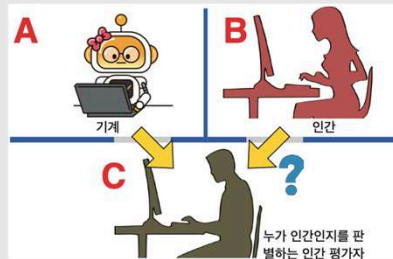


잠깐 - 컴퓨터의 아버지 앨런 튜링과 튜링 테스트

앨런 튜링은 영국의 수학자, 암호학자, 논리학자이자 선구적인 컴퓨터 과학자이다. 그는 알고리즘과 계산 개념을 튜링 기계라는 추상 모델을 통하여 형식화하였다. 이 업적은 컴퓨터 과학이라는 분야를 여는데 큰 기여를 하게 된다.

그는 **튜링 테스트(turing test)**의 고안자로 유명한데, 컴퓨터가 인간과 같은 사고를 한다는 것은 어떻게 보여줄 수 있을까 하는 것이 튜링 테스트의 목적이다. 이 테스트는 인간 평가자 C가 볼 수 없는 벽 건너편에 컴퓨터와 인간이 있고 C는 누가 인간인지를 모르는 상태이다. A와 B가 키보드를 통해서만 인간 평가자와 대화를 하였을 때 인간 평가자가 컴퓨터와 인간을 확실하게 구분할 수 없을 경우 이 컴퓨터 A는 튜링 테스트를 통과했다고 볼 수 있다.

튜링 테스트는 컴퓨터가 인간처럼 생각을 한다고 판정하는 데에 사용할 수 있는 테스트로 여겨졌다. 그러나 기계학습이 발전하고 인공지능 분야의 연구가 심화됨에 따라 다양한 비판이 나타났다. 이 테스트는 지능보다는 인간과의 유사성을 측정하는 것에 가깝다는 비판이 제기되었기 때문이다. 실제로 이 테스트를 통과했다는 논란이 있었던 몇몇 프로그램은 사실 자신이 한 답변을 전혀 이해하지 못한 상태에서 주어진 데이터를 바탕으로 대화를 한 것으로 보여지기 때문이다.



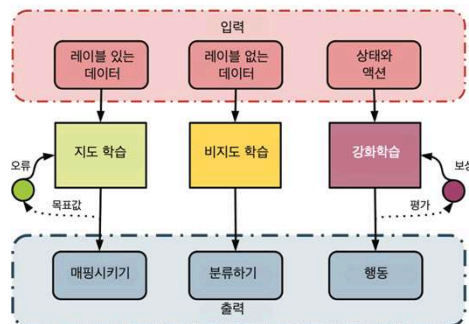
195

195

기계학습이란?

다양한 종류의 기계학습 기법

- 기계학습은 일반적으로 가르쳐주는 "교사"의 존재 여부에 따라 크게 지도 학습과 자율 학습으로 나누어진다.



다양한 기계학습 알고리즘과 그 차이점

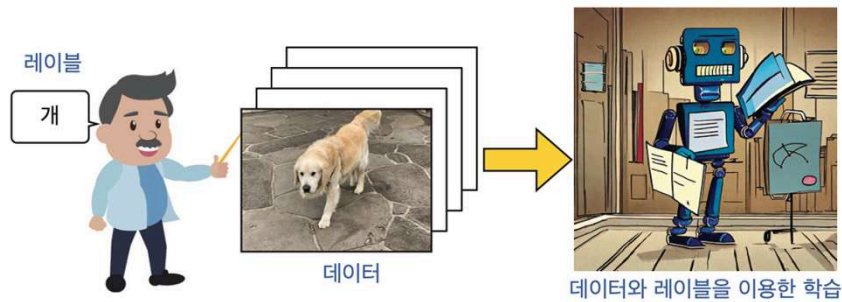
196

196

기계학습이란?

다양한 종류의 기계학습 기법 (계속)

- **지도 학습** *supervised learning*: 컴퓨터는 "교사"에 의해 주어진 예제와 정답(혹은 레이블)을 제공받는다.
- 지도학습의 목표는 입력을 출력에 매핑하는 일반적인 규칙을 학습하는 것이다.

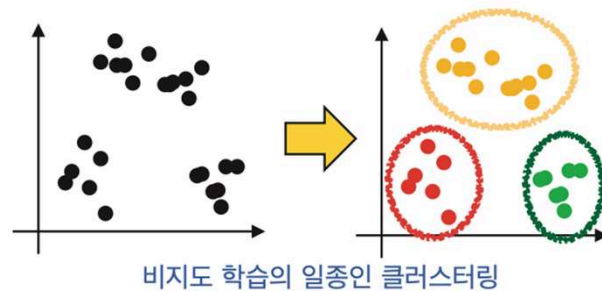


197

197

기계학습이란?

다양한 종류의 기계학습 기법 (계속)



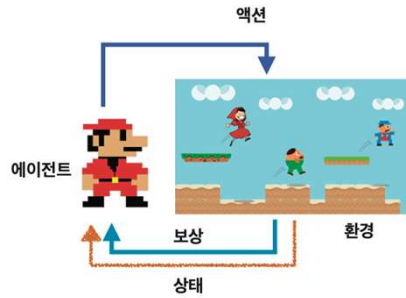
- **자율 학습** *unsupervised learning*은 대표적인 것이 위에 나타난 **클러스터링** *clustering*이다.
- 이 데이터의 경우 두 개의 큰 그룹으로 나누어 질 수 있는데, 이 규칙은 컴퓨터가 데이터를 보고 스스로 학습하는 것이다

198

198

기계학습이란?

다양한 종류의 기계학습 기법 (계속)



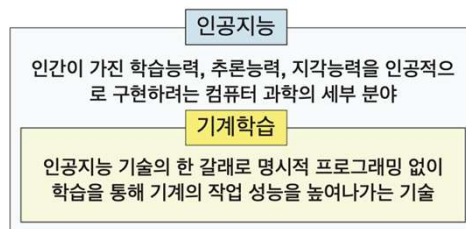
- **강화 학습** reinforcement learning은 보상 및 처벌의 형태로 학습 데이터가 주어진다.
- 주로 차량 운전이나 상대방과의 경기 같은 동적인 환경에서 프로그램의 행동에 대한 피드백만 제공되는 경우이다.

199

199

기계학습을 깊이 알아보자

- 이번 장에서는 인공지능 기술의 여러 분야 중에서 핵심 분야에 해당하는 **기계학습**에 대하여 알아보도록 하자.
- 기계학습은 명시적 프로그래밍 없이 컴퓨터가 학습을 통해 작업 성능을 높여나가는 기술을 말한다.
- 따라서 두 분야를 다이어그램으로 표기하면 그림과 같이 인공지능이 기계학습을 포함하는 관계로 기술할 수 있을 것이다.



200

200

기계학습을 깊이 알아보자

- 기계학습의 영어 이름인 **머신 러닝**이라는 용어에 대한 공학적인 정의로는 카네기 멜런 대학교 교수인 톰 미첼Tom Mitchell이 그의 저서 "머신러닝"에서 제시한 것이 흔히 사용된다.
- 그는 기계학습을 다음과 같이 정의했다.

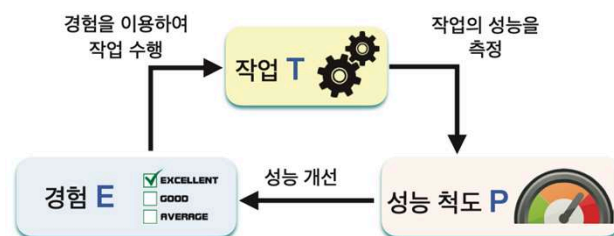
"컴퓨터 프로그램이 어떤 작업 T에 속한 작업을 수행하면서 경험 E에 따라서 P로 측정하는 성능이 개선된다면, 이 프로그램은 작업 T와 성능 척도 P에 대해 경험 E로부터 학습을 한다고 말할 수 있다."

201

201

기계학습을 깊이 알아보자

- 기계학습의 정의에서 해결해야 할 문제(**작업**)가 T이다.
- 그리고 이 일을 수행하는 동작을 P라는 **성능 척도**를 통해 평가할 수 있어야 한다.
- 그리고 지속적인 훈련 **경험 E**를 통해 이러한 평가의 점수를 더 나은 상태로 바꿀 수 있어야 하는 것이다.



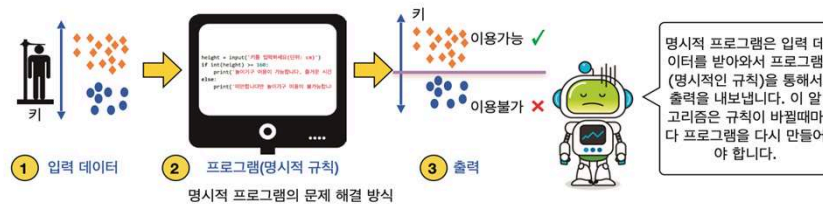
톰 미첼의 정의에 따른 기계학습의 요소

202

202

기계학습을 깊이 알아보자

- 키가 160cm 이상인 사람만 이용 가능한 놀이기구가 있다고 가정하고 명시적 프로그램으로 이 문제를 해결해 보자.
- 다음 그림과 같이 입력과 출력 그리고 명시적인 규칙(프로그램)이라는 세 가지 구성요소를 얻을 수 있을 것이다.
- 입력 데이터는 사람의 키가 될 것이며, 프로그램은 데이터를 입력으로 받아 키가 160cm 이상인지 미만인지를 판단하고 이 판단에 따라 서로 다른 출력을 내는 방식으로 동작할 것이다.

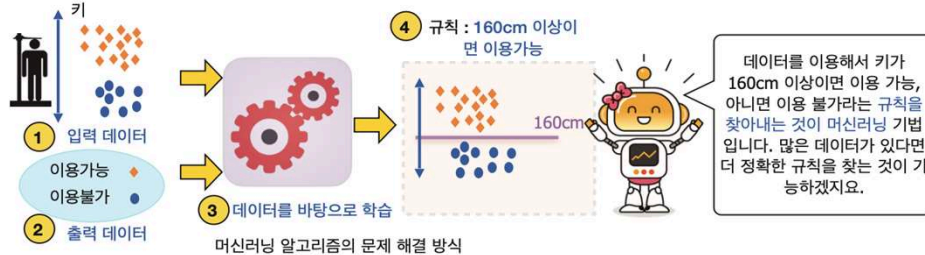


203

203

기계학습을 깊이 알아보자

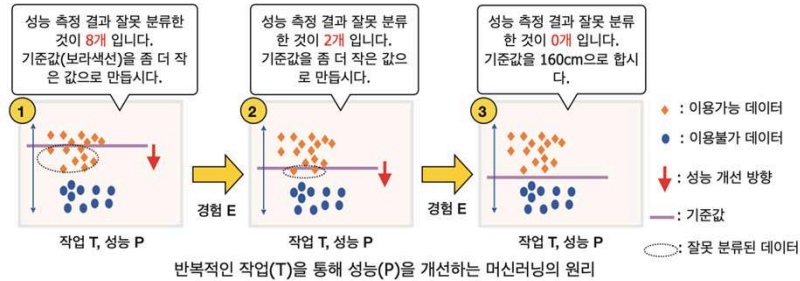
- 다음 그림과 같은 상황에서 머신러닝 알고리즘은
- ① 입장객의 키 데이터와 ② 이용 가능/이용 불가의 결과를 바탕으로
- ③ 스스로 학습하여 가장 적절한 규칙인 ④ "키 160cm 이상 이용 가능"이라는 규칙을 만들어낸다.
- 이 규칙은 새로운 데이터에도 적용되어 키가 입력되면 이용 가능/이용 불가의 결과를 반환한다.



204

204

기계학습을 깊이 알아보자



- 그림에서 보면 세로 축을 키라고 할 때 사람들의 키 데이터와 이용 가능/이용 불가의 정보가 주어져있다.
- 주황색 다이아몬드는 이용 가능한 사람의 데이터이고, 파란색 원은 이용 불가능한 사람의 데이터이다.

205

205

기계학습을 깊이 알아보자

- 기준값이 보라색 선으로 그림 ①과 같이 주어져 있다면, 이 기준 값은 이용가능/이용 불가를 판단하는 기준으로는 부적합할 것이다.
- 적합/부적합의 기준은 잘못 분류된 데이터의 개수가 될 수 있을 것 이며 그림 ①에서 이 값은 8이다.
- 이러한 경험을 바탕으로 기준값을 그림 ② 와 같이 성능을 개선시켜본다면 어떻게 될까?
- 그림 ①보다는 잘못 분류한 데이터의 개수가 줄어들었을 것이다.
- 이 경험을 바탕으로 기준값을 좀 더 아래로 이동해 그림 ③과 같이 위치시킨다면 잘못 분류한 것이 0개가 된다.
- 이와 같은 반복적인 작업을 통해서 성능을 개선시키는 것이 머신러닝의 원리이다.

206

206

기계학습을 깊이 알아보자

- **작업 T:** 키 데이터를 사용하여 이용 가능/이용 불가 고객의 분류 기준값을 찾는다.
- **성능 척도 P:** 기준값을 적용하여 발생한 오류의 개수이다.
- **경험 E:** 오류의 개수가 적게 나오도록 분류를 위한 기준값을 변경한다.

207

207

회귀문제를 알아보자

- **회귀regression**란 일반적으로 데이터들을 다차원 공간에 표시한 후에 이들 데이터들을 가장 잘 설명하는 직선이나 곡선을 찾는 문제
- 어떤 연구자가 특정 지역의 주택 면적과 최근 2년의 거래 가격의 관계를 조사하는 경우를 생각해보자.
- 일반적으로 주택의 면적 값이 큰 경우 거래 가격도 높은 경우가 많은데, 여기서 다른 변수에 영향을 덜 받는 변수인 **주택의 면적**은 **독립변수**가 되며, 이에 영향을 받아서 변화할 수 있는 **거래 가격**이 **종속변수**가 될 것이다.
- 종속변수는 연구자가 **독립변수의 변화에 따라 어떻게 변하는가 알고 싶은 변수**가 된다.

용어	설명
변수	변경될 수 있는 양이나 조건
독립변수	연구자가 임의로 조절할 수 있는 변수로 실험 영역에 있어 다른 변수에 영향을 받지 않는 변수
종속변수	관측이나 측정이 가능한 변수로 독립변수에 영향을 받아서 변화하는 변수

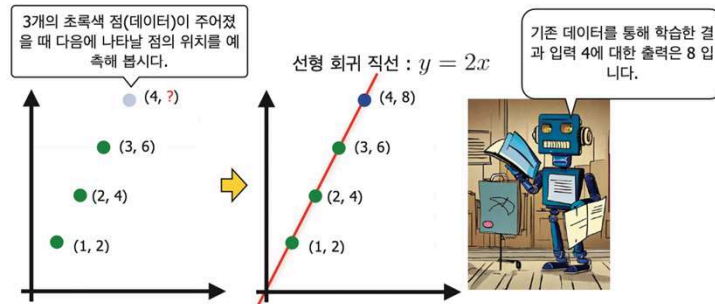


208

208

회귀문제를 알아보자

- 매우 간단한 회귀 문제의 예를 들어보자. (x, y) 형태의 입력 데이터로 점 $(1, 2), (2, 4), (3, 6)$ 이 주어져 있다고 하자. 컴퓨터는 x 값에 대하여 y 값이 $y=2x$ 의 방정식으로 표현될 수 있는 데이터라는 것을 **아직 모르는 상태**이다.
- 입력된 값을 바탕으로 컴퓨터가 스스로 이 입력을 설명할 수 있는 가장 좋은 함수를 찾는 것이 바로 **회귀분석** regression이라 할 수 있다.



209

209

회귀문제를 알아보자

정답과 예측값의 차이를 줄여주는 수학 도구

- 다음그림 ①번과 같이 임의의 기울기 파라미터 m 을 이용하여 직선을 그려보면 이 직선이 데이터의 분포를 제대로 잘 설명하지 못하는 것을 볼 수 있는데, "데이터의 분포를 잘 설명한다", "데이터의 분포를 잘 설명하지 못한다"와 같은 주관적인 표현을 수학자들과 컴퓨터 과학자들은 **정량화**를 하는 방법을 사용한다.
- 그림을 살펴보면 빨간색 실선이 선형 회귀 직선이며 초록색 점들이 데이터, 그리고 e_i 로 표기된 하늘색 직선이 선형 회귀 직선과 실제 데이터 i 와의 거리임을 알 수 있다.
- 이 거리는 **오차** error, **손실** loss 또는 **잔차** residual라고 부르는데, i 번째 데이터와의 오차를 표현하기 위하여 영문자 error의 앞글자를 따서 e_i 와 같이 표기하도록 한다.

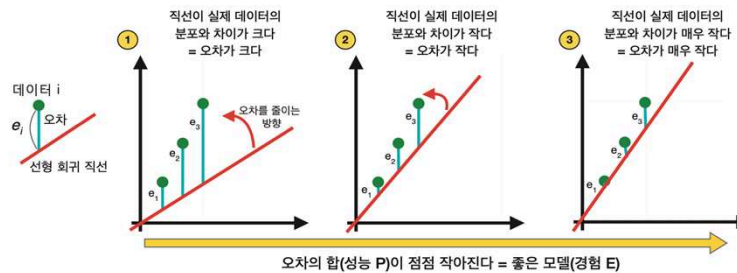
210

210

회귀문제를 알아보자

정답과 예측값의 차이를 줄여주는 수학 도구 (계속)

- 그림을 살펴보면 ①번에 있는 직선보다 ②번의 직선이 더 실제 데이터의 분포를 잘 설명하는 직선이며, ③번에 있는 직선은 ②번의 직선보다 더 나은 직선이다.
- 이와 같이 ①번 → ②번 → ③번과 같이 직선이 오차를 줄이는 방향으로 스스로 고쳐나가는 것을 수학적으로 이야기하면 오차의 합이 점점 작아진다고 할 수 있으며 이는 곧 좋은 모델이 된다는 의미이다.



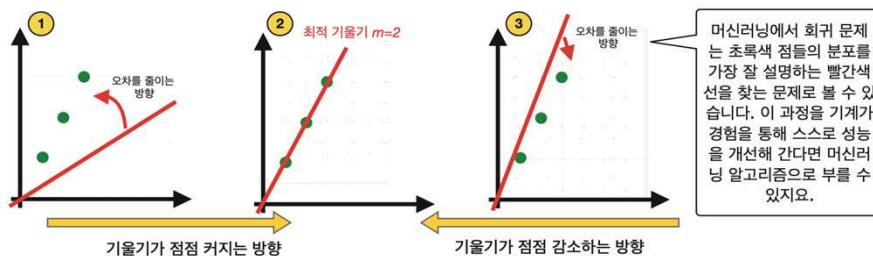
211

211

회귀문제를 알아보자

정답과 예측값의 차이를 줄여주는 수학 도구 (계속)

- 앞서 살펴본 최적의 기울기를 구하는 과정은 다음 그림과 같이 최적의 기울기보다 기울기가 더 클 때(그림 ③번) 오차가 줄어들도록 기울기를 감소시키는 방향으로 개선되는 것을 포함해야만 한다.
- 이에 관한 수학적인 모델링과 기법은 이 책의 범위를 벗어나며 우리는 사이킷런을 사용하여 최적 기울기를 구하도록 명령을 내리는 것 위주로 살펴볼 것이다.



212

212

가장 간단한 회귀 : 선형 회귀 분석

- Scikit-learn
 - 기계학습을 위한 라이브러리
 - 선형회귀, k-NN 알고리즘, 서포트 벡터머신, 랜덤 포레스트, 그래디언트 부스팅, k-means 등의 분류, 회귀, 클러스터링 알고리즘을 포함하고 있어서 기계학습을 처음 접하는 사람들에게 좋은 도구로 인기를 얻고 있다
- 선형 회귀는 임의의 변수 x (독립 변수)와 이 변수에 따른 또 다른 변수 y (종속 변수)와의 상관관계를 모델링하는 기법
 - 두 변수의 관계를 알아내거나 이를 이용하여 y 가 없는 x 값에 대하여 y 를 예측하는데 사용되는 통계학의 기법

213

213

가장 간단한 회귀 : 선형 회귀 분석

- **선형 회귀**는 변수 x 와 이 변수에 따른 또 다른 변수 y 와의 상관관계를 모델링하는 기법이다.
- 여기서 m 은 직선의 **기울기**이고 입력 변수에 곱해지는 **계수**coefficient이다.
- b 는 **절편**intercept이다.

$$y = mx + b$$

- 이 개념은 2개 이상의 변수가 있는 경우까지 확장될 수 있다.
- 이를 다중회귀분석이라고 한다. p 개의 변수가 포함된 회귀 모델은 다음과 같이 나타낼 수 있다.

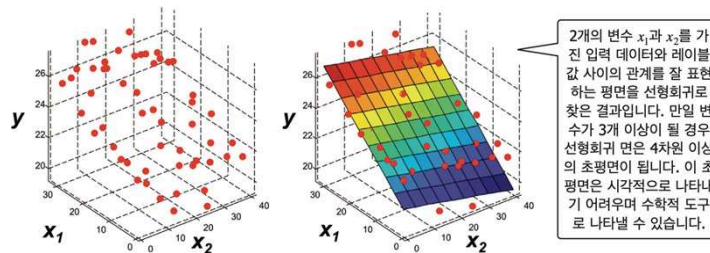
$$\hat{y}(\mathbf{w}, \mathbf{x}) = w_0 + w_1x_1 + \cdots + w_px_p$$

214

214

가장 간단한 회귀 : 선형 회귀 분석

- 여기서 $\mathbf{w}$ 와 $\mathbf{x}$ 는 모두 벡터이고 w_0 을 제외한 $\mathbf{w} = (w_1, w_2, \dots, w_p)$ 를 계수coefficient, w_0 를 절편intercept이라고 한다.
- 이것은 사실 평면의 방정식이다.
- 2차원 공간에서 선형 회귀 모형은 직선이고 3차원에서는 평면이고, 3차원 이상에서는 초평면hyperplane이다. 아래 그림은 2개의 변수를 가진 입력 데이터와 레이블 값 사이의 관계를 잘 표현하는 평면을 선형회귀로 찾은 결과이다.



215

215

가장 간단한 회귀 : 선형 회귀 분석

선형 회귀를 사이킷런 라이브러리로 구현해 보자

- 선형회귀를 사이킷런 라이브러리로 구현하는 방법을 알아보자.
- 철수네 반에 있는 4명의 학생을 임의로 추출하여 키와 몸무게를 측정하였더니 키가 164, 179, 162, 170 cm로 나타났으며, 이들의 몸무게는 각각 53, 63, 55, 59kg으로 나타났다.
- 일반적으로 키와 몸무게는 상관관계가 있기때문에 이 상관관계를 선형 방정식으로 만들 수 있다면 측정한 학생이 아닌 **키가 169cm인 학생의 몸무게를 유추**할 수 있을 것이다 (이하 cm, kg 단위는 생략한다).
- 선형회귀를 위해 선형 모델 linear_model을 import한 뒤에 LinearRegression() 함수를 통해 선형회귀 모델을 생성한다. 생성된 모델을 변수 regr로 받아 보자.

```
import numpy as np
from sklearn import linear_model # scikit-learn 모듈을 가져온다

regr = linear_model.LinearRegression()
```

216

216

가장 간단한 회귀 : 선형 회귀 분석

선형 회귀를 사이킷런 라이브러리로 구현해 보자 (계속)

- 선형회귀를 위한 입력 데이터 x 를 $[[164], [179], [162], [170]]$ 와 같은 2차원 리스트에 넣도록 한다. 다음으로 y 변수를 $[53, 63, 55, 59]$ 와 같이 초기화 하도록 하자.
- 이제 이 데이터를 이용하여 선형 회귀 학습을 시작해 보자. `regr.fit(X, y)`와 같이 선형 회귀 모델에 입력과 출력을 지정하면 된다.

```
▶ X = [[164], [179], [162], [170]] # 다중회귀에도 사용하도록 함
  y = [53, 63, 55, 59] # y = f(X)의 결과
  regr.fit(X, y)
```

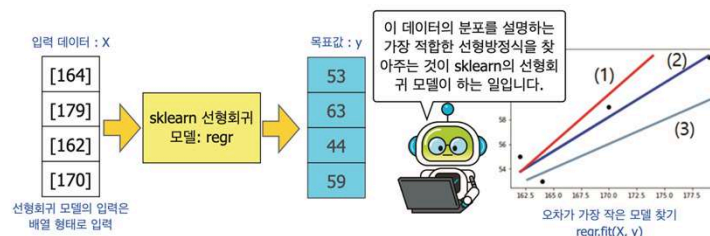
217

217

가장 간단한 회귀 : 선형 회귀 분석

선형 회귀를 사이킷런 라이브러리로 구현해 보자 (계속)

- 그림을 보면 이 점들의 분포를 표현하는 직선이 (1), (2), (3)의 세 종류가 있는데 `sklearn`의 `LinearRegression` 객체는 가능한 모든 직선들 가운데 가장 데이터를 잘 따르는 직선을 계산해낼 수 있다.



218

218

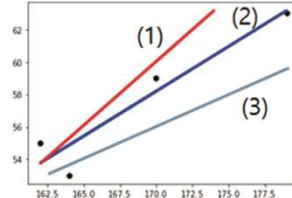
선형회귀로 예측하기: 키와 몸무게는 상관관계가 있을까

선형회귀 학습결과를 확인하고 예측하기

- 앞에서 설명한 선형회귀 방정식을 구하는 기계학습은 sklearn에서 다음과 같은 코드로 간단히 이루어졌다.

```
regr.fit(X, y)
```

$$y = w_0 + w_1 X_1$$



- 여기서 직선의 기울기는 (w_1)이고 절편은 w_0 이다. 이것들이 이 직선의 방정식을 결정한다. 이 값을 확인하고 싶으면 선형회귀 모델이 가지고 있는 특성값 `coef_`(기울기)와 `intercept_`(절편)를 확인하면 된다.
- 그리고 이 값들이 입력 x 에 대해 y 를 예측하는 데에 얼마나 적합한지는 `score()` 함수를 통해 확인할 수 있다.

219

219

선형회귀로 예측하기: 키와 몸무게는 상관관계가 있을까

선형회귀 학습결과를 확인하고 예측하기 (계속)

- 선형관계에 가까운 데이터를 사용했기 때문에 1에 가까운 0.9 정도의 값을 얻을 수 있다.

```
# 직선의 기울기
coef = regr.coef_
# 직선의 절편
intercept = regr.intercept_
# 학습된 직선이 데이터를 얼마나 잘 따르나
score = regr.score(X, y)

print("y =", coef, "* X + ", intercept)
print("The score of this line for the data: ", score)

y = [0.55221745] * X + -35.686695278969964
The score of this line for the data: 0.903203123105647
```

- 두 건의 입력에 대한 예측을 해 보자. X 의 값이 180일 경우와 185일 경우에 y 의 값은 어떨지 예측하는 것이다.

```
input_data = [ [180], [185] ]
```

220

220

선형회귀로 예측하기: 키와 몸무게는 상관관계가 있을까

선형회귀 학습결과를 확인하고 예측하기 (계속)

- 이것을 predict() 함수에 넘겨주면 예측결과를 얻을 수 있다.

```
result = regr.predict(input_data)
print(result)

[63.71244635 66.47353362]
```

- 선형관계에 가까운 데이터를 사용했기 때문에 1에 가까운 0.9 정도의 값을 얻을 수 있다.

```
coef = regr.coef_          # 직선의 기울기
intercept = regr.intercept_ # 직선의 절편
score = regr.score(X, y)    # 학습된 직선이 데이터를 얼마나 잘 따르나

print("y =", coef, "* X + ", intercept)
print("The score of this line for the data: ", score)

y = [0.55221745] * X + -35.686695278969964
The score of this line for the data: 0.903203123105647
```

221

221

선형회귀로 예측하기: 키와 몸무게는 상관관계가 있을까

선형회귀 학습결과를 확인하고 예측하기 (계속)

- 두 건의 입력에 대한 예측을 해 보자. X의 값이 180일 경우와 185일 경우에 y의 값은 어떠할지 예측하는 것이다.

```
input_data = [ [180], [185] ]
```

- 이것을 predict() 함수에 넘겨주면 예측결과를 얻을 수 있다.

```
result = regr.predict(input_data)
print(result)

[63.71244635 66.47353362]
```

222

222

선형회귀로 예측하기: 키와 몸무게는 상관관계가 있을까

선형회귀 학습결과를 확인하고 예측하기 (계속)

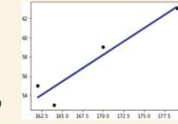
- 이제 이 선형회귀 모델이 키가 169인 학생의 몸무게를 얼마로 예측하는지를 알아보자. 이를 위하여 다음과 같은 predict() 함수를 사용한다.

```
regr.predict([[169]])  
array([57.63805436])
```



- 이 선형회귀를 그래프로 그려보자. 앞에서 다룬 코드와 함께 플로팅을 위한 matplotlib 라이브러리도 함께 사용해 보도록 하자.

```
import matplotlib.pyplot as plt  
import numpy as np  
from sklearn import linear_model # scikit-learn 모듈을 가져온다  
  
regr = linear_model.LinearRegression()  
  
X = [[164], [175], [162], [178]] # 선형회귀의 입력은 2차원으로 만들어야 함  
y = [53, 63, 55, 59] # y = f(X)의 결과값  
regr.fit(X, y)  
  
# 학습 데이터의 y 값을 선모도로 그린다.  
plt.scatter(X, y, color='black')  
# 학습 데이터를 입력으로 하여 예측값을 계산한다.  
y_pred = regr.predict(X)  
# 학습 데이터의 예측값으로 선그래프로 그린다.  
# 계산된 기울기와 y 절편을 가지는 직선이 그려진다  
plt.plot(X, y_pred, color='blue', linewidth=1)  
plt.show()
```



223

223

선형회귀로 예측하기: 키와 몸무게는 상관관계가 있을까

선형회귀 학습결과를 확인하고 예측하기 (계속)



잠깐 - 선형회귀의 원리는 오류를 줄이기 위한 최적화 기법에 있다

사이킷런에서 제공하는 LinearRegression 객체가 입력된 데이터의 분포를 가장 잘 설명하는 선형방정식을 찾아내는 것을 확인하였다. 그렇다면 어떻게 이 선형방정식을 찾은 것일까?

선형회귀를 위한 절편과 계수를 찾는 방법 중에 **정규 방정식(normal equation)**이라는 닫힌 형태의 계산식을 이용하여 구하는 방법이 있다. 사이킷런이 선형회귀 문제를 풀 때, 이 방법에 근거하여 푼다. 그런데 이 방법은 특징의 개수(입력의 차원)가 많을 경우에 느려지고, 다양한 문제에 최적화 방법에 쓰기 어렵다. 다양한 문제에 널리 이용되는 최적화 방법은 **경사하강법(gradient descent)**이라는 방법이다. 경사하강법은 임의의 기울기와 절편을 가지는 선형방정식을 생성한 후 이 선형방정식의 예측값 $\hat{y}$ 와 실제 데이터의 값 y 를 비교한다. 임의의 기울기와 절편을 통해 생성된 선형방정식의 예측값 $\hat{y}$ 은 당연히 실제 데이터의 값과 일치하지 않을 가능성이 크다. 이 예측값과 실제 값의 차이를 **오차**, **잔차**, 또는 **비용(cost)**이라고 하는데, 경사하강법은 계수와 절편을 어느 방향으로 움직이면 이 비용값이 줄어드는지를 찾는 것이다. 그리고 그 방향을 찾고 나면 실제로 계수와 절편으로 그 쪽으로 옮겨 놓는다. 이를 위해서는 비용함수에 대한 편미분이 필요하며 점진적으로 비용을 줄여나가기 위한 **간격(step)** 값을 잘 설정해야 한다.

파이썬의 사이킷런은 이러한 **복잡한 과정을 내부적으로 구현해** 두었으며 우리는 입력 데이터를 잘 준비하는 것으로 충분히 기계학습을 이용할 수 있다.

224

224

LAB¹⁴⁻¹ 키가 비슷해도 남, 여의 몸무게는 다를 것 : 다차원 선형회귀

실습 시간



일반적으로 여자의 몸무게는 남자의 몸무게보다 가볍다. 이러한 점을 보완하여 이 모델의 특징 *feature*에 키와 성(남, 여)을 추가하면 이 모델은 더욱더 성공적일 것이다. 이제 철수의 반에 있는 8명의 학생들에 대하여 키, 몸무게를 성별까지 함께 표기해보자. 이 데이터에 남자는 0, 여자는 1과 같은 값을 부여하였다. 이들의 키와 몸무게는 다음과 같은 데이터로 주어진다.

	학생1	학생2	학생3	학생4	학생5	학생6	학생7	학생8
키	164	167	165	170	179	163	159	166
성별	1	1	0	0	0	1	0	1
몸무게	43	48	47	66	67	50	52	44

- (1) 이제 이 데이터를 바탕으로 선형회귀 모델을 생성하여라. 이 선형회귀 모델의 *coef_*와 *intercept_* 값은 각각 얼마인가? 또한 이 모델의 *score* 값은 얼마인가?
- (2) 다음으로 키가 166cm인 여학생(1)은지와 같은 키의 남학생(0) 동민이의 몸무게를 선형회귀를 사용하여 추정하여라.

원하는 결과

```
계수 : [ 0.88542825 -8.87235818]
절편 : -90.97330367074522
점수 : 0.7404546306026769
은지와 동민이의 추정 몸무게 : [47.13542825 56.00778643]
```



힌트 선형회귀를 위한 입력 데이터 *X*를 `[[164], [167], [165], ...]` 대신, `[[164, 1], [167, 1], [165, 0], ...]`과 같은 2차원 데이터로 만들도록 하자. 이 입력값을 앞서 호출한 `regr.fit(X, y)`에 넣어 주도록 하자. 이 `regr` 모델의 *coef_*와 *intercept_*, *score*를 출력해 보자.

225

225

LAB¹⁴⁻¹ 키가 비슷해도 남, 여의 몸무게는 다를 것 : 다차원 선형회귀

해답 코드



```
import numpy as np
from sklearn import linear_model

regr = linear_model.LinearRegression()

# 남자는 0, 여자는 1을 넣어 차원을 추가하였음
# 입력데이터를 2차원으로 만들어야 함
X = [[164, 1], [167, 1], [165, 0], [170, 0], [179, 0], [163, 1], [159, 0], [166, 1]]
y = [43, 48, 47, 66, 67, 50, 52, 44] # y 값은 1차원 데이터
regr.fit(X, y) # 학습
print('계수 :', regr.coef_)
print('절편 :', regr.intercept_)
print('점수 :', regr.score(X, y))
print('은지와 동민이의 추정 몸무게 :', regr.predict([[166, 1], [166, 0]]))
```

226

226

LAB¹⁴⁻² 주택의 실면적과 대중교통 접근성 그리고 가격

실습 시간



일반적으로 주택의 실면적이 크고 대중교통과의 접근성이 좋을 경우 거래가격은 높은 편이다. 다음과 같이 비슷한 지역의 10개 주택 샘플이 주어졌을 경우에 대하여 선형회귀 모델을 만들고 주택 거래 가격을 예측해 보도록 하자. 주택의 면적은 제곱미터 단위이며, 버스나 지하철과 같은 대중교통과의 접근성은 1에서 10까지로 표시하였다. 이 때 숫자가 높을수록 접근성이 더 좋은 편이다. 그리고 주택의 거래 가격 단위는 억 단위이다.

	주택1	주택2	주택3	주택4	주택5	주택6	주택7	주택8	주택9	주택10
실면적	85	76	50	44	57	88	78	97	45	76
접근성	6	5	4	3	4	5	7	7	3	6
거래 가격	8.9	7.7	3.1	1.8	6.7	9.5	8.4	12.2	2.3	8.5

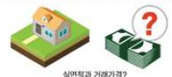
- (1) 이제 이 데이터를 바탕으로 선형회귀 모델을 생성하여라. 이 선형회귀 모델의 coef_와 intercept_ 값은 각각 얼마인가? 또한 이 모델의 score 값은 얼마인가?
- (2) 실면적이 65m²이고 접근성 점수가 5인 주택의 예상 거래 가격을 선형회귀를 사용하여 추정하여라.
- (3) 실면적이 75m²이고 접근성 점수가 5인 주택의 예상 거래 가격을 선형회귀를 사용하여 추정하여라.

원하는 결과

```
계수 : [0.15613505 0.2656739 ]
절편 : -5.285369201207788
점수 : 0.9455485534480466
실면적 65 m^2, 접근성 점수 5 : [6.19177875]
실면적 75 m^2, 접근성 점수 5 : [7.75312929]
```

힌트

선형회귀를 위한 입력 데이터 X를 [[85, 6], [76, 5], ..., [76, 5], ...]과 같은 2차원 데이터로 만들도록 하자. 이 모델의 출력값이 되는 y는 거래 가격인 [8.9, 7.7, 3.1, ...]으로 두도록 하자. 그리고 이 값들을 regr.fit(X, y)에 넣어 주도록 하자. 이 regr 모델의 coef_와 intercept_, score를 출력해 보자. 그리고 실면적과 접근성 점수를 넣어 가격을 예측해보자.



실면적과 거래가격?

227

227

LAB¹⁴⁻² 주택의 실면적과 대중교통 접근성 그리고 가격

해답 코드



```
import numpy as np
from sklearn import linear_model

regr = linear_model.LinearRegression()
# 입력데이터를 다음과 같이 2차원으로 만들어야 함
X = [[85, 6],[76, 5],[50, 4],[44, 3],[57, 4],[88, 5],[78, 7],[97, 7],\
     [45, 3], [76, 6]]
y = [8.9, 7.7, 3.1, 1.8, 6.7, 9.5, 8.4, 12.2, 2.3, 8.5] # y 값은 1차원 데이터
regr.fit(X, y) # 학습
print('계수 :', regr.coef_)
print('절편 :', regr.intercept_)
print('점수 :', regr.score(X, y))
print('실면적 65 m^2, 접근성 점수 5 :', regr.predict([[65, 5]]))
print('실면적 75 m^2, 접근성 점수 5 :', regr.predict([[75, 5]]))
```

228

228

사이킷런의 당뇨병 예제와 학습 데이터 생성

- sklearn 라이브러리에는 **당뇨병**diabetes 환자의 데이터가 기본적으로 포함되어 있다.

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn import datasets

# 당뇨병 데이터 세트를 sklearn의 데이터집합으로부터 읽어들인다.
diabetes = datasets.load_diabetes()
```



229

229

사이킷런의 당뇨병 예제와 학습 데이터 생성

- 이 데이터는 입력으로 사용되는 data, 그리고 학습 시 결과값으로 사용되는 target, 그리고 입력이 가진 특징들의 이름을 저장하고 있는 feature_names 등이 담겨 있다.
- 데이터 세트의 data 항목의 shape 속성을 통해서 당뇨병 데이터 세트의 형태를 출력해보자.
- 다음과 같이 442×10 형태임을 알 수 있다. 여기서 442는 행의 개수로 모두 442개의 데이터가 있다는 의미이며, 10은 데이터 세트의 특징이 10개가 있다는 의미이다.

```
print('shape of diabetes.data: ', diabetes.data.shape)
print(diabetes.data)

shape of diabetes.data: (442, 10)
[[ 0.03807591  0.05068012  0.06169621 ... -0.00259226  0.01990842
 -0.01764613]
 ...
 [-0.04547248 -0.04464164 -0.0730303 ... -0.03949338 -0.00421986
  0.00306441]]
```

230

230

사이킷런의 당뇨병 예제와 학습 데이터 생성

특성들

```
print('입력데이터의 특성들')
print(diabetes.feature_names)
```

```
입력데이터의 특성들
['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
```

목표값

```
print('target data y:', diabetes.target.shape)
print(diabetes.target)
```

```
target data y: (442,)
[151.  75. 141. 206. 135.  97. 138.  63. 110. 310. 101.  69. 179. 185.
 118. 171. 166. 144.  97. 168.  68.  49.  68. 245. 184. 202. 137.  85.
 ...
 146. 212. 233.  91. 111. 152. 120.  67. 310.  94. 183.  66. 173.  72.
  49.  64.  48. 178. 104. 132. 220.  57.]
```

231

231

사이킷런의 당뇨병 예제와 학습 데이터 생성

- 입력데이터
 - 환자들의 신체적 특성 : 나이(age), 성(sex), 체질량지수(bmi), 혈압(bp),....
 - 당뇨수치에 영향을 줄 수 있는 각종 검사 결과값들로 **스케일된 실수값**
- 출력데이터
 - 기준일로부터 1년 후 당뇨수치의 진행 정도
- 어떠한 특성들이 환자의 당뇨수치 값에 영향을 많이 줄까?
- 아래 가설은 타당할까?

체질량지수가 높은 사람은 당뇨 수치가 높아질 가능성이 있다.

- BMI 항목만을 추출해서 선형회귀의 입력으로 사용해 볼까?

232

232

사이킷런의 당뇨병 예제와 학습 데이터 생성

- 10가지 특징 중에서 체질량지수 BMI에 해당하는 세번째 항목 하나만을 추출

```
X = diabetes.data[:, 2]
print(X) # (442, ) 형태의 행렬이 출력됨 - 선형회귀의 입력데이터로 부적합
```

```
[ 0.06169621 -0.05147406  0.04445121 -0.01159501 -0.03638469 -0.04069594
 -0.04716281 -0.00189471  0.06169621  0.03906215 -0.08380842  0.01750591
 ...
 0.05522933 -0.06009656  0.00133873 -0.02345095 -0.07410811  0.01966154
 -0.01590626 -0.01590626  0.03906215 -0.0730303 ]
```

앞서 배운 regr의 입력으로 이 데이터를 사용하려고 한다.
이 데이터를 2차원 배열로 변환해야만 한다(평균값이 0이 되도록 조정된 값)

- 함수의 입력으로 사용되는 데이터는 2차원 배열이어야 한다

```
X = diabetes.data[:, np.newaxis, 2] # 배열의 차원을 증가시킴
print(X) # (442, 1) 형태의 행렬이 되었는가 확인
```

```
[[ 0.06169621]
 [-0.05147406]
 ...
 [ 0.03906215]
 [-0.0730303 ]]
```

np.newaxis를 이용하여 배열의 차원을 증가시키자
이제 이 데이터 X를 선형회귀 모델의 입력으로 사용할 수 있다.

233

233

사이킷런의 당뇨병 예제와 학습 데이터 생성



잠깐 - newaxis를 이용한 배열의 차원 증가 방법

넘파이의 newaxis 특성은 현재 배열의 차원을 1 증가시키는 역할을 한다. 예를 들어 1차원 배열에 적용하면 2차원 배열이 된다. A = np.array([1, 2, 3])의 배열을 생성하고 형태를 보면 다음과 같다.

```
>>> A = np.array([1, 2, 3])
>>> A.shape
(3,)
>>> B = A[:, np.newaxis]
>>> B.shape
(3, 1)
>>> C = A[np.newaxis, :]
>>> C.shape
(1, 3)
```

A = [1, 2, 3]

B = [[1],
[2],
[3]]

C = [[1, 2, 3]]

234

234

체질량지수와 당뇨수치는 어떤 상관관계가 있을까

- 입력 데이터에서 특징으로 사용할 것만을 X로 추출하였다.
- 이를 입력으로 하고 diabetes.target에 담긴 배열을 출력으로 하는 함수를 찾기 위한 선형회귀 학습은 간단히 다음과 같이 진행하면 될 것이다.

```

▶ regr.fit(X, diabetes.target) # 학습을 통한 선형회귀 모델을 생성
  print(regr.coef_, regr.intercept_)

[949.43526038] 152.1334841628967
    
```

- 이 모델을 적용할 새로운 데이터가 없다?
- 원래 데이터를 몽땅 학습용으로 사용하였기 때문이다.
- 학습(또는 훈련)을 위해서 전체 442개 중에서 80%만을 사용
- 나머지 20%를 테스트에 사용

235

235

체질량지수와 당뇨수치는 어떤 상관관계가 있을까

```

▶ # 학습 데이터와 테스트 데이터를 분리한다.
  from sklearn.model_selection import train_test_split
  X_train, X_test, y_train, y_test = train_test_split(diabetes.data, diabetes.target,
                                                    test_size = 0.2)
    
```

```

▶ # 학습 데이터와 테스트 데이터를 분리한다.
  from sklearn.model_selection import train_test_split
  X_train, X_test, y_train, y_test = train_test_split(diabetes.data[:, np.newaxis, 2],
                                                    diabetes.target,
                                                    test_size = 0.2)

  regr = LinearRegression()
  regr.fit(X_train, y_train)
    
```

x_train과 y_train과 같은 학습용 데이터를 이용하여 선형 회귀 모델로 학습을 수행 BMI 데이터만 사용함

```

▶ score = regr.score(X_train, y_train) # 학습용 데이터의 적합 점수
  print(score)
  score = regr.score(X_test, y_test) # 테스트용 데이터의 적합 점수
  print(score)
    
```

```

0.35142918496398023
0.31664029149785233
    
```

훈련 데이터, 테스트 데이터 모두 35, 31점 정도의 점수를 보여주고 있다.

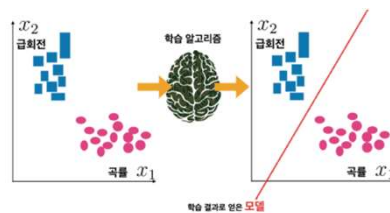
236

236

체질량지수와 당뇨수치는 어떤 상관관계가 있을까

학습 데이터와 테스트 데이터를 나누자

- 입력의 형태를 "원"과 "사각형"으로 분류하는 기계학습 시스템을 생각해보자.
- 이 입력 객체들은 모양도 다르고 색상도 다르다고 가정하자.
- 우리는 다음과 같이 "원"과 "사각형"이라는 정답 레이블(label)이 붙어 있는 학습용 데이터로 시스템을 학습시킬 수 있으며, 학습 알고리즘은 입력 데이터의 특징에 따라 입력을 "원"과 "사각형"으로 분류할 수 있는 모델을 내부적으로 생성한다.
- 여기에서는 객체의 곡률과 급회전 구간의 여부를 특징으로 하였다.

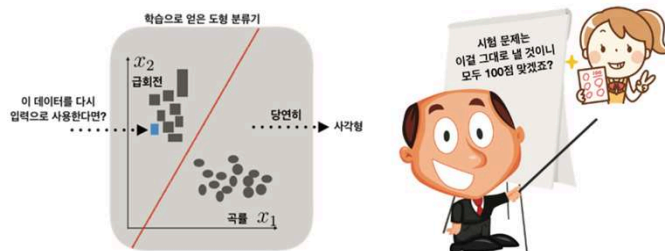


237

237

체질량지수와 당뇨수치는 어떤 상관관계가 있을까

학습 데이터와 테스트 데이터를 나누자 (계속)



- 이전에 입력한 데이터가 들어온다면 학습 알고리즘은 이 데이터를 이미 다루었기 때문에 **당연히 좋은 결과를 얻을 것**
- 한 번도 본 적이 없는 '새로운 데이터'로 시스템을 테스트하는 것이 바람직할 것이다.

238

238

당뇨병 예제를 학습 데이터와 테스트 데이터로 구분하자

```
print(y_pred)
print(y_test)
```

[238.47145247 248.93170646 164.05404165 120.30794355 187.42422054
...
94.94046865 145.2955051 194.03776373 132.78534336]
[321. 215. 127. 64. 175. 275. 179. 232. 142. 99. 252. 174. 129. 74.
...
104. 49. 103. 142. 59.]

예측값
실제값

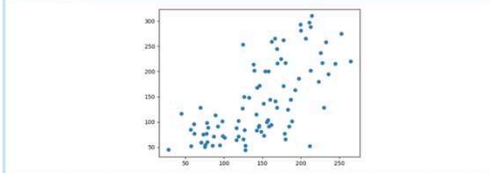
241

241

LAB¹⁴⁻³ 데이터 80%로 학습하여 예측한 결과와 실제 데이터 비교

실습 시간
사이킷런 모듈에서 제공하는 데이터의 80%를 학습용으로 사용하고, 이를 이용하여 테스트용 입력 20%에 대해 당뇨병수치를 예측할 수 있다. 이렇게 예측한 결과와 실제 당뇨병 수치를 시각적으로 비교하여 보자.
2차원 공간에 한 축은 예측한 당뇨병 수치, 다른 한 축은 실제 당뇨병 수치로 하여 (예측값, 실제값)의 점을 아래와 같이 산포도 그래프로 그리면 될 것이다.

원하는 결과



힌트
지금까지 학습한 내용을 바탕으로 예측치를 구한 뒤에 이를 실측치와 쌍을 이뤄 산포도 그래프를 그리면 된다.
학습 데이터와 테스트 데이터를 분리하는 것은 아래와 같다.
`X_train, X_test, y_train, y_test = train_test_split(...)`
테스트 입력 데이터를 넣고 이에 해당하는 예측치 배열을 얻는 방법은 다음과 같다.
`y_pred = regr.predict(X_test)`

242

242

LAB¹⁴⁻³ 데이터 80%로 학습하여 예측한 결과와 실제 데이터 비교

해답 코드



```
import numpy as np
from sklearn import linear_model # scikit-learn 모듈을 가져온다
from sklearn import datasets
import matplotlib.pyplot as plt

diabetes = datasets.load_diabetes()
regr = linear_model.LinearRegression()
# 학습 데이터와 테스트 데이터를 분리한다.
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(diabetes.data,
                                                    diabetes.target,
                                                    test_size=0.2)

regr.fit(X_train, y_train)
print(regr.coef_, regr.intercept_)

y_pred = regr.predict(X_test)

plt.scatter(y_pred, y_test)
plt.show()
```

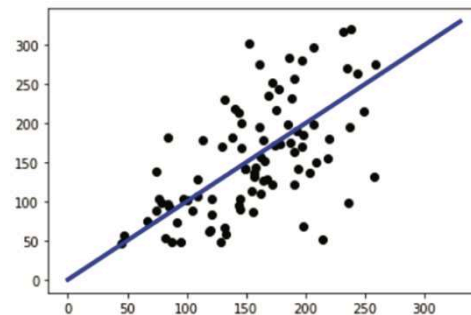
243

243

알고리즘이 가지는 오차

- 오차로 인해서 y_{pred} 가 추정된 값이 실제값 y_{test} 와 차이가 발생

```
plt.scatter(y_pred, y_test, color='black')
x = np.linspace(0, 330, 100) # 특정 구간의 점
plt.plot(x, x, linewidth=3, color='blue')
plt.show()
```



244

244

알고리즘이 가지는 오차

```

from sklearn.metrics import mean_squared_error

... # 이전 절에서 구한 선형회귀 모델의 코드를 삽입함

print('Mean squared error:', mean_squared_error(y_test, y_pred))

Mean squared error: 3424.316688213733
    
```

오차를 얻기 위해서는 다음과 같은 함수를 import문에 넣어야 함

평균제곱오차를 구하는 함수

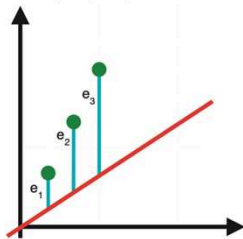
- 평균제곱오차(Mean Squared Error:MSE)는 오차의 척도 중 하나이다.
- 단순한 오차의 합은 오차값의 부호(+,-)에 영향을 받을 수 있기 때문에 오차의 척도로 부적합하다.

245

245

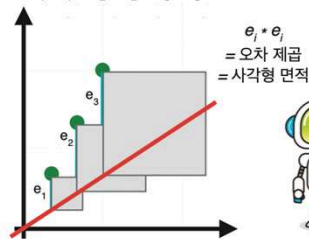
알고리즘이 가지는 오차

① 오차값의 평균 : MAE
 $(e_1 + e_2 + e_3) / 3$



VS

② 음영 상자 면적의 평균 : MSE
 $(e_1^2 + e_2^2 + e_3^2) / 3$



오차값의 평균(MAE)으로 전체 오차를 측정할 수도 있고, 오차 제곱값의 평균(MSE)으로 오차를 측정할 수 있습니다. MSE의 경우 오차의 제곱을 통해 예측이 잘못된 경우 벌칙을 최대화하는 데 더 유리합니다.

$$MSE := \frac{1}{N} \sum_{i=1}^N (H(X_i) - y_i)^2$$

246

246

내용 출처 :

